SVEUČILIŠTE J.J. STROSSMAYER U OSIJEKU

FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKH TEHNOLOGIJA OSIJEK

Sveučilišni studij

UGRADBENI RAČUNALNI SUSTAVI

Profesor/mentor: **prof. dr. sc., Tomislav Keser**

TEHNIČKA DOKUMENTACIJA PROJEKTA

PEUGEOT INFOTAINMENT SUSTAV

Mislav Milinković, Ivan Brajinović

Akademska godina 2024/2025

Osijek, 19.09.2025.

Sadržaj

1 Uvod

U današnje vrijeme infotainment sustavi predstavljaju ključan dio korisničkog iskustva u vozilima, jer omogućuju vozaču i putnicima pristup informacijama i multimediji na intuitivan i siguran način. Rješenja poput Android Auto, Apple CarPlay te tvorničkih rješenja renomiranih proizvođača automobila (npr. BMW iDrive, VW MIB, Tesla infotainment) postala su industrijski standard. Njihova široka rasprostranjenost proizlazi iz sposobnosti integracije funkcija poput navigacije, hands-free poziva, upravljanja glazbom i sustava vozila, čime značajno doprinose sigurnosti i udobnosti vožnje.

Ono što ta rješenja čini korisnima jest kombinacija modernog korisničkog sučelja i snažne povezivosti s vozilom putem komunikacijskih sabirnica poput CAN-a ili FlexRaya. Standardizirani protokoli i bogata dokumentacija omogućuju proizvođačima jednostavnu implementaciju i brži razvoj. Većina sustava oslanja se na dobro dokumentirane mrežne protokole, brze procesore i modularne arhitekture koje olakšavaju nadogradnju i prilagodbu softvera. Međutim, upravo zbog toga rješenja su često zatvorenog tipa i teško ih je prilagoditi specifičnim starijim modelima automobila, osobito onima koji koriste zastarjele komunikacijske standarde.

Kod starijih vozila, poput Peugeot 206, komunikacija se odvija preko VAN sabirnice koja je specifična za PSA grupu i slabije dokumentirana u odnosu na CAN. To predstavlja izazov za razvoj vlastitog infotainment sustava, jer je potrebno implementirati dekodiranje i generiranje VAN okvira kako bi se uspostavila dvosmjerna komunikacija. Na tržištu gotovo da i nema komercijalno dostupnih modula koji to podržavaju, što motivira razvoj prilagođenih rješenja.

Motiv za izradu ovog projekta leži upravo u potrebi za modernizacijom starijeg vozila i dodavanjem funkcionalnosti koje se nalaze u novijim automobilima – prikaz telemetrije u stvarnom vremenu, multimedijska reprodukcija, informiranje o statusu vozila – uz zadržavanje potpune kompatibilnosti s postojećom elektronikom. Rješenje koje je ovdje predstavljeno ima za cilj stvoriti modularni i proširivi infotainment sustav temeljen na modernom mikroupravljaču i otvorenoj softverskoj platformi, čime se korisniku omogućuje funkcionalnost na razini novijih vozila, ali po pristupačnoj cijeni i uz potpunu kontrolu nad sustavom.

1.1 Zadatak i struktura rada

Zadatak ovog projekta bio je razviti i implementirati infotainment sustav za vozilo Peugeot 206, s naglaskom na kompatibilnost s postojećom elektroničkom infrastrukturom vozila. Glavni izazov je komunikacija putem VAN sabirnice, koja je specifična za PSA grupu i slabo dokumentirana, što zahtijeva izradu vlastitog hardverskog i softverskog rješenja za dekodiranje i generiranje poruka.

Cilj projekta je izraditi potpuno funkcionalan prototip koji omogućuje:

- prikaz telemetrijskih podataka vozila u stvarnom vremenu (brzina, okretaji motora, temperatura, status goriva),
- integraciju multimedijskih funkcija poput prikaza radijskih stanica i reprodukcije sadržaja s vanjskih uređaja,
- stabilan i energetski učinkovit rad koji ne opterećuje akumulator vozila,
- modularnost i mogućnost budućeg proširenja funkcionalnosti.

Očekivani rezultat projekta je pouzdan, proširiv i cjenovno pristupačan infotainment sustav temeljen na modernim mikroupravljačima i otvorenim softverskim rješenjima, koji omogućuje starijem vozilu funkcionalnosti usporedive s novijim modelima.

1.1.1 Struktura rada

Poglavlje 1 – Uvod Daje pregled motivacije za projekt, opisuje specifičnosti Peugeot 206 vozila i problematiku komunikacije putem VAN sabirnice. Objašnjava zašto je potrebno izraditi vlastiti modul i što se planira postići projektom.

Poglavlje 2 – Peugeot Infotainment sustav Objašnjava teorijsku osnovu projekta. Detaljno opisuje principe rada VAN sabirnice, e-Manchester kodiranje, strukturu poruka, te način na koji su moduli u vozilu međusobno povezani. Na kraju se daje prijedlog sklopovskog i programskog rješenja.

Poglavlje 3 – Realizacija Peugeot Infotainment sustava Sadrži konkretan opis realizacije: korištene komponente, razvoj tiskane pločice, sheme i logiku povezivanja. Objašnjava kako je izvedena programska podrška – od primanja i slanja VAN okvira do upravljanja napajanjem i događajima.

Poglavlje 4 – Testiranje i rezultati Prikazuje metodologiju testiranja sustava u stvarnim uvjetima (u vozilu) te rezultate testova. Opisuje ispravnost prikaza telemetrije, rad multimedije i potvrđuje da sustav radi u skladu s očekivanjima.

Poglavlje 5 – Programska podrška infotainment računala Ukratko opisuje softver na Raspberry Pi računalu: korištenu Linux distribuciju, grafičko sučelje (Qt) i glavne funkcionalnosti koje korisnik vidi na ekranu.

Poglavlje 6 – Zaključak i budući planovi Sumira naučene lekcije, ističe glavne prednosti i izazove projekta te daje prijedloge za buduća poboljšanja (npr. integracija s tvorničkim zaslonom, dodatna optimizacija potrošnje energije).

Literatura Navodi korištene izvore, datasheetove, tehničku dokumentaciju i prethodne radove na kojima se projekt temelji.

2 Peugeot Infotainment sustav

2.1 Principi i fizikalne zakonitosti

VAN (eng. *Vehicle Area Network*) je serijska komunikacijska sabirnica koju su razvili PSA grupa (Peugeot, Citroën) i Renault kao alternativu CAN sabirnici. Definirana je u ISO standardu **ISO 11519-3**. Koristi isti fizički sloj kao CAN sabirnica (diferencijalni par) i identičnu metodu arbitraže na sabirnici, ali se razlikuje u načinu pakiranja podataka. Gotovo sve informacije navedene ovdje preuzete su iz Atmel TSS463C dokumentacije [3].

Kao i CAN, VAN protokol koristi identifikatore za razlikovanje poruka i izvođenje arbitraže na sabirnici. Dodatno, omogućuje tzv. „odgovore unutar okvira" (eng. *In-frame*), veće veličine poruka (28 bajtova umjesto 8 bajtova kao u CAN-u) te mogućnost izravne komunikacije s drugim uređajem.

Podaci na VAN mreži pakiraju se u okvire. Okviri se sastoje od više dijelova: početka okvira (**SOF**), identifikatora poruke (**IDEN**), naredbe (**COM**), samih podataka (**DATA**), kontrolnog zbroja okvira (**FCS**), signala kraja podataka (**EOD**), signala potvrde (**ACK**) te signala kraja okvira (**EOF**).

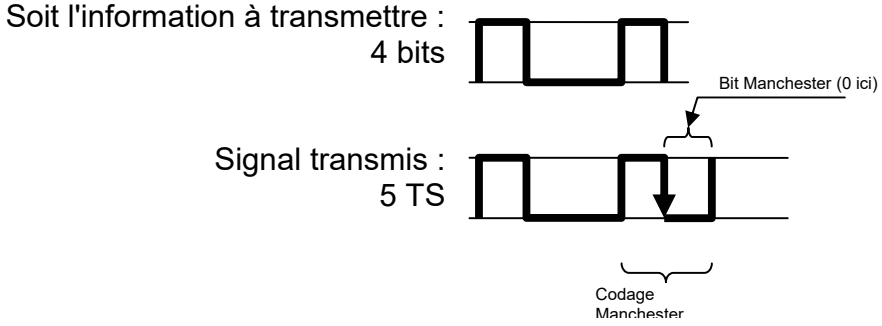
SOF	IDEN	Command (4 bit, 5 TS)				DATA max.	FCS	EOD	ACK	EOF
10 TS	12 bit					28 byte	15 bit	2 TS	2 TS	4 TS
	15 TS	EXT	RAK	R/W	RTR	280 TS	18 TS			

Tablica 2.1: Oblik VAN okvira

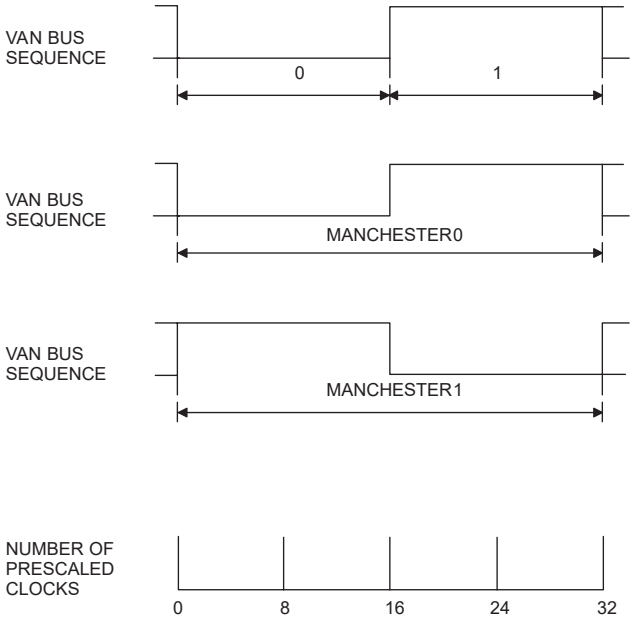
2.1.1 e-Manchester kodiranje

Svi VAN okviri kodirani s pomoću unaprijeđenog Manchesterovog kodiranja (eng. *Enhanced Manchester, e-Manchester*). Kod e-Manchestera, nakon slanja tri NRZ bita, četvrti bit šalje se kao Manchesterov bit. Slanje podataka na ovaj način omogućuje lakšu sinkronizaciju na sabirnici. Ovo je predvidljivija metoda ubacivanja bitova (eng. *bit-stuffing*) u usporedbi s metodom korištenom na CAN sabirnici.

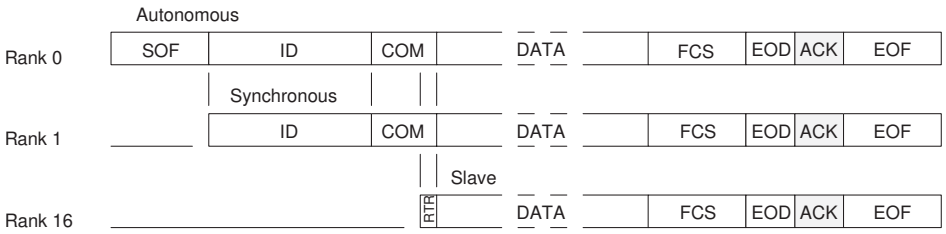
U Tablici 2.1 prikazan je broj bitova za svaki dio okvira, kao i broj vremenskih intervala (eng. *Time Slots*, TS). Budući da Manchester bit zahtjeva dva vremenska intervala za prijenos, četiri bita u nizu kodirana e-Manchester metodom prenose se u 3 + 2 vremenska intervala.



Slika 2.1: e-Manchester kodiranje [1]



Slika 2.2: NRZ i Manchester bitovi. [3]



Slika 2.3: Vrste modula, [3]

2.1.2 Poruke

Uređaj ili modul povezan na mrežu može biti jednog od tri tipa:

- Autonomni modul - glavni uređaj sabirnice. Može slati **SOF** niz, započeti prijenos podataka i primati poruke.
- Modul sa sinkronim pristupom - ne može slati **SOF** niz, ali može započeti prijenos podataka i primati poruke.
- Podređeni modul (eng. *Slave*) - može slati poruke samo s pomoću odgovora unutar okvira (eng. *in-frame*) i također može primati poruke.

VAN protokol podržava više tipova poruka. Razlikuju se s pomoću polja identifikatora (**IDEN**) i naredbe (**COM**). Identifikator je dug 12 bita i slijedi ga 4 bita naredbe. Zbog načina na koji funkcionira arbitraža na sabirnici, manji identifikator ima prednost na sabirnici. Dok je vrijednost identifikatora potpuno proizvoljna, bitovi naredbe su strogo definirani i određuju tip poruke koja će se poslati na sabirnici:

- EXT - uvijek 1

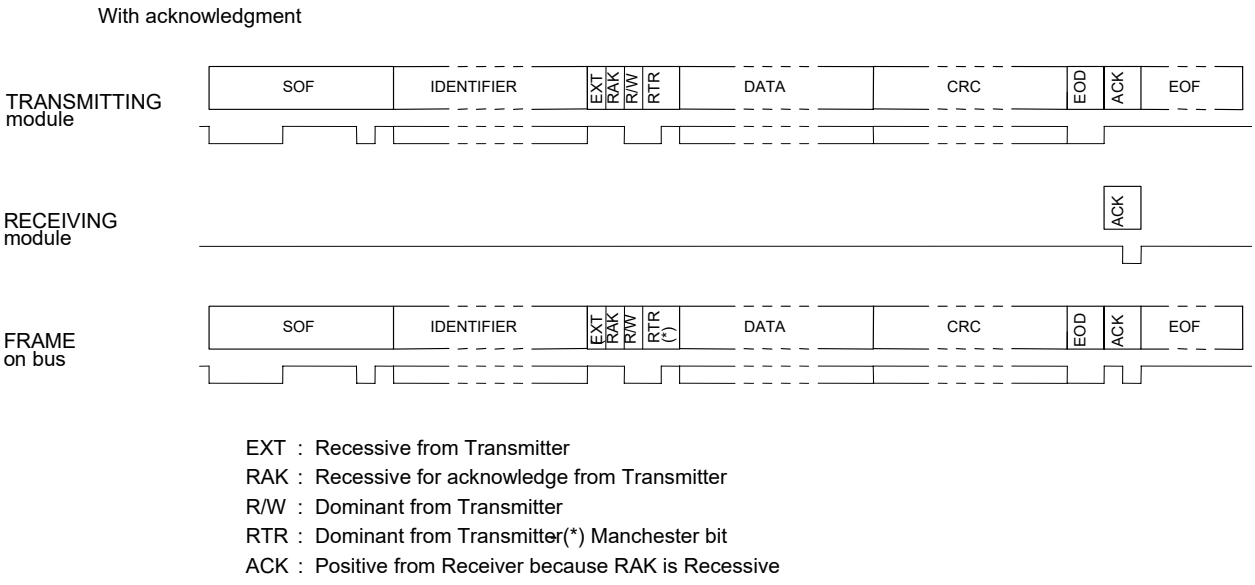
- RAK - '*Request Acknowledge*' (zahtjev za potvrdom). Ako je 1, primatelj mora potvrditi primitak poruke. Ako je 0, signal ACK se ne šalje.
- R/W - '*Read/Write*' (čitanje/pisanje). Označava smjer podataka. Ako je 0, radi se o poruci za pisanje ('*write*'), gdje uređaj šalje podatke namijenjene drugom uređaju. Ako je 1, radi se o poruci za čitanje („*read*"), što podrazumijeva zahtjev za podacima i očekuje odgovor.
- RTR - '*Remote Transmission Request*' (zahtjev za daljinskim prijenosom). Ako je 0, poruka sadrži podatke. Ako je 1, poruka ne sadrži podatke i implicira zahtjev za odgovor unutar okvira (in-frame response). Kombinacija R/W = 0 i RTR = 1 je zabranjena na sabirnici.

2.1.3 Vrste poruka

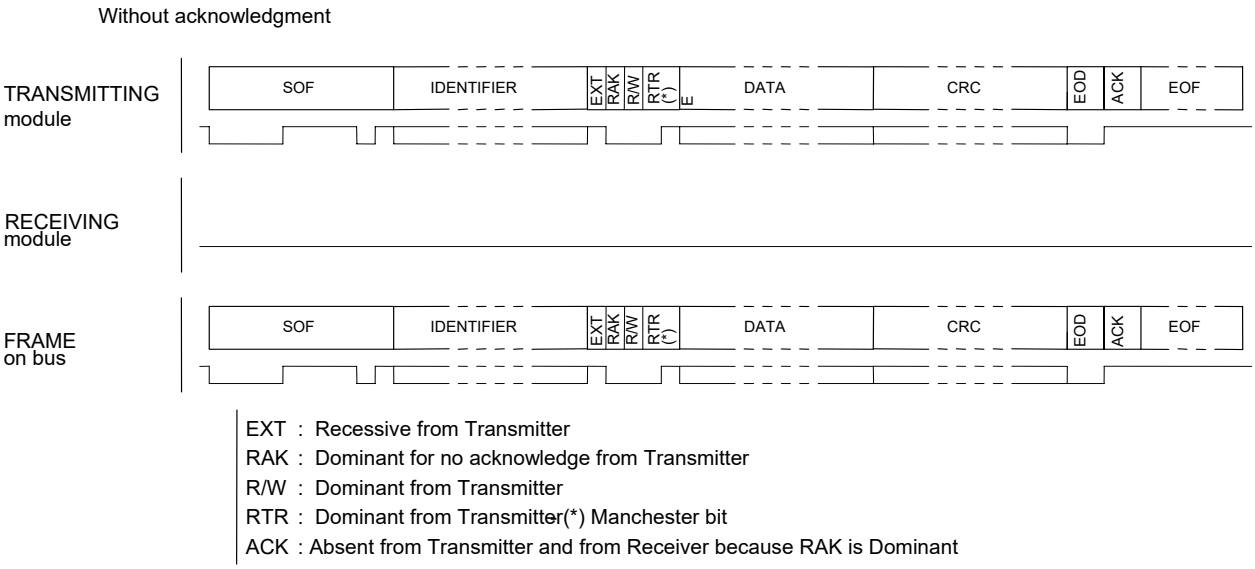
Ovdje ćemo navesti tipove okvira relevantne za ovaj projekt. Postoje i drugi, ali koristimo samo ove.

Podsjetnik: Dominantno = 0, Recesivno = 1.

Normalni okvir Najjednostavniji tip okvira. Jedan modul šalje cijeli okvir s podacima, te može, ali ne mora, zatražiti potvrdu (ACK). U slučaju da se potvrda ne traži, poruka se tretira kao poruka za emitiranje (eng. *broadcast*).

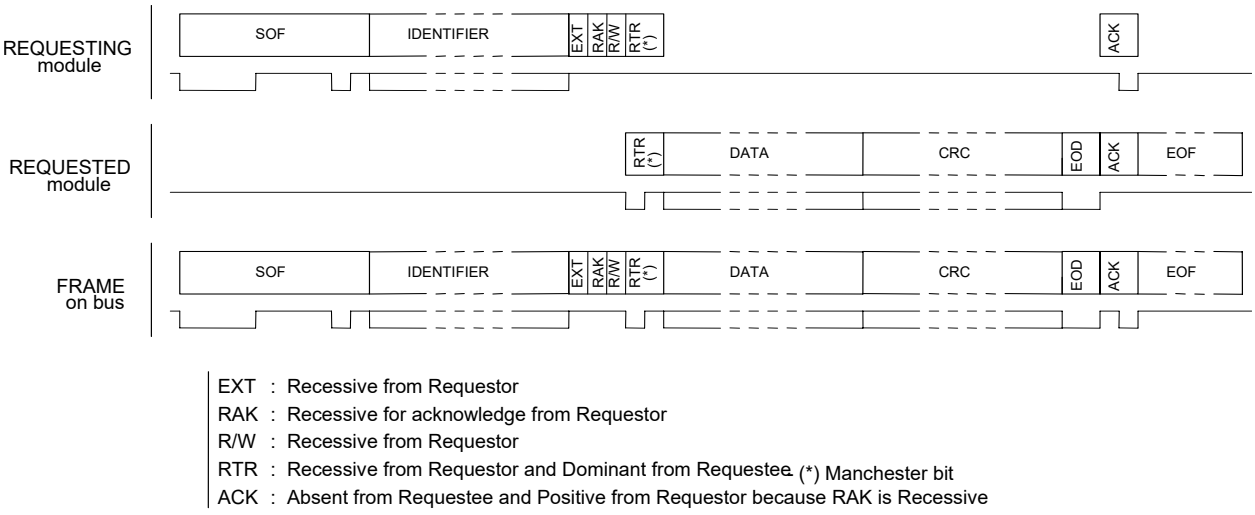


Slika 2.4: Prijenos normalnog okvira s ACK, [3]



Slika 2.5: Prijenos normalnog okvira bez ACK, [3]

Zahtjev s trenutačnim odgovorom Ovo je jedini okvir u kojem podređeni modul (eng. *slave*) može slati podatke. Jedina razlika na sabirnici je promjena stanja bita R/W u odnosu na normalni okvir. Glavni modul sabirnice (eng. *bus master*) generira **SOF**, inicijator generira identifikator i prva tri bita naredbe, te signal ACK. Ostalo (RTR, podaci i kontrolnu sumu) generira modul koji odgovara.

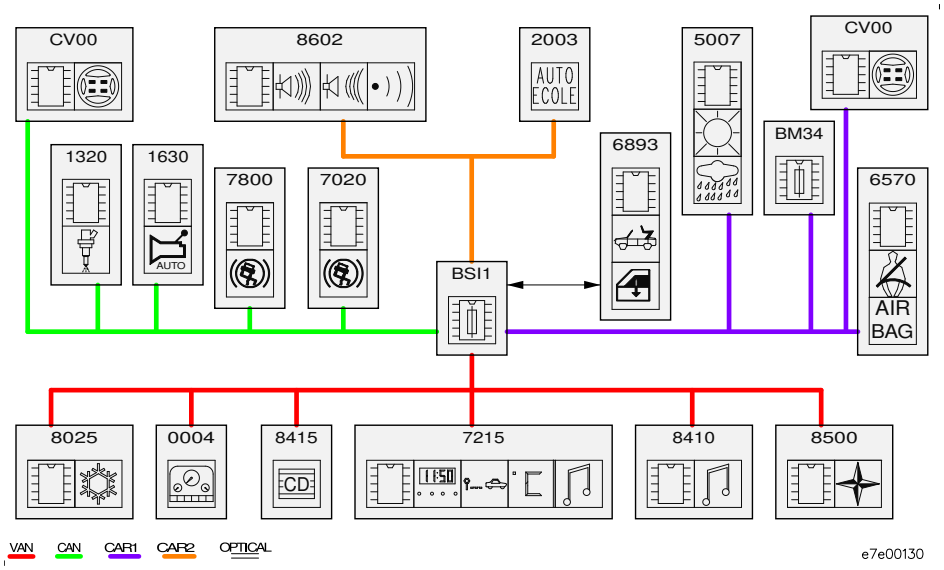


Slika 2.6: Ilustracija prijenosa zahtjeva s trenutačnim odgovorom, [3]

2.1.4 VAN mreža

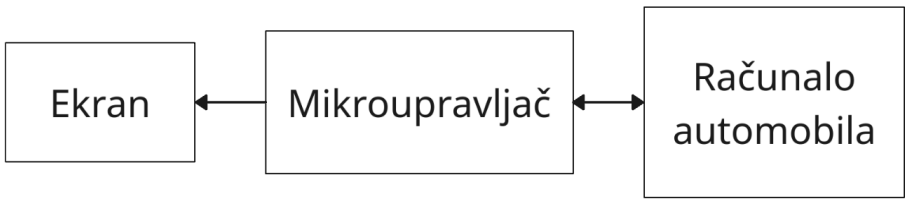
Više modula možemo povezati zajedno kako bismo stvorili mrežu. Također je moguće povezati više mreža i koristiti pristupni modul (eng. *gateway*) za usmjeravanje podataka između mreža. U vozilu Peugeot 206 imamo jednu CAN mrežu *CAN Engine* i tri VAN mreže: *VAN Comfort*, *VAN Body 1* i *VAN Body 2*.
Raspored mreža prikazan je na slici 2.7. Tamo također možemo vidjeti pristupni modul, BSI. Ovaj modul prisutan je u svim vozilima PSA grupe. Konkretni BSI dolazi iz genera-

cije AEE2001, jedine generacije koja koristi VAN sabirnicu. Druge generacije, AEE2004 i AEE2010, koriste isključivo CAN. Naš infotainment sustav bit će spojen umjesto CD mjenjača, modul broj 8415 na slici 2.7 na *VAN Comfort* sabirnici.



Slika 2.7: Pregled VAN i CAN mreže u Peugeot 206 automobilu [2]

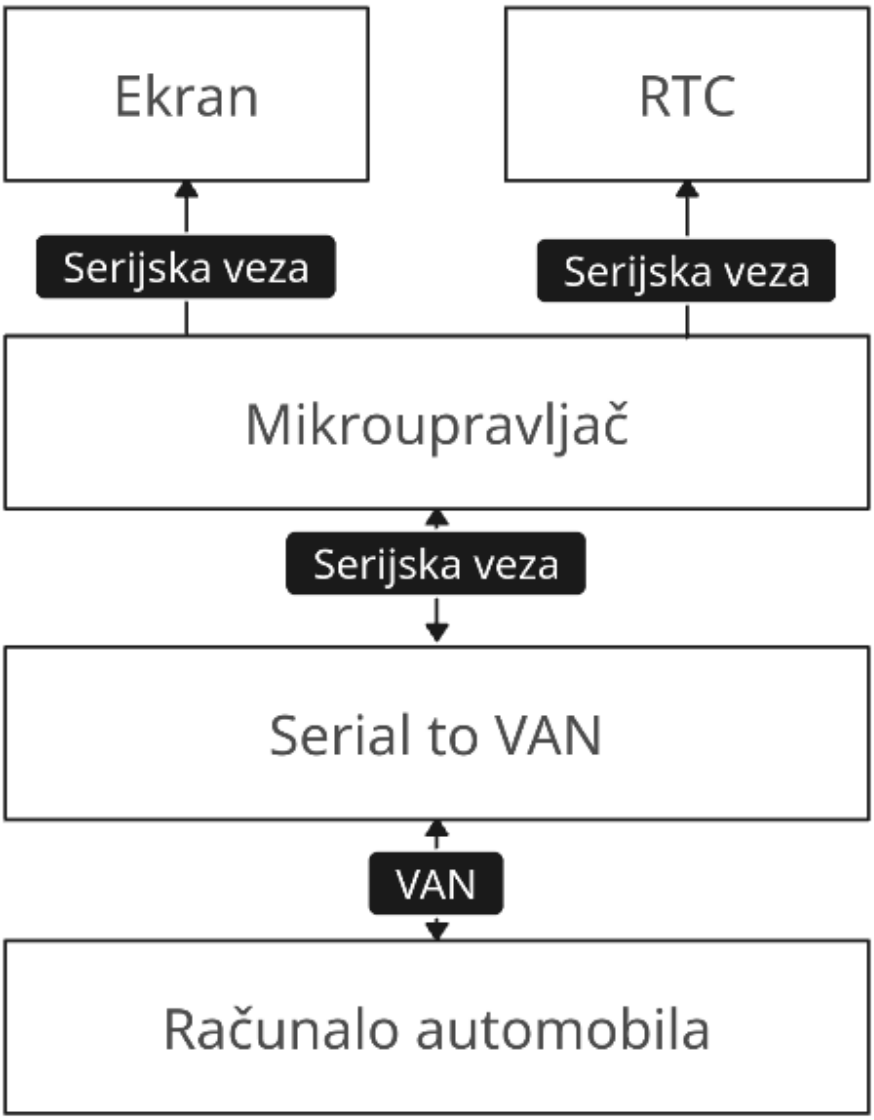
Na slici 2.8 je prikazan block dijagram idejnog rješenja iz kojeg se mogu prepoznati tri dijela, kao i dio koji je centralni za ovaj projekt. Mikroupravljač je zadužen i za komunikaciju s računalom automobila, obradu dobivenih informacija te za njihov prikaz na ekranu.



Slika 2.8: Block dijagram idejnog rješenja

2.2 Pregled sklopovskog rješenja

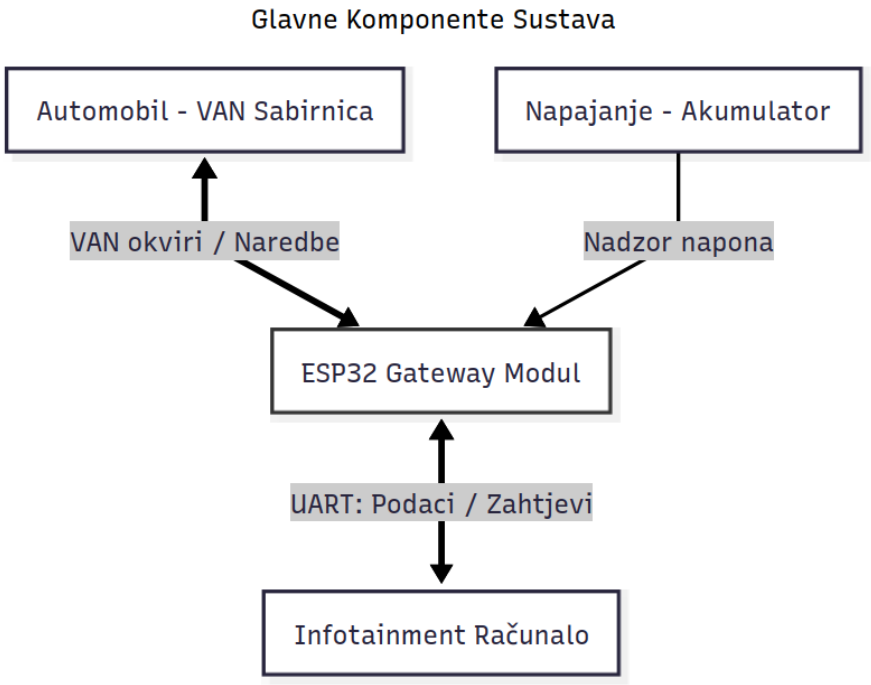
Na slici 2.9 se nalazi prikaz osnovnog sklopovskog dijagrama sustava. Glavni dio sustava je mikroupravljač koji ima ulogu komunikacije informacija s računalom automobila te obradu i prikaz tih istih informacija na ekranu. Kako mikroupravljač ne podržava VAN protokol, potrebno je koristiti upravljačke integrirane krugove koji pružaju sučelje s protokolom koji je podržan od strane mikroupravljača. Za slučaj da se ne pronađe upravljač koji ne podržava diferencijalni VAN protokol, bit će potrebno koristiti tzv. VAN Transceiver kako bi preveli TTL logiku u diferencijalni par. Također zbog dodatne funkcionalnosti prikaza vremena na ekranu, potrebno je dodati RTC integrirani krug čija je zadaća praćenje stvarnog vremena te komunikacija istog s mikroupravljačem.



Slika 2.9: Osnovni blok dijagram sklopovskog rješenja

2.3 Prijedlog programskog rješenja

Na slici 2.10 je prikazan blok dijagram programskog rješenja koji daje apstraktan uvid u moguće programsko rješenje. Glavni program bi se vrtio na mikroupravljaču ESP32 koji je odgovaran za slanje i primanje paketa na VAN mreži tj. odgovaran je za komunikaciju s računalom automobila. Nadalje, pored toga je potrebno riješiti problem upravljanja potrošnjom energije i uvjetnog paljenja sustava tj. postavljanjem dijelova sustava u aktivno stanje ili stanje mirovanja. I na kraju je potrebno dobivene informacije obraditi te prikazati na ekranu.



Slika 2.10: Osnovni blok dijagram programskog rješenja

3 Realizacija Peugeot Infotainment sustava

3.1 Korištene komponente, alati i programska podrška

ESP32 - WROOM32E glavni mikroupravljač sustava. Komunicira s računalom automobila, obrađuje zaprimljene informacije te ih prosljeđuje RPi mikroupravljaču.

RPi 4 prihvaća i obrađuje dobivene informacije od ESP mikroupravljača te ih prikazuje na ekranu.

RTC DS3231MZ broji trenutačno vrijeme koje se komunicira s RPi mikroupravljačem u svrhu prikaza istog na ekranu.

TSS463C integrirani strujni krug koji omogućuje jednostavno komuniciranje s VAN mrežom preko SPI sučelja. Apstrahira sam VAN protokol koji je kompleksan te ubrzava razvoj.

MCP2551 je integrirani strujni krug koji pretvara naponske razine logičkih razina VAN sabirnice u diferencijalni signal.

NTS0104 je integrirani strujni krug koji prevodi naponske razine signala.

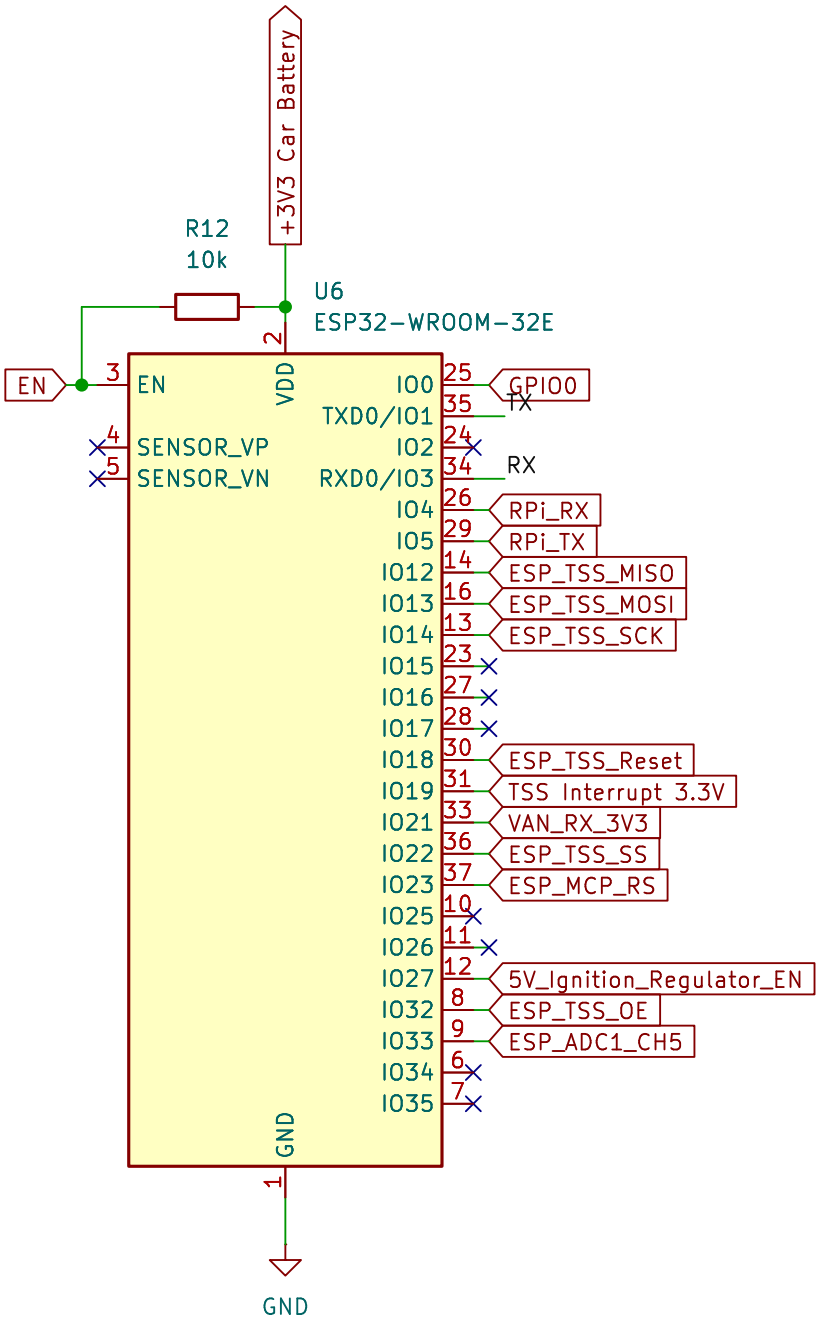
MP8759GD1 je integrirani strujni krug koji služi kao regulator napona. U ovom konkretnom slučaju se koristi za pretvorbu napona od 12 V u 5 V.

NCP691 je integrirani strujni krug koji služi kao regulator napona za 3.3 V.

ESP-IDF programski paket razvijen od Espressif-a se koristi za programiranje za ESP mikroupravljača.

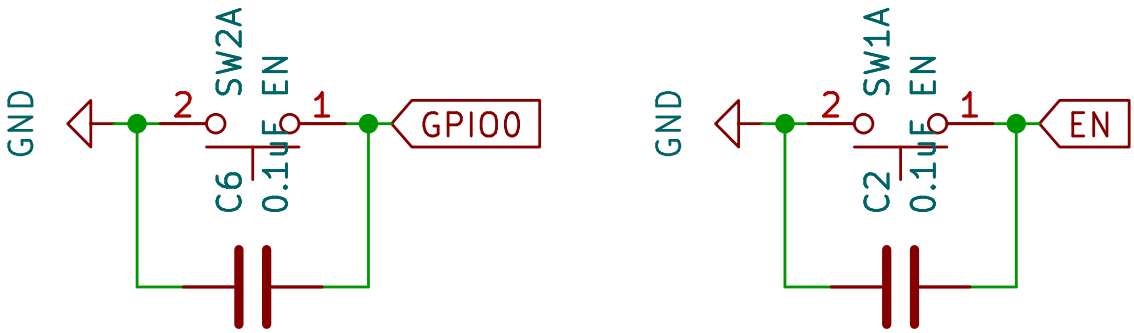
3.2 Realizacija konstrukcijskog i sklopovskog rješenja

Napaja se s 3.3 V sa stalnog izvora napajanja (3.13), EN nožica je spojena preko otpornika na napon napajanja (eng. *pull-up*) kako bi se omogućio rad mikroupravljača. Shema mikroupravljača je prikazana na slici 3.1.



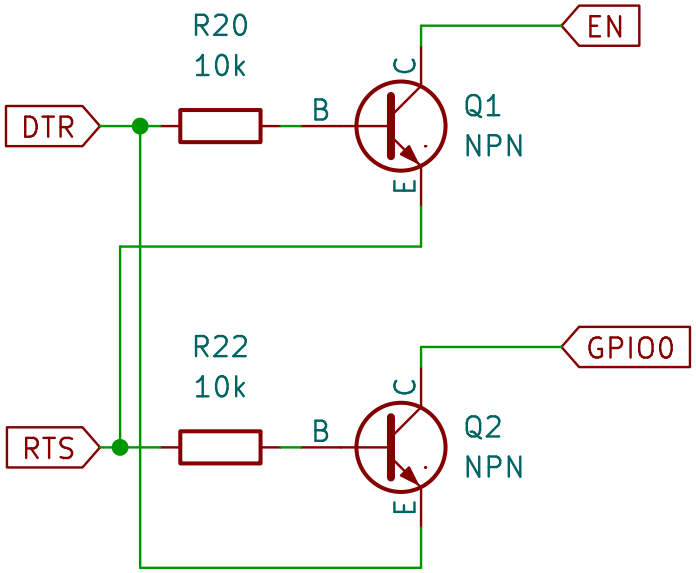
Slika 3.1: Pregled sheme - ESP32 mikroupravljač

Prije samog procesa prijenosa koda na ESP, isti se mora postaviti u određeni način rada gdje je spreman prihvatiti programski kod. Zbog toga se koristi gumb SW2A koji kad je pritisnut dovodi nisku naponsku razinu na GPIO0 što je uvjet kod ponovnog pokretanja mikroupravljača za odlazak u spomenuti način rada. Zbog lakšeg ponovnog pokretanja, implementiran je još jedan gumb za ponovno pokretanje koji, kad se pritisne, dovede nisku naponsku razinu na EN nožicu. Shema gumbova se nalazi na slici 3.2.



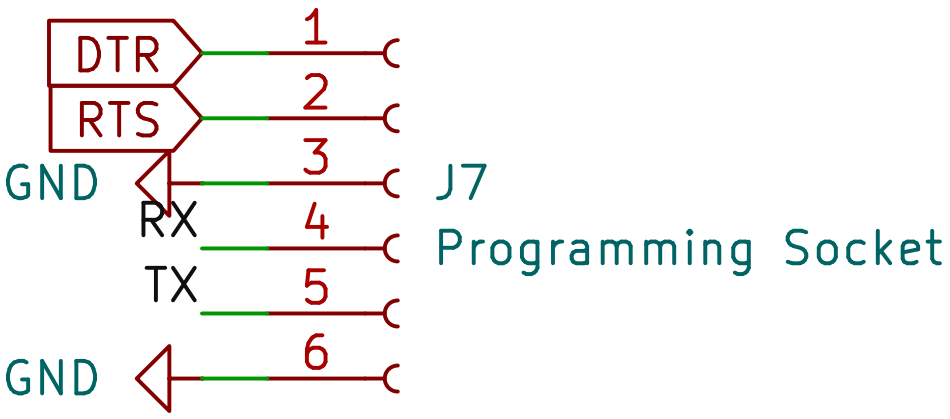
Slika 3.2: Pregled sheme - Gumbovi za programiranje

Dugmad na slici 3.2 se koriste za ručno tj. fizičko programiranje. Međutim, kako bi omogućili programsko programiranje tj. omogućili programiranje mikroupravljača bez fizičkog pritiskanja gumba, implementirana su dva tranzistora koji pružaju programsko upravljanje naponskih razina nožica mikroupravljača. Shema tih tranzistora je prikazana na slici 3.3.



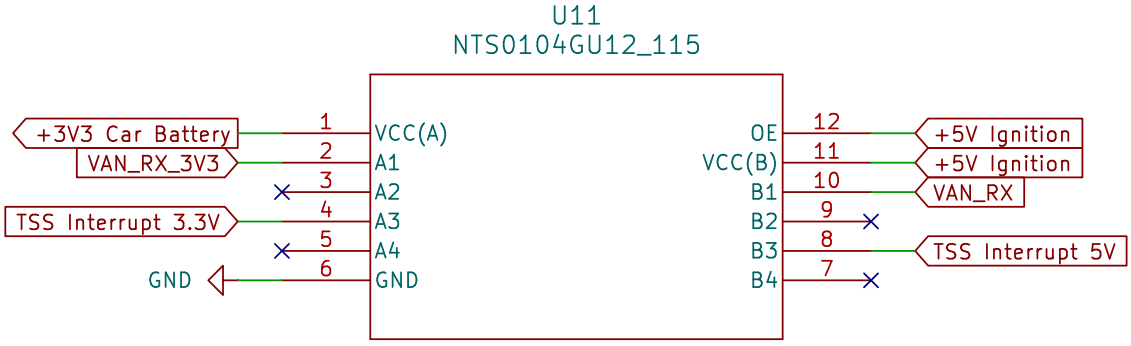
Slika 3.3: Pregled sheme - Tranzistori za programiranje

U svrhe programiranja je kreirano posebno sučelje koje koristi tranzistore za programiranje koji su prethodno spomenuti kako bi odabrali način rada mikroupravljača kao i UART serijski protokol za sam prijenos programskog koda. Shema sučelja je prikazana na slici 3.4.



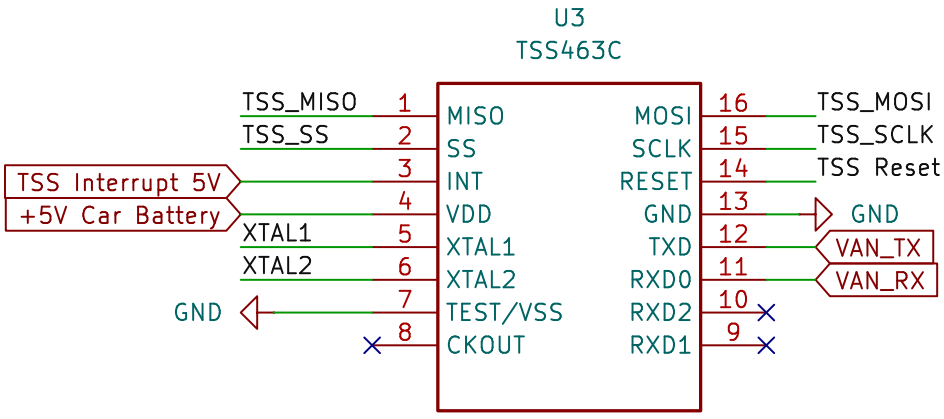
Slika 3.4: Pregled sheme - spojnica za programiranje

Kako imamo komponente koje rade na različitim naponskim razinama, potrebno je upotrijebiti pretvarač naponskih razina kako bi se pouzdano i bez razmišljanja o pregaranju, razmjenjivale informacije između njih. Pretvarač naponskih razina na slici 3.5 vrši pretvorbu na signalima prekida TSS integriranog kruga te na liniji dolaznih podataka VAN sabirnice.



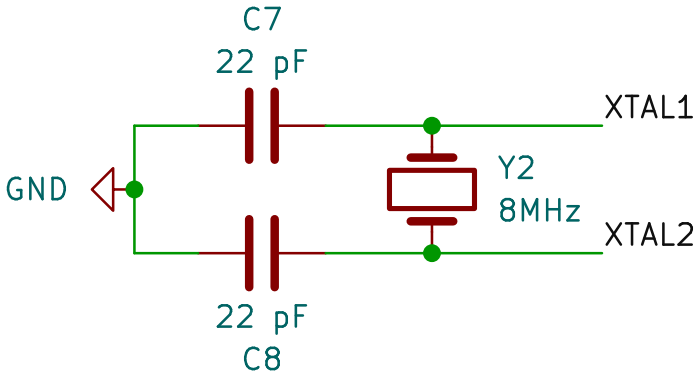
Slika 3.5: Pregled sheme - pretvarač naponskih razina A

TSS463C je VAN upravljački integrirani krug koji se koristi za komunikaciju sa sustavom automobila. Komunicira s ESP mikroupravljačem putem SPI serijskog protokola. Od periferije sadržava 8 MHz oscilator koji je neophodan za rad integriranog kruga. Sa sustavom automobila komunicira preko VAN protokola (nožice 11 i 12). Shema spoja je prikazana na slici 3.6.



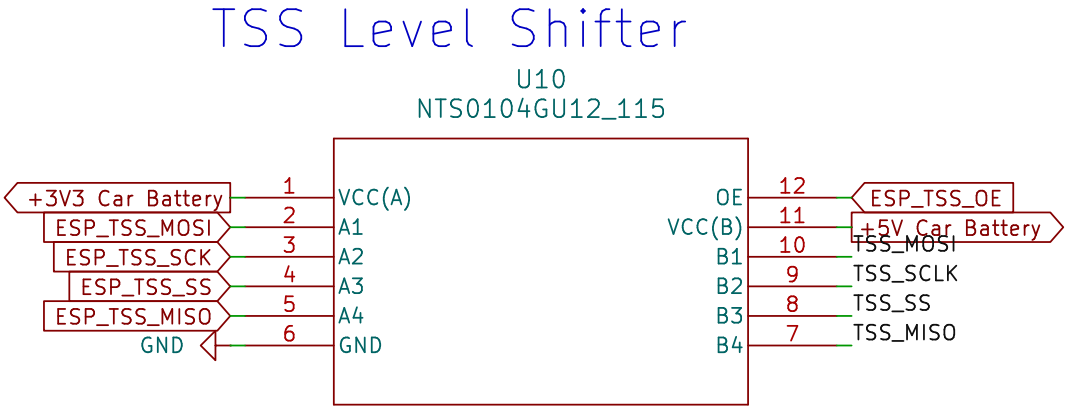
Slika 3.6: Pregled sheme - TSS

Slika 3.7 pokazuje oscilator koji je spojen na TSS463C integrirani krug.



Slika 3.7: Pregled sheme - TSS oscilator

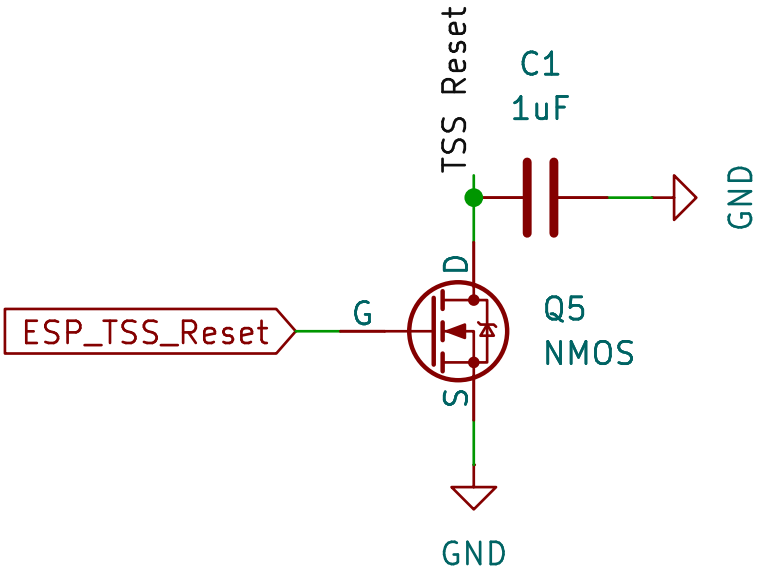
Kako TSS ne može direktno komunicirati s mikroupravljačem, između njih je postavljen pretvarač naponskih razina kako bi preveo 5 V signale od TSS-a u 3.3 V signale koje ESP očekuje. Shema spoja je prikazana na slici 3.8.



Slika 3.8: Pregled sheme - Pretvarač naponskih razina B

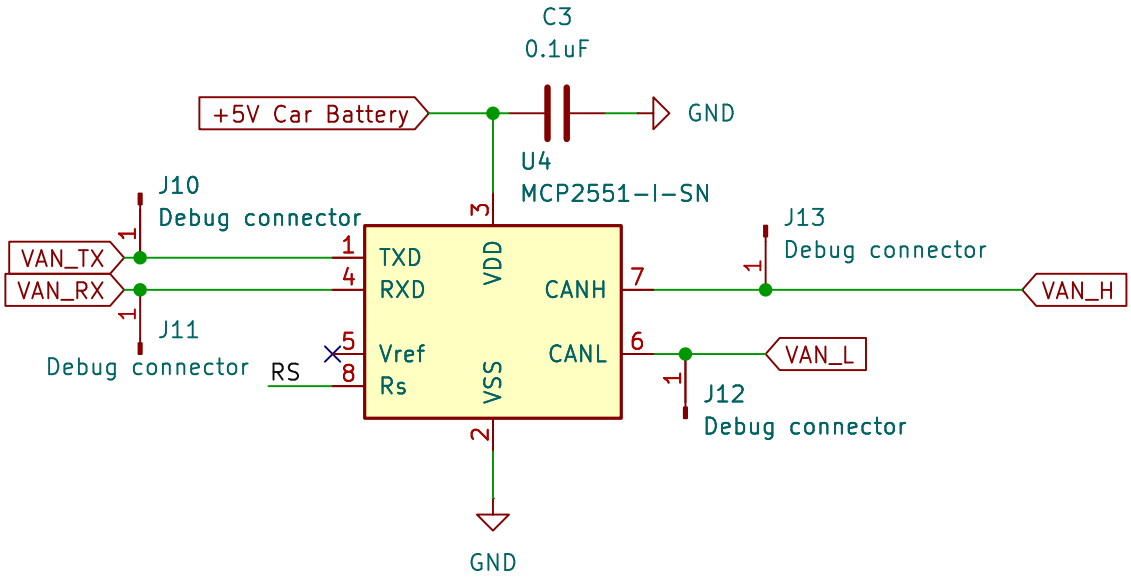
Kao mehanizam za ponovno pokretanje TSS integriranog kruga preko ESP mikroupravljača, ugrađen je N kanalni unipolarni tranzistor koji je spojen na nožicu za ponovno pokretanje TSS integriranog kruga. Kako je ta nožica aktivna na nisku naponsku razinu, kad je

tranzistor u zasićenju, nožica se spaja na uzemljenje te se tako TSS integrirani krug ponovno pokreće.



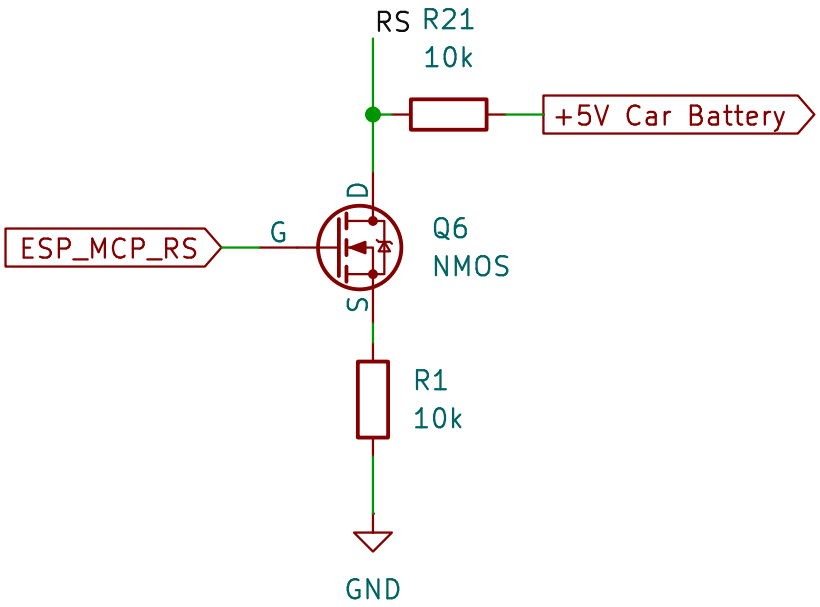
Slika 3.9: Pregled sheme - Tranzistor za ponovno pokretanje TSS-a

Integrirani krug koji prevodi TTL signale u diferencijalni par.



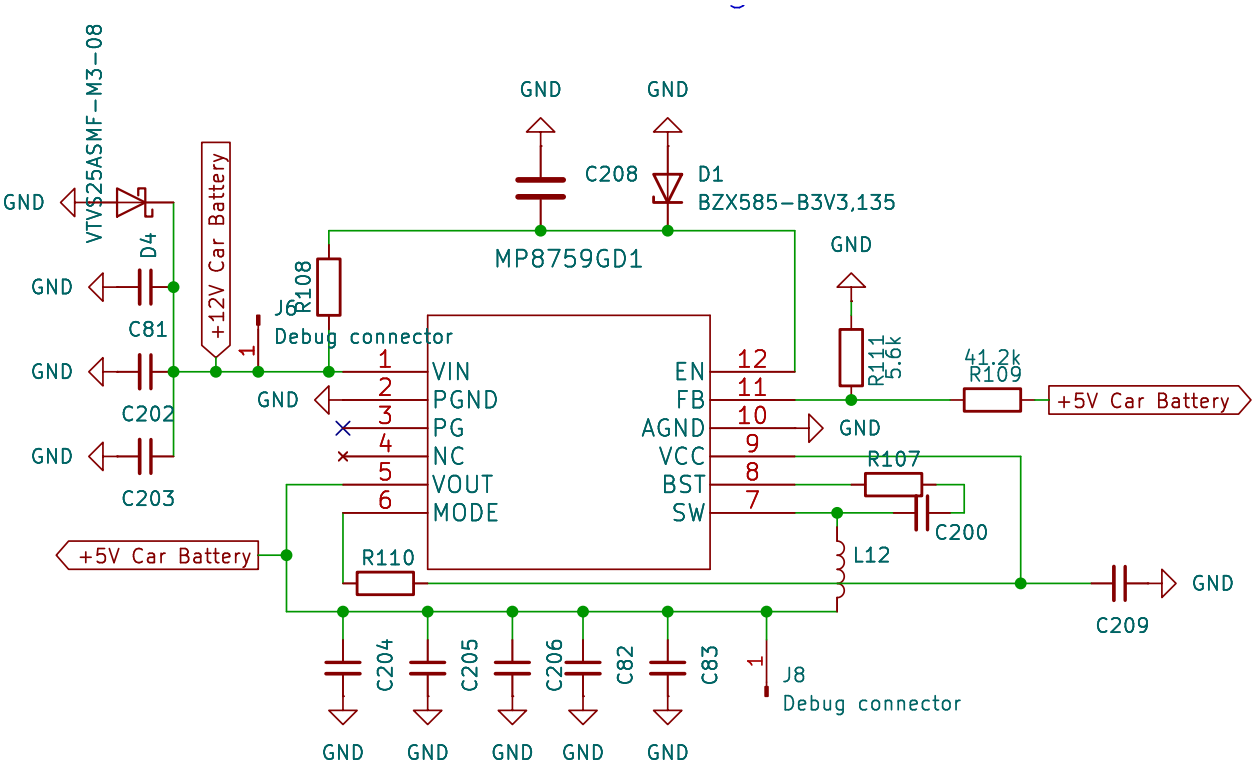
Slika 3.10: Pregled sheme - MCP

Kao mehanizam ponovnog pokretanja MCP integriranog kruga preko ESP mikroupravljača, implementiran je n kanalni unipolarni tranzistor koji je spojen na nožicu za ponovno pokretanje MCP integriranog kruga. Kako je ta nožica aktivna na nisku naponsku razinu, kad je tranzistor u stanju zasićenja, nožica se spaja na pull-down otpornik te se tako MCP integrirani krug ponovno pokreće.



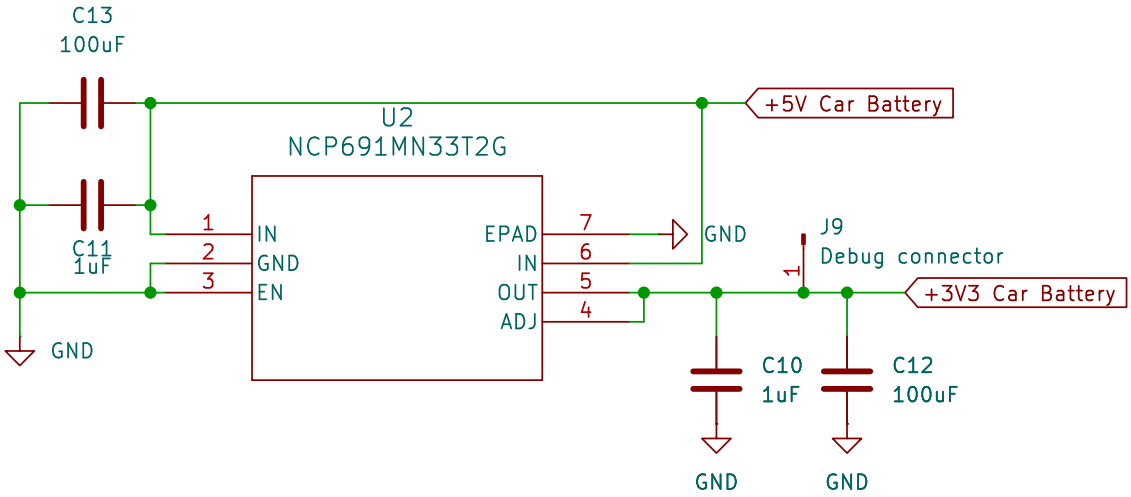
Slika 3.11: Pregled sheme - Tranzistor za ponovno pokretanje integriranog kruga MCP...

Strujni krug je preuzet iz tehničkog lista [6] integriranog kruga MP8759GD1, na slici 11.Na strujnom krugu se nalazi jedna TVS dioda kako bi osigurala ulaznu nožicu od previsokog napona kao i razne vrijednosti kondenzatora čija je uloga filtriranje i peglanje oblika ulaznog napona. Na izlazu se nalaze isto tako kondenzatori za filtriranje i peglanje oblika izlaznog napona. Kako je riječ o regulatoru napona koji nije ograničen na jednu vrijednost izlaznog napona, potrebno je odabrati željeni izlazni napon. A to je nešto što se radi na nožici 7 običnog naponskog djelila R111 i R109. Tablica s parovima vrijednosti za željene vrijednosti izlaznog napona se nalaze u tehničkom listu [6] na stranici 16.



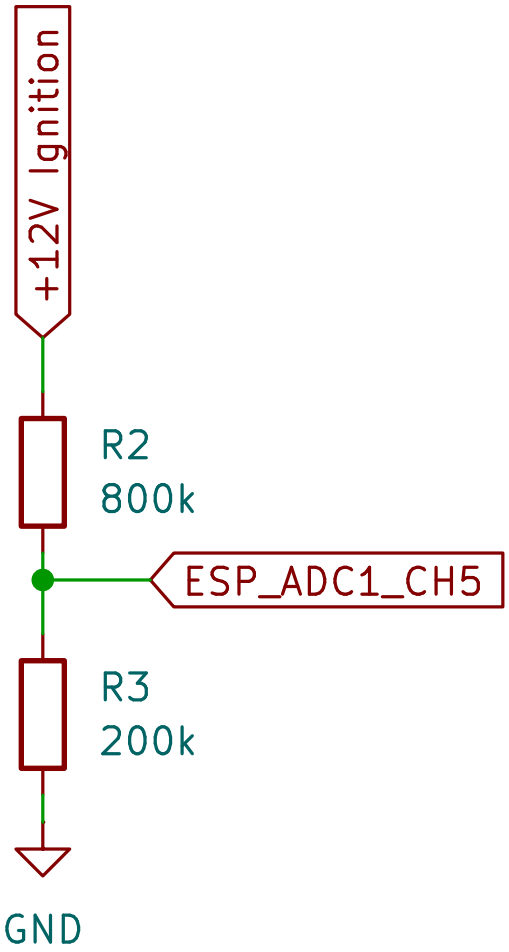
Slika 3.12: Pregled sheme - Regulator napona stalnih 5 V

Linearni regulator napona koji kao ulaz prima 5 V i na izlazu daje 3.3 V. Posjeduje par kondenzatora na ulazu i izlazu u svrhu filtriranja i peglanja napona.



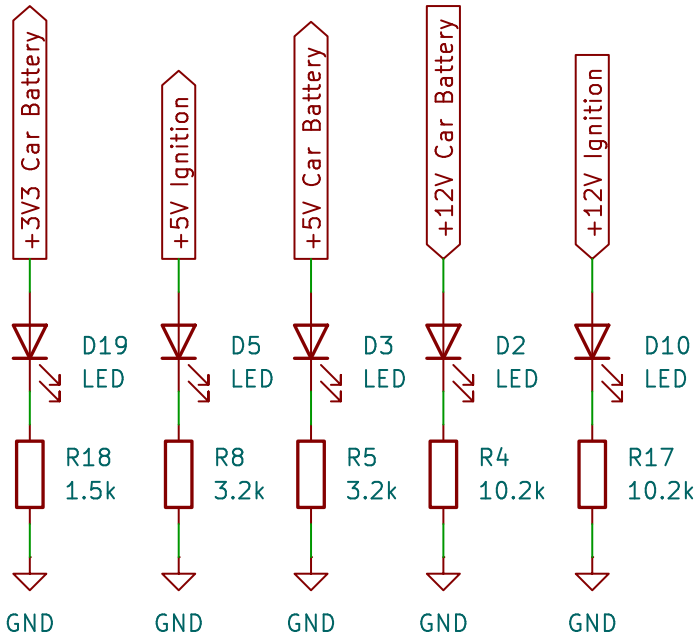
Slika 3.13: Pregled sheme - 3.3 V regulator napona

Kako bi se mjerila razina napona baterije automobila, korišten je obični djelitelj napona realiziran s pomoću dva otpornika. Kako bi se što više ograničila struja koja ovim djeliteljem teče odabrane su vrijednosti otpornika u rasponu od stotina tisuća ohma. Omjer otpora koji je korišten je 4:1 tj. na nožici ESP-a će biti 1/5 napona baterije automobila.

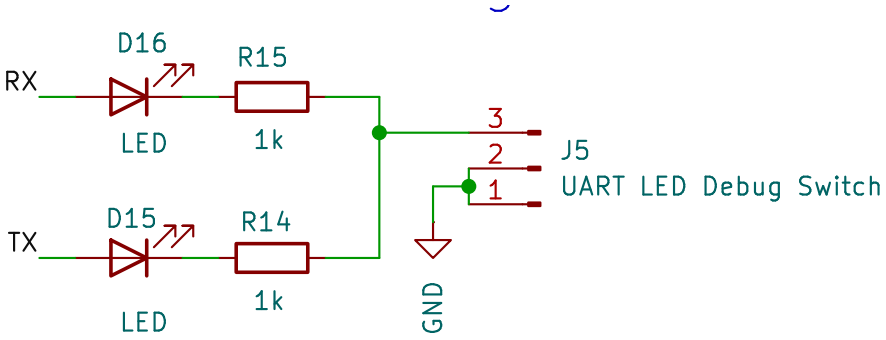


Slika 3.14: Pregled sheme - Djelitelj napona baterije

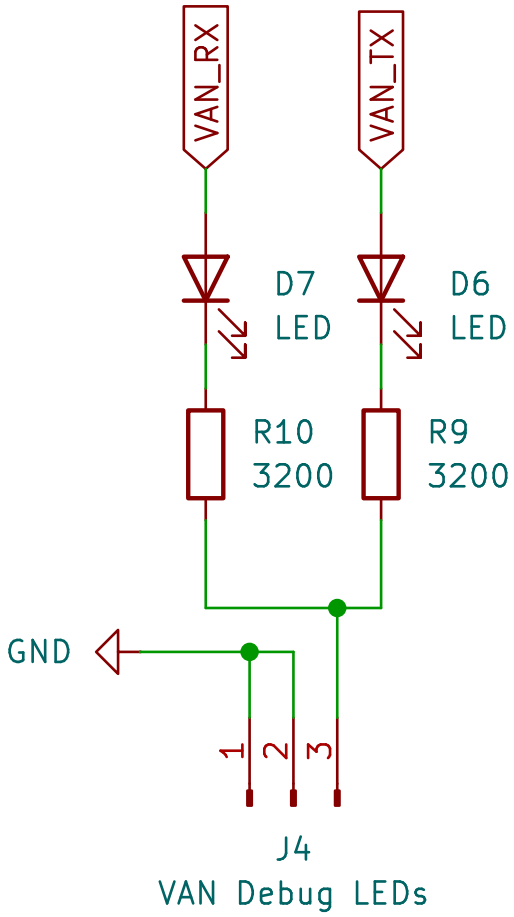
LE diode za detekciju grešaka su postavljene na mnogim bitnim mjestima kako bi ubrzale postupak razvoja. Svaka LED posjeduje otpornik koji ograničava struju koja prolazi kroz LED na 1 mA te se na njemu dešava preostali pad napona.



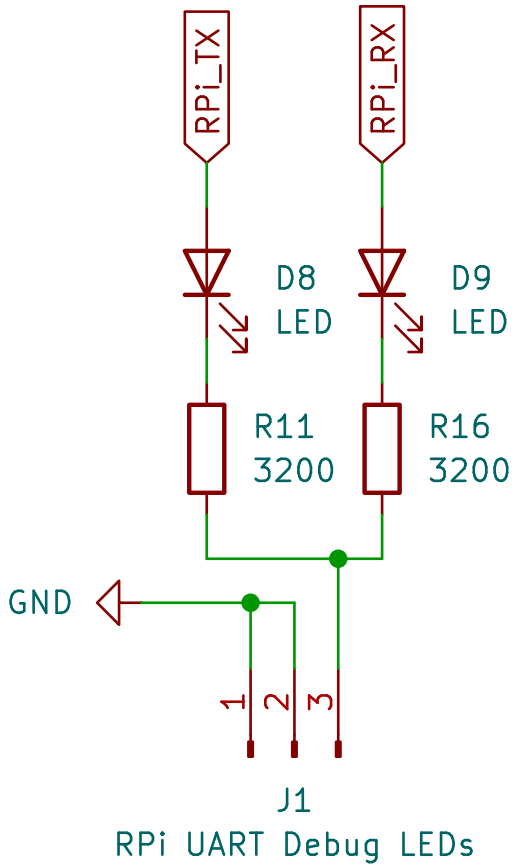
Slika 3.15: Pregled sheme - LE diode za detekciju i otklanjanje grešaka



Slika 3.16: Pregled sheme - LE diode za detekciju i otklanjanje grešaka na UART sabirnici

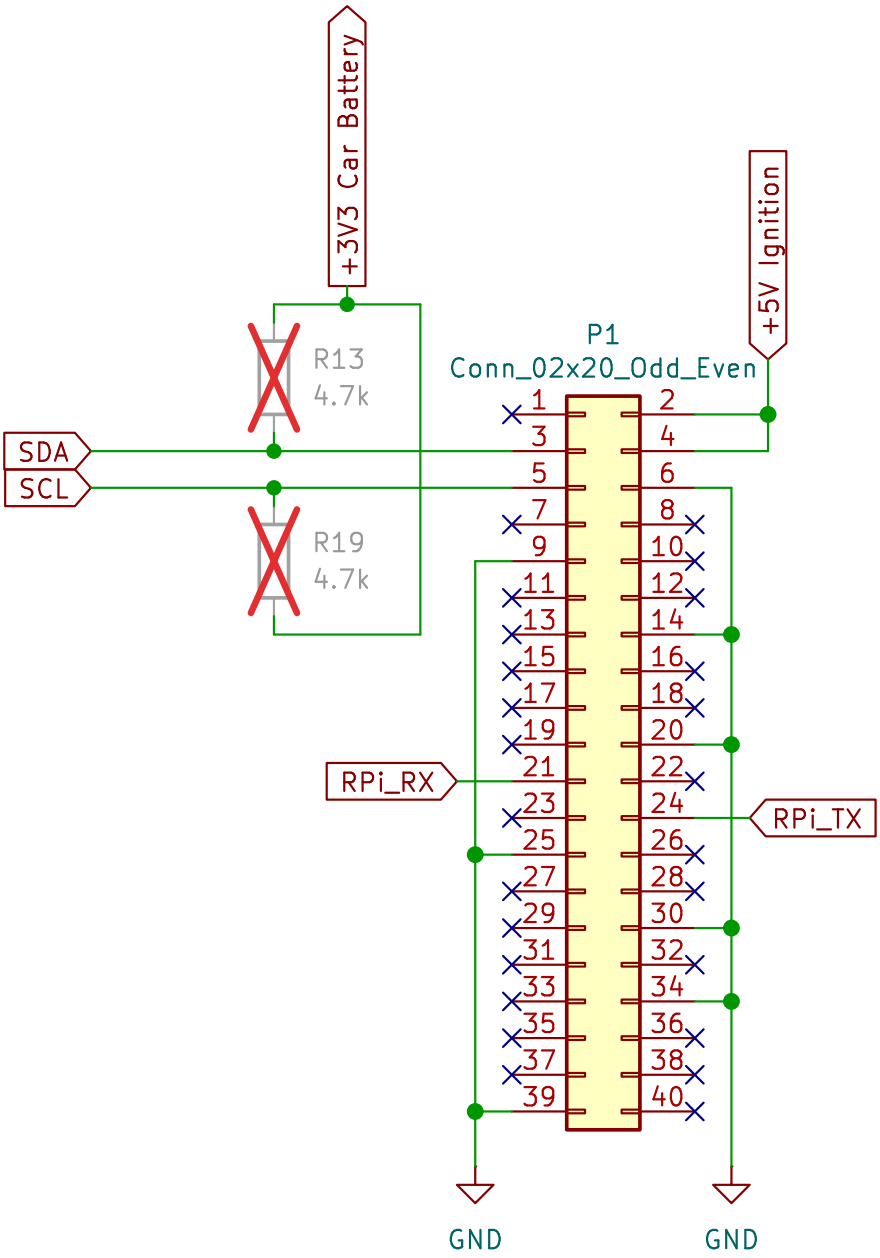


Slika 3.17: Pregled sheme - LE diode za detekciju i otklanjanje grešaka na VAN sabirnici



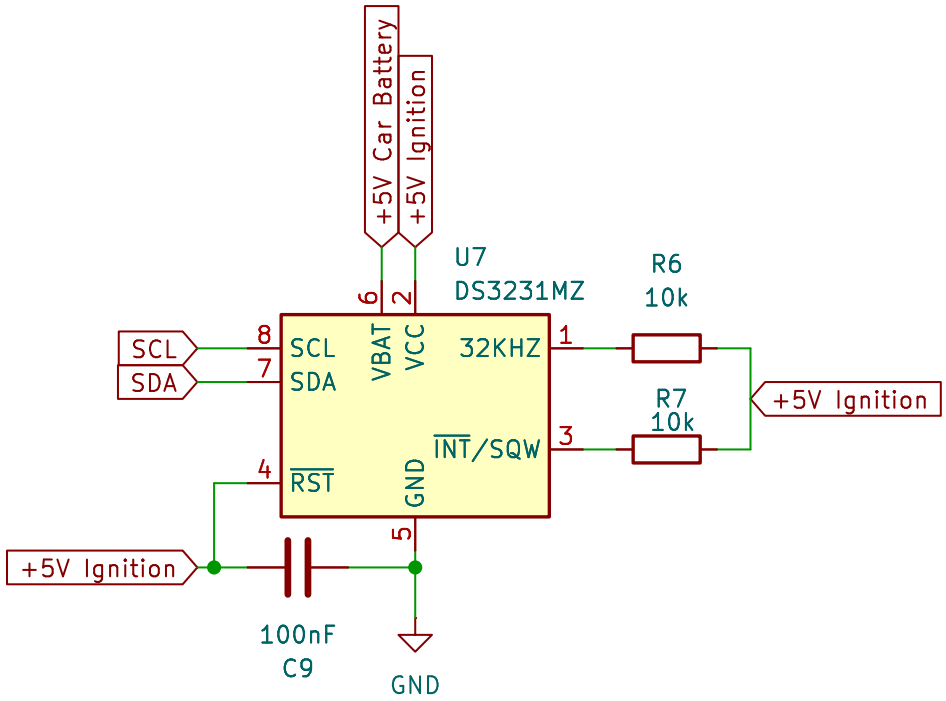
Slika 3.18: Pregled sheme - Shema spoja LE dioda za detekciju i otklanjanje žohara na UART sabirnici koja povezuje RPi i ESP32

Spojnica s 40 nožica karakteristična za RPi liniju mikroupravljača je prikazana na slici 3.19. Preko ove spojnice se UART sabirnicom povezuju ESP i RPi. I2C sabirnica se koristi za komunikaciju RPi mikroupravljača s RTC modulom te LCD ekranom. Kao opcija su ucrtane ali ne postavljene komponente otpornika R13 i R14.



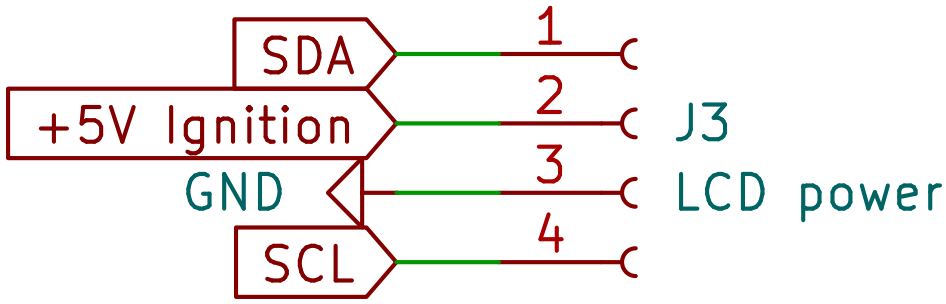
Slika 3.19: Pregled sheme - Spojnica za RPi mikroupravljač

Integrirani krug DS3231MZ obnaša ulogu RTC-a. Spoj je odrađen prema naputcima iz tehničkog lista [7]. Nožice 1 i 3 se spajaju preko otpornika na napon napajanja kako se ne koriste, nožice 7 i 8 su spojene na I2C sabirnicu RPi mikroupravljača. Postoji stalan izvor napajanja te drugi koji je aktivan kad se auto probudi.



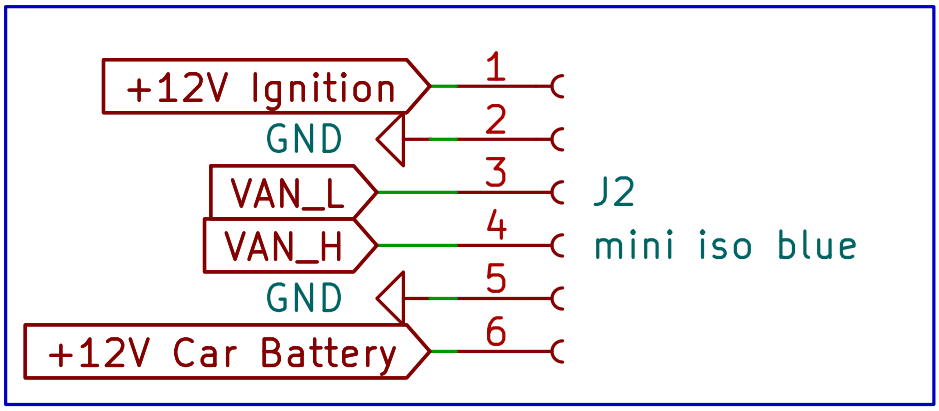
Slika 3.20: Pregled sheme - Shema spoja RTC modula za RPi mikroupravljač

Na slici 3.21 je prikazana JST spojnica za LCD ekran kojeg upogoni RPi. Nudi pristupne točke na I2C sabirnicu kao i izvor napajanja od 5 V.



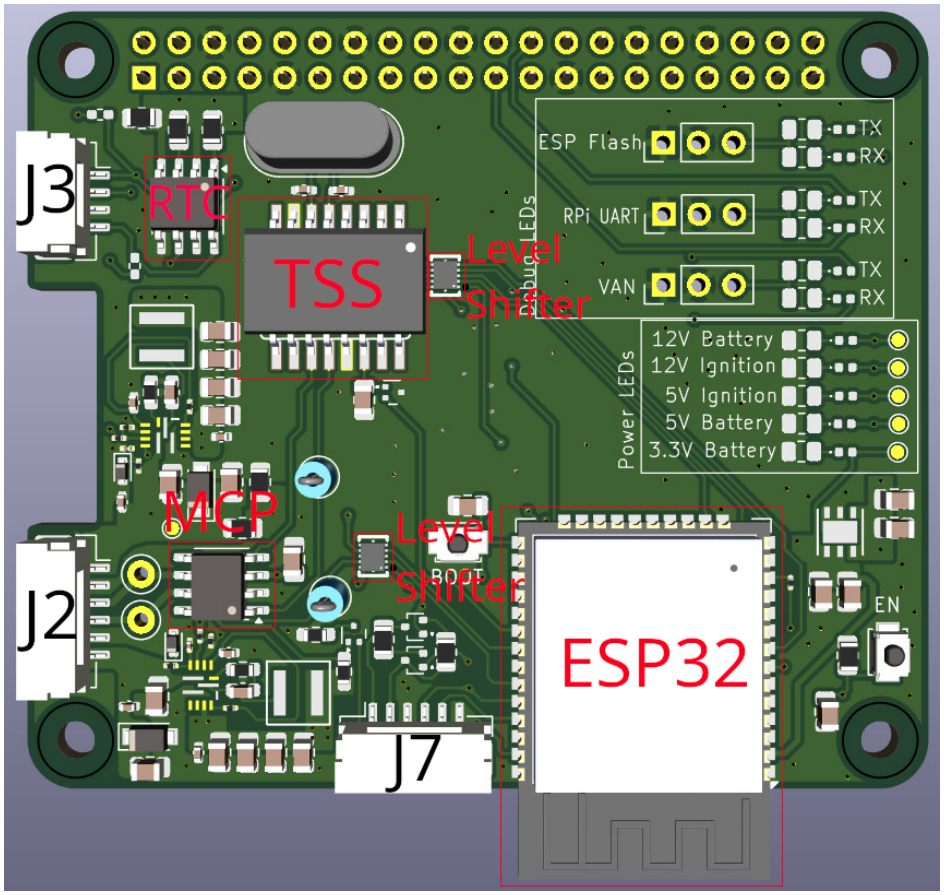
Slika 3.21: Pregled sheme - Spojnica za LCD ekran

Na slici 3.22 je JST spojnica koja omogućava spajanje na VAN mrežu automobila.

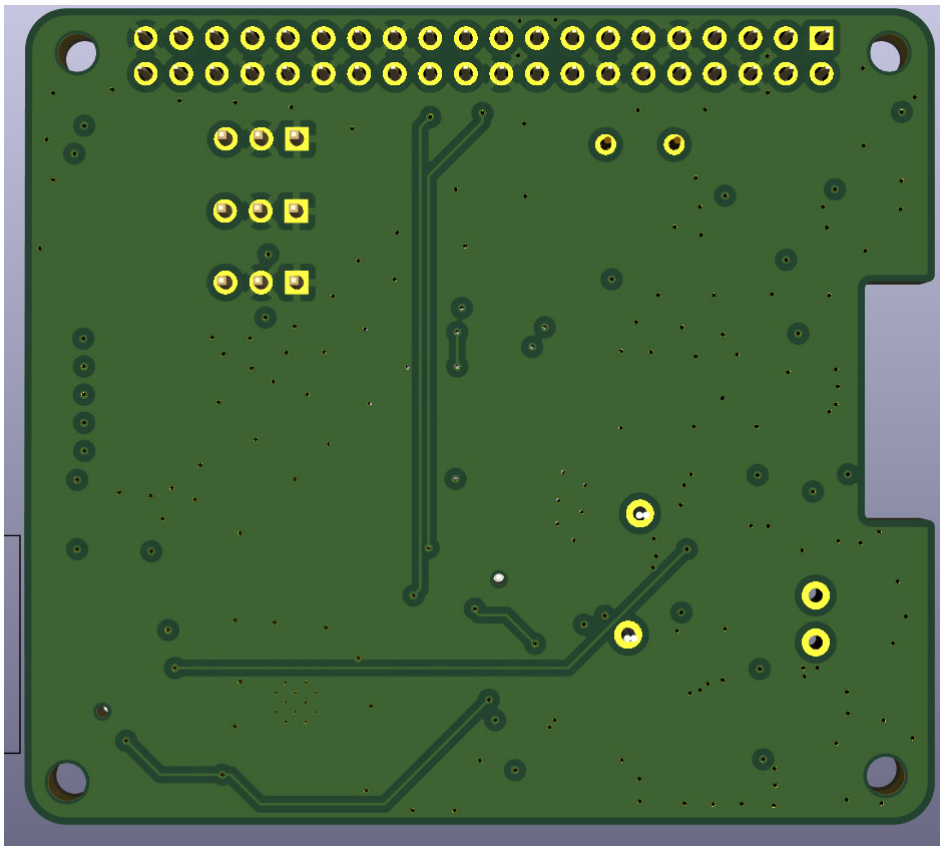


Slika 3.22: Pregled sheme - Spojnica za VAN mrežu automobila

Na slikama 3.23 i 3.24 se nalazi izgled printane tiskane pločice koja predstavlja sklopovski dio projekta. Sve komponente se nalaze s iste strane kako bi se olakšao i ubrzao proces lemljenja. Radi lakšeg snalaženja je svaka značajnija komponenta naznačena na slici. Oznake se manifestiraju bojama kao i pratećim tekstom. Spojnica J3 označava mjesto gdje se spaja LCD ekran, J2 označava mjesto povezivanja projekta s mrežom auta te J7 označava mjesto s kojeg je moguće programirati ESP32 mikroupravljač.

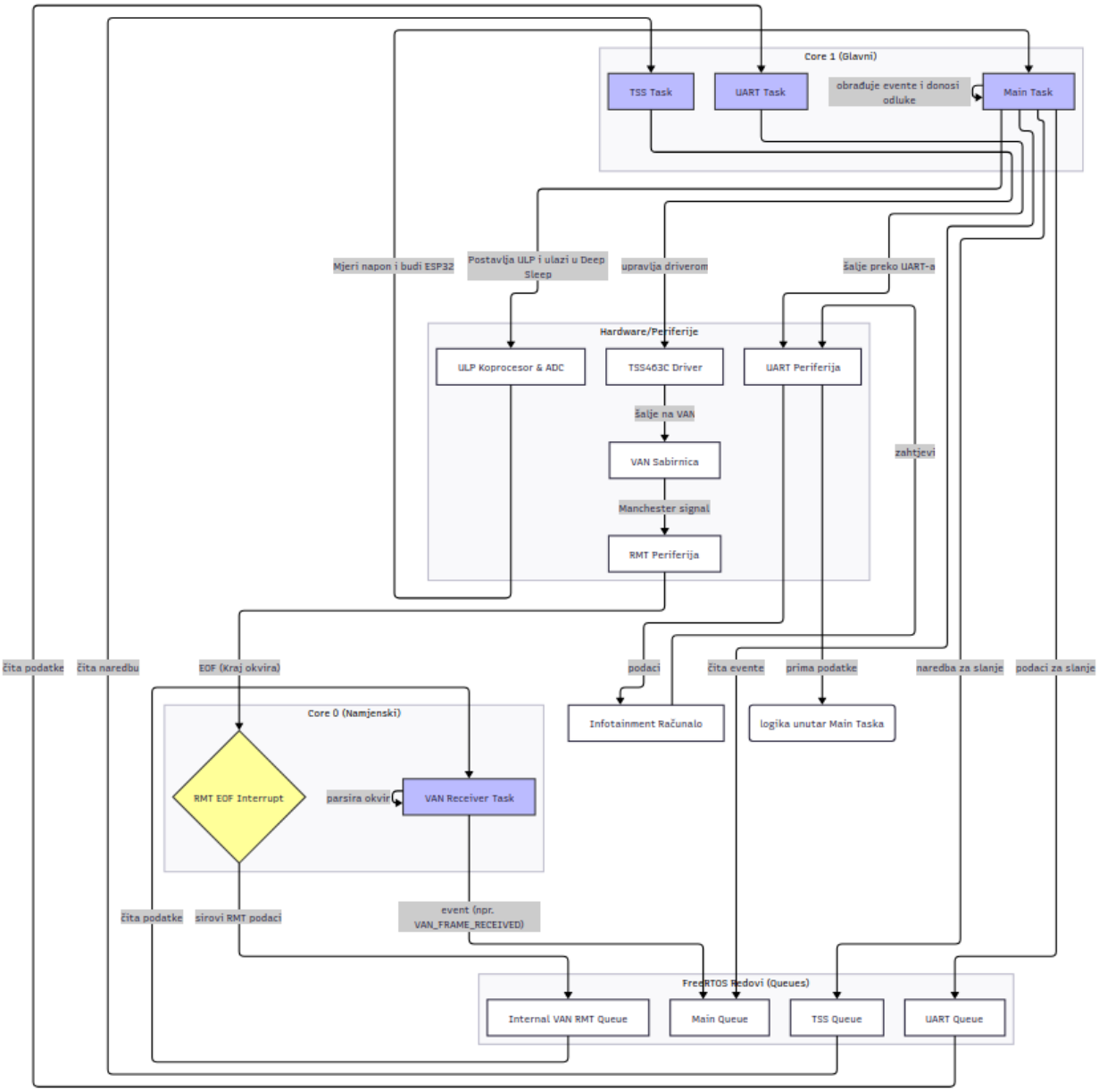


Slika 3.23: Izgled prototipa - PCB prednja strana



Slika 3.24: Izgled prototipa - PCB zadnja strana

3.3 Realizacija programskog rješenja



Slika 3.25: Blok dijagram programskog rješenja

Pristupni modul je najvažniji dio ovog projekta jer se sučeljava s vozilom i odgovoran je za upravljanje napajanjem cijelog sustava. Modul prima i analizira VAN okvire, organizira podatke i zatim ih prosljeđuje infotainment računalu. Također obavlja i obrnuti proces: može primati podatke s infotainment računala i prenijeti odgovarajući VAN okvir na sabirnicu. upravljanje napajanjem također je važan dio programske podrške, budući da prekomjerna potrošnja u stanju mirovanja brzo iscrpljuje akumulator vozila. Detaljan blok dijagram je prikazan na slici 3.25 čije objašnjenje se nalazi u nastavku.

3.3.1 Okvir i pristup

Kao program za razvoj koristimo ESP-IDF programski okvir koji pruža Espressif. Okvir se temelji na operativnom sustavu FreeRTOS. Zbog prirode OS-a i programskog okruženja, programska podrška modula napisana je koristeći pristup temeljen na događajima.

Koristimo sljedeće događaje kao osnovu za glavne petlje/zadatake:

- Primanje VAN okvira
- Primanje UART podataka s računala
- Promjena stanja vozila
- Timer događaji
- Glavni zadatak

Budući da ESP32 ima dvije jezgre, jedna je posvećena isključivo primanju i analiziranju VAN okvira, jer su oni vrlo česti i ne želimo propustiti niti jedan.

3.3.2 Primanje VAN okvira

Za primanje VAN okvira potrebno je ručno čitati i vremenski pratiti stanja RX pina. Srećom, ESP32 ima sklopovski periferni uređaj koji upravo to omogućuje: RMT periferni uređaj. On može pratiti stanja GPIO pinova i mjeriti koliko dugo su u tom stanju. Nakon što postavimo pragove za minimalnu i maksimalnu duljinu signala, periferni uređaj može generirati prekid (eng. *interrupt*) svaki put kada detektira sekvencu **EOF**.
Kada se detektira sekvenca **EOF**, RMT periferalni upravljački program generira prekid (eng. *interrupt*). U okviru tog prekida, u red zadataka (eng. *task queue*) modula za primanje VAN okvira postavljamo novi događaj i ponovno pokrećemo RMT. Modul za primanje VAN okvira preuzima primljene RMT podatke iz reda i analizira ih.

VAN okvir rekonstruiramo bit po bit iz tog polja, uzimajući u obzir Manchester bitove. Nakon provjere kontrolnog zbroja (eng. *checksum*), izdvajamo identifikator i podatke, stavljamo ih u novi međuspremnik i postavljamo u red glavnog zadatka (eng. *Main task*) za daljnju obradu. Logika primanja inspirirana je Peter Pinterovom Arduino bibliotekom [4] i prilagođena je arhitekturi naše programske podrške.

3.3.3 Analiza VAN okvira

Kada modul za primanje VAN okvira postavi novi događaj, glavni zadatak (eng. *Main task*) poziva parser paketa koji provjerava identifikator i, ako je prepoznat, analizira podatke u globalne međuspremnike te postavlja odgovarajuće događaje u red glavnog zadatka ovisno o paketu koji se obrađuje. Velika većina poznatih dekodiranih VAN okvira dostupna je na web stranici Petera Pintera [5].

3.3.4 Slanje VAN okvira

Za slanje VAN okvira prema vozilu koristimo TSS463C. TSS je zapravo dodatnih 128 bajtova RAM-a kojima moramo upravljati, budući da moramo kontrolirati svu memoriju korištenu za slanje i primanje podataka. Trenutačno se TSS koristi samo za slanje paketa CD izmjenjivača

kako bismo ga mogli emulirati, te za slanje zahtjeva za resetiranje stanja putnog računala. Poruke CD mjenjača su poruke s neposrednim odgovorom (vidi 2.1.3), dok je zahtjev za resetiranje stanja putnog računala normalni okvir s zahtjevom za potvrdom (ACK). Paketi CD mjenjača šalju se periodično, svake sekunde, inače će ugrađeni radio smatrati da CD mjenjača nije prisutan i onemogućiti ga.
TSS komunicira preko SPI sučelja i malo je nezgrapan za korištenje, pa je napisan vlastiti upravljački program radi lakšeg rukovanja. Osim toga, postoji zaseban zadatak koji nadzire stanje TSS poruka i stavlja ih u red za slanje.

3.3.5 UART komunikacija

Informacijsko-zabavno računalo komunicira s ESP32 preko UART sučelja. Razlog korištenja UART-a je njegova jednostavnost. ESP32 uglavnom šalje podatke računalu, ali također može primati zahtjeve s računala.
UART kod se izvodi u vlastitom zadatku koji prima događaje iz drugih zadataka i također može slati događaje glavnom zadatku. Kada glavna petlja (eng. *Main loop*) detektira promjenu u globalnim međuspremnicima podataka (primljen VAN okvir, promjena napona akumulatora, itd.), postavlja odgovarajući događaj u UART zadatak s podacima koji se šalju računalu. Protokol i strukture podataka su inspirirane MSP protokolom korištenim u programskoj podršci upravljača leta na FPV dronovima.

3.3.6 Upravljanje događajima i zadacima

Programska podrška koristi nekoliko zadataka koji se izvode u vlastitim petljama i čekaju događaje iz više redova. Ukupno postoje 4 globalna reda u programskoj podršci, po jedan za svaku glavnu komponentu:

- Glavni red - Čita ga glavni zadatak. Svi ostali zadaci šalju svoje događaje u red glavnog zadatka. Glavni zadatak zatim odlučuje što učiniti i gdje proslijediti događaj.
- TSS red - Čita ga TSS zadatak. Koristi se za stavljanje paketa u red za slanje od strane TSS-a.
- UART red - Čita ga UART zadatak. Koristi se za slanje paketa informacijsko-zabavnom računalu.
- VAN red (neiskorišteno) - Čita ga VAN RMT zadatak. Ostavljen za buduću upotrebu.

Osim toga, postoji interni red u VAN RMT zadatku. Taj red se isključivo koristi između prekida (interrupt) RMT uređaja i VAN RMT zadatka.

Događaji su definirani kao struktura s identifikatorom događaja i pokazivačem na podatke. Podaci ovise o primljenom događaju.

3.3.7 Upravljanje režimima napajanja

Važno je upravljati potrošnjom energije sustava. Radi lakšeg upravljanja definirali smo nekoliko „stanja vozila”. Pogledajte tablicu 3.1 za popis tih stanja i ponašanje sustava. Jedna važna komponenta spomenuta u tablici je stanje dubokog spavanja ESP32-a (eng. *deep sleep*). Kada je u dubokom snu, ESP je praktički isključen. Jedino što radi na čipu je

posebni dio RAM-a, unutarnji RTC sklop i, po potrebi, ULP koprocesor. U našem slučaju koristimo isti ADC pin koji se koristi za mjerenje napona baterije kako bismo probudili ESP. Zbog naponskog raspona nožice, ne možemo koristiti jednostavno buđenje putem GPIO-a, već moramo koristiti ULP koprocesor za mjerenje napona i buđenje ESP-a kada napon prijeđe određeni prag.

3.3.8 ULP koprocesor

ESP32 ima vrlo niskopotrošni koprocesor. Ima vrlo jednostavnu arhitekturu i troši vrlo malo energije. Idealno je koristiti ga za ADC mjerenja i buđenje ESP-a. Ne može se programirati izravno u C-u, ali postoje dva načina pisanja koda. Prvi način je pisanje asembler koda u zasebnoj datoteci i njegovo integriranje u datoteku firmwarea. Drugi način je korištenje unaprijed definiranih C makroa i pisanje programa u polju unutar C programa te njegovo učitavanje na taj način. Odabrali smo drugu opciju jer je znatno jednostavnija.

Opis	**Stanje računala**	**ESP32 stanje**
Automobil u stanju mirovanja	Isključeno (bez napajanja)	Stanje spavanja
Motor zaustavljen, automobil zaključan i nije u stanju mirovanja	Isključeno	Aktivno stanje
Motor zaustavljen, automobil nije zaključan i nije u stanju mirovanja	Ako dolazi iz gornjih stanja, isključeno, inače samo ekran isključen	Aktivno stanje
Motor zaustavljen, radio uključen	Uključen, ekran uključen	Aktivno stanje
Motor zaustavljen, ključ u poziciji ACC	Uključen, ekran uključen	Aktivno stanje
Motor zaustavljen, ključ u poziciji IGN	Uključen, ekran uključen	Aktivno stanje
Motor se pokreće	Uključen, ekran isključen	Aktivno stanje
Motor je pokrenut	Uključen, ekran uključen	Aktivno stanje

Tablica 3.1: Tablica stanja auta

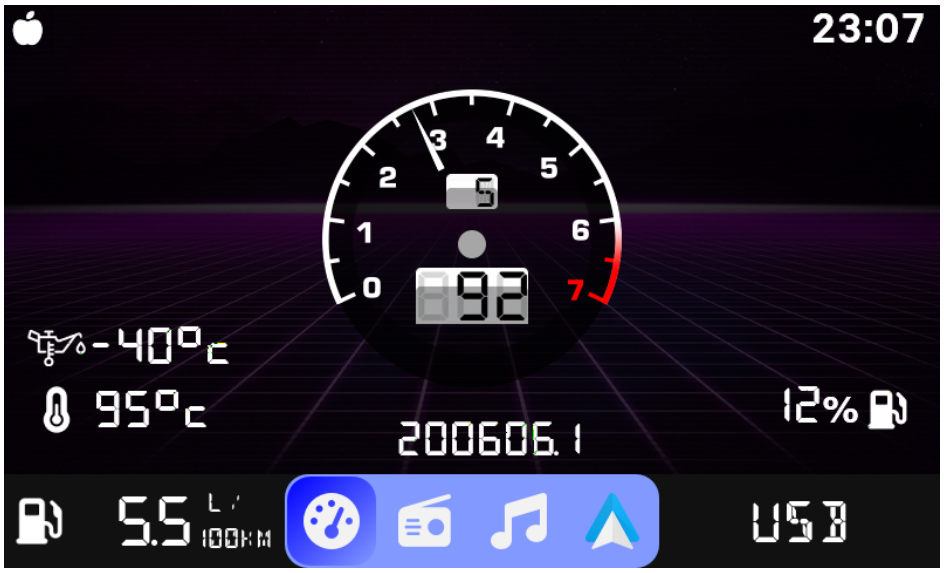
4 Testiranje i rezultati

4.1 Metodologija testiranja

Način testiranja je ugraditi projekt u automobil te testirati njegov rad tijekom vožnje. Očekivano ponašanje je ispravan prikaz brzine automobila, obrtaja te trenutačnog stupnja prijenosa na ekranu. Ispravnost prikaza se određuje usporedbom prikazanih vrijednosti s pokaznicima brzine, obrtaja i trenutačne brzine koji se tvornički nalaze u automobilu. Dozvoljeno je malo odstupanje zbog nepreciznosti pokazivača koji se nalaze već par desetljeća u automobilu. Pored Navedenih indikatora tu su još i indikatori za razinu spremnika, temperaturu motora automobila, ukupan broj prijeđenih kilometara te prosječnu potrošnju na 100 kilometara čija točnost se procjenjuje na isti način koji je naveden. Nadalje, testira se i funkcionalnost multimedije tako da se prikazuje izbornik za radiostanice kao i mogućnost reprodukcije multimedijskog sadržaja iz neke vanjske memorije npr. USB uređaj za pohranu podataka.

4.2 Rezultati testiranja

Prvo je testiran prikaz telemetrije automobila čiji prikaz je vidljiv na slici 4.1. Uzastopnim vožnjama i prometnim kamerama je utvrđeno kako se na ekranu prikazuje točna vrijednost brzine automobila i obrtaja motora. Kako se stupanj prijenosa računa na osnovu obrtaja i brzine kretanja automobila, to je veličina koja bi bila podložna greškama u izračunima međutim u praksi se pokazalo kao vjerodostojan izračun bez krivih indikacija. Indikatori razine spremnika goriva, temperaturu motora automobila te prosječna potrošnja se podudaraju s vrijednostima prikazanih na glavnoj tabli čime zaključujemo kako sva komunikacija ispravno radi.



Slika 4.1: Prikaz ekrana tijekom testiranja - Telemetrija automobila

Na slici 4.2 je prikazan test učitavanja multimedijskog sadržaja sa vanjskog uređaja za pohranu podataka. Kako je sadržaj bez ikakvih problema učitan te reproduciran, zaključuje se kako i taj dio sustava radi kao što je predviđeno.

Slika 4.2: Prikaz ekrana tijekom testiranja - Multimedija - Uređaj za pohranu podataka

Na slikama 4.3 i 4.4 je prikazan test prikaza aktivne radio stanice (ako je radio upaljen) iz kojih se može vidjeti kako i ta funkcionalnost ima predviđeno ponašanje.

Slika 4.3: Prikaz ekrana tijekom testiranja - Multimedija - Radio

Slika 4.4: Infotainment sustav smješten u autu

5 Zaključak

Realizacija ovog projekta pokazala je da je moguće uspješno integrirati moderan infotainment sustav u starije vozilo koristeći kombinaciju vlastitog hardverskog modula i open-source softverskih alata. Ključni izazovi bili su razumijevanje i implementacija VAN protokola, osiguranje stabilnog napajanja te izrada pouzdanog firmwarea za ESP32 mikroupravljač. Pažljivim dizajnom tiskane pločice, korištenjem provjerenih komponenti i pisanjem modularnog koda, postignuta je robusna i stabilna komunikacija s vozilom.

Prednosti rješenja su otvorenost i mogućnost proširenja funkcionalnosti, niska cijena u odnosu na komercijalne alternative te potpuna kontrola nad softverom i hardverom. Sustav se pokazao pouzdanim tijekom testiranja – prikaz telemetrije, statusa vozila i reprodukcija multimedije radili su bez većih odstupanja od tvorničkih pokazivača. Najveći izazovi, poput dekodiranja VAN okvira i implementacije režima štednje energije, uspješno su savladani korištenjem ESP-IDF okvira i pažljivim upravljanjem događajima i zadacima.

Ipak, postoje područja za daljnje unaprjeđenje. Potrebno je dodatno optimizirati potrošnju energije u stanju mirovanja kako bi se maksimalno smanjilo pražnjenje akumulatora te doraditi korisničko sučelje na Raspberry Pi računalu radi intuitivnijeg korištenja. Budući razvoj mogao bi uključivati potpunu integraciju s postojećim zaslonom u vozilu, dodavanje bežične povezivosti (Wi-Fi/Bluetooth sinkronizacija s pametnim telefonom) te naprednije funkcije poput OTA (Over-the-Air) nadogradnje softvera. Time bi se sustav dodatno približio funkcionalnosti i fleksibilnosti modernih tvorničkih infotainment rješenja.

Literatura

[1] Alain Chautar. „Info Tech n°6". *Educauto.org* (2003.). URL: http://graham.auld.me.uk/projects/vanbus/datasheets/Mux1.pdf.

[2] PSA Peugeot Citroën. *Peugeot Service Box backup - SEDRE*.

[3] Atmel Corporation. *VAN Data Link Controller with Serial Interface, TSS463C*. 2006.

[4] Peter Pinter. *ESP32 RMT peripheral Vehicle Area Network (VAN bus) reader*. URL: https://github.com/morcibacsi/esp32_rmt_van_rx.

[5] Peter Pinter. *PSA VAN bus, All data related to Peugeot 206 307 406 S2, Citroen C3 2004*. URL: http://pinterpeti.hu/psavanbus/PSA-VAN.html.

[6] Monolithic power. *MP8759 Datasheet*. URL: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP8759GD-Z/document_id/1011/.

[7] Maxim Integrated Products. *DS3231MZ Datasheet*. URL: https://www.alldatasheet.com/datasheet-pdf/download/423153/MAXIM/DS3231MZ.html.

Prilozi i dodaci

P.1 Kompletna shema sklopovlja

ESP32-WROOM

File: ESP32_WROOM.kicad_sch

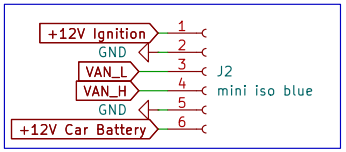
ECU_comms

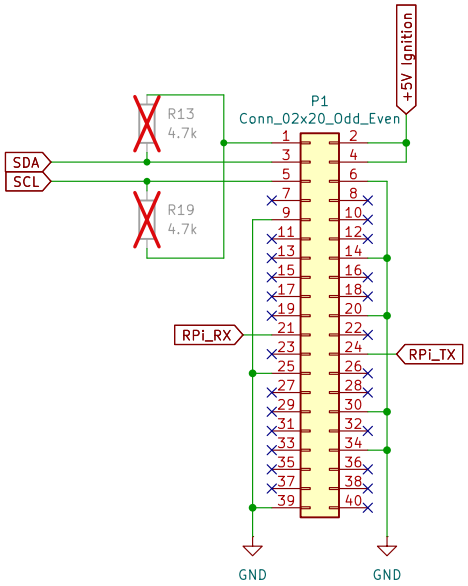
File: ECU_comms.kicad_sch

Power

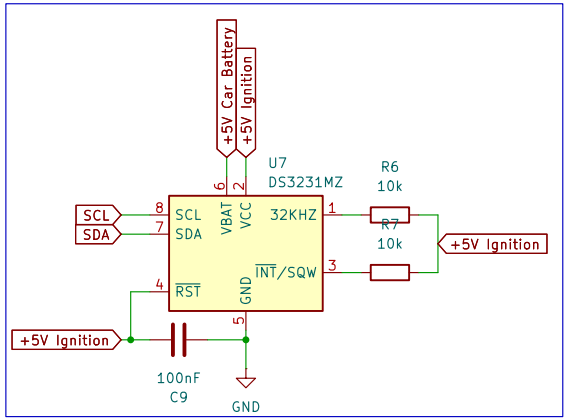
File: power.kicad_sch

Mini ISO Blue -> JST connector

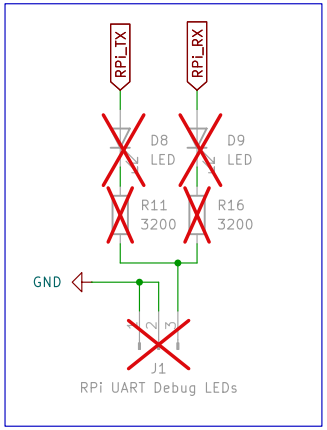




RPi RTC



UART Debug LEDs



This board is designed to act as a HAT
for a Raspberry Pi 3B (non plus).

It would work for a 3B+, but the RUN header is not in the same place,
so the power control would not work.

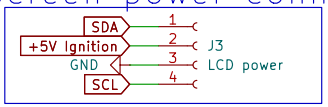
There are 2 power rails, one for the ESP, and one for the Pi.
The ESP is designed to be powered by the permanent 12 volt rail
from the car, and the Pi from the VAN+ rail that comes on when
the car wakes up from sleep.

By using the ${f7473ed4-a3a0-4b2d-bc64-33b6e5e2e529:VALUE} jumper, you can power the ESP
from the VAN+ Power rail.

Note: The VAN+ power rail is also 12 volts,
but this board expects it to be regulated by
an external voltage regulator (for now).

The board can also be used as a standalone
VAN bus reader.

LCD screen power connector



Mounting Holes

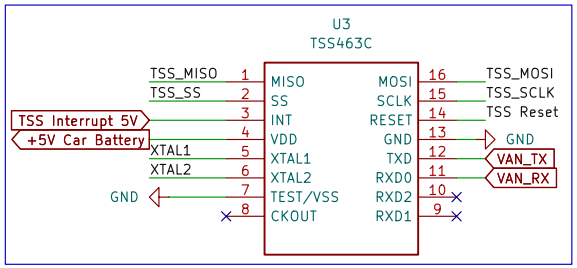
MK1 M2.5
MK2 M2.5
MK3 M2.5
MK4 M2.5

Sheet: /
File: main.kicad_sch

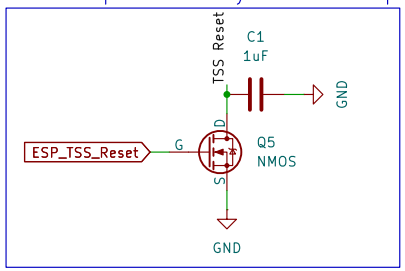
Title: Peugeot Project PCB

Size: A4	Date:	Rev: v1.0
KiCad E.D.A. 9.0.4		Id: 1/4

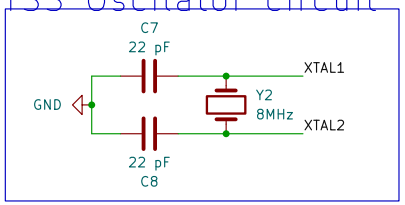
TSS – VAN DLC



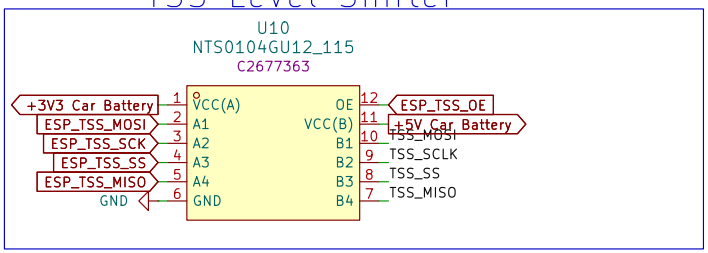
TSS Reset pin driven by a 3V3 ESP pin



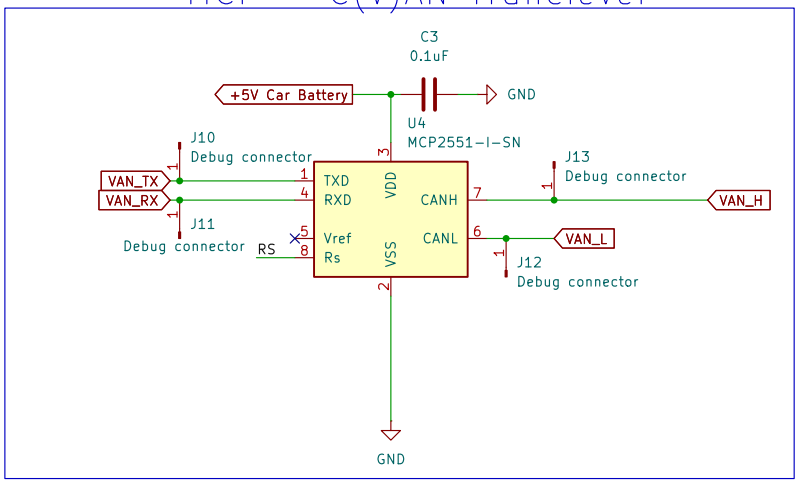
TSS Oscilator circuit



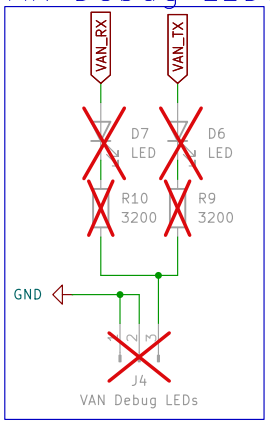
TSS Level Shifter



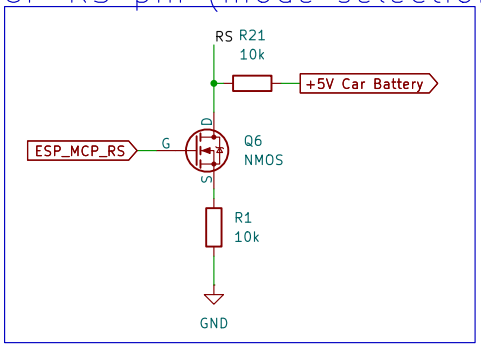
MCP – C(V)AN Tranciever



VAN Debug LEDs

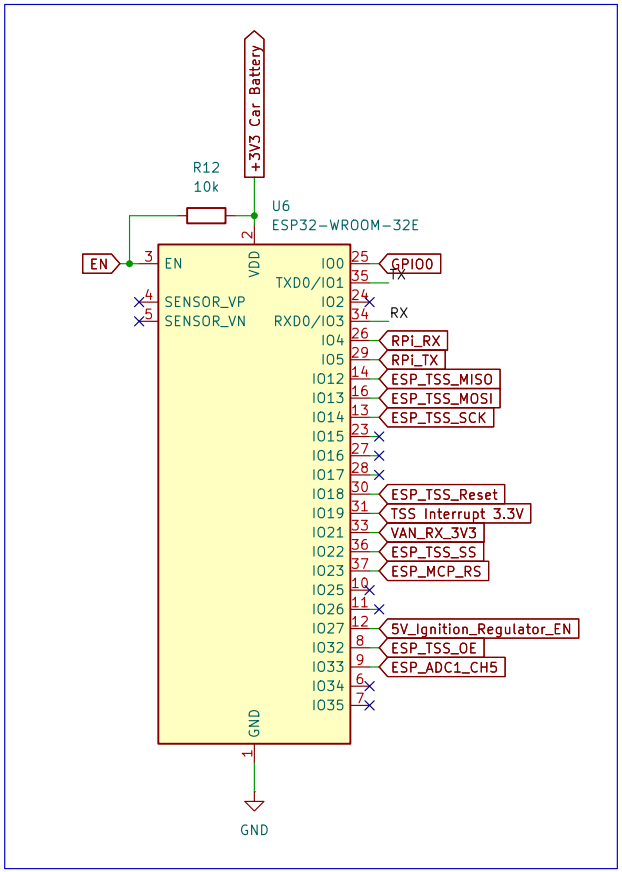


MCP RS pin (mode selection)

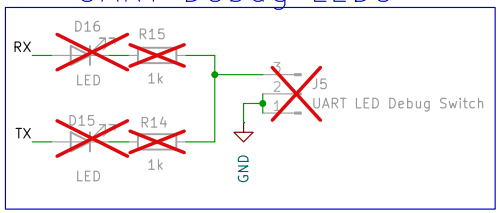


Sheet: /ECU_comms/		
File: ECU_comms.kicad_sch		
Title: Peugeot Project PCB		
Size: A4	Date:	**Rev: v1.0**
KiCad E.D.A. 9.0.4		Id: 2/4

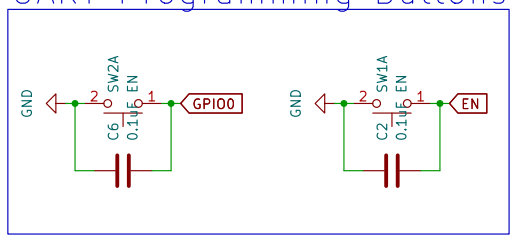
ESP 32 WROOM 32E



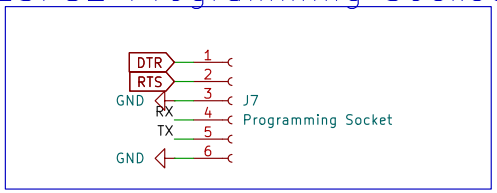
UART Debug LEDs



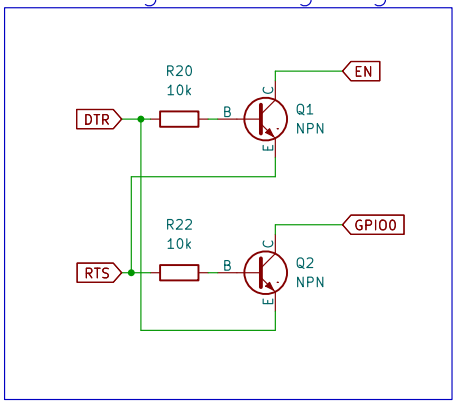
UART Programming Buttons



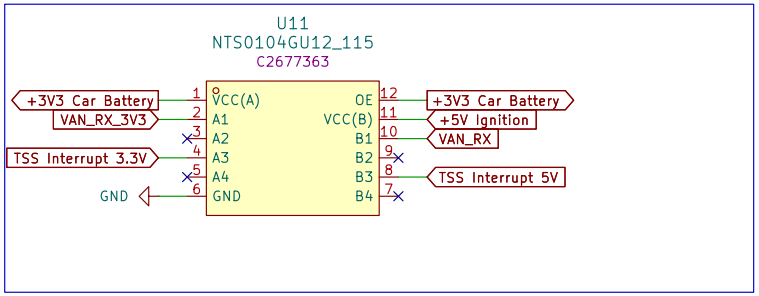
ESP32 Programming Socket



UART Programming Signals



GP Level Shifter

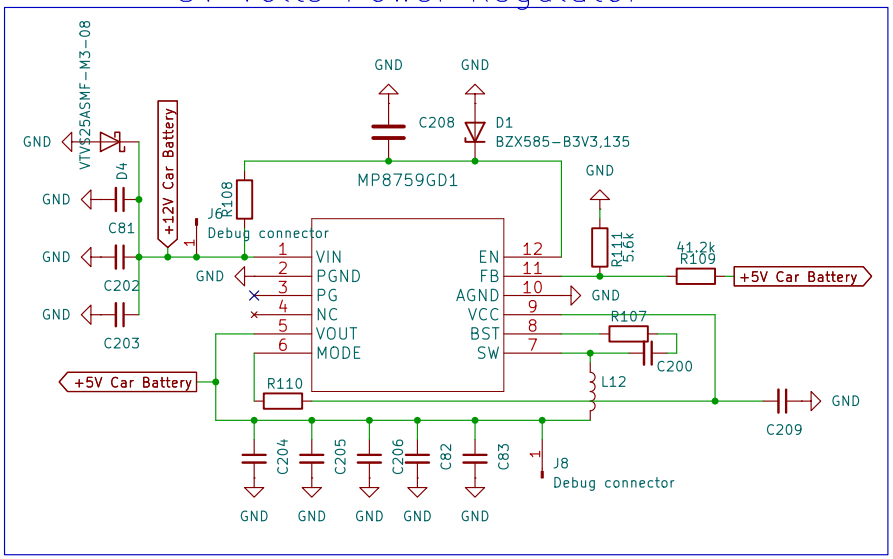


Sheet: /ESP32-WROOM/
File: ESP32_WROOM.kicad_sch

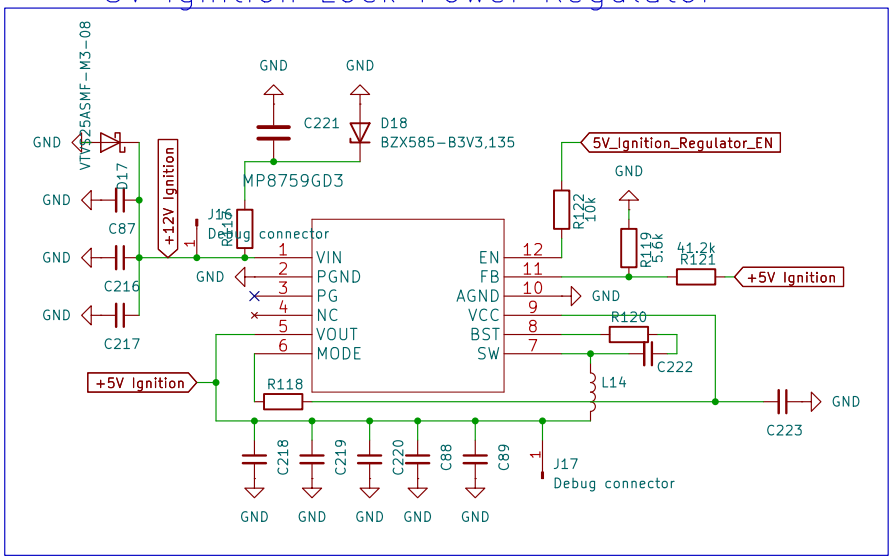
Title: Peugeot Project PCB

Size: A4	Date:	**Rev: v1.0**
KiCad E.D.A. 9.0.4		Id: 3/4

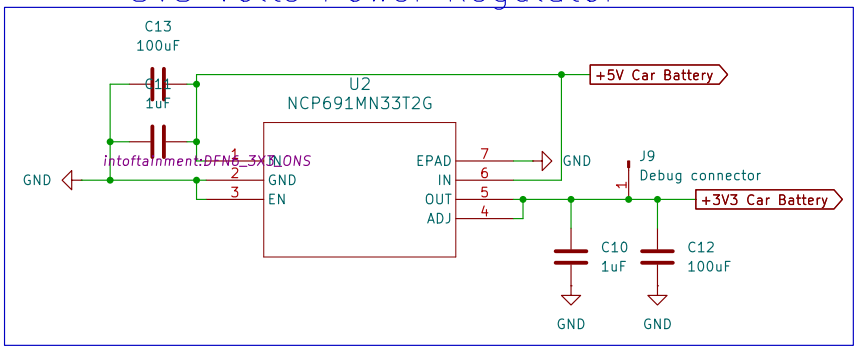
5V Volts Power Regulator



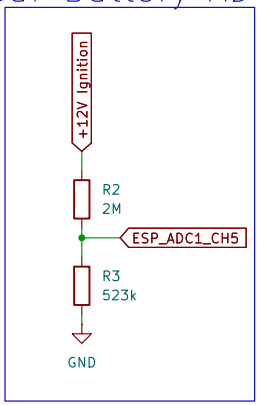
5V Ignition Lock Power Regulator



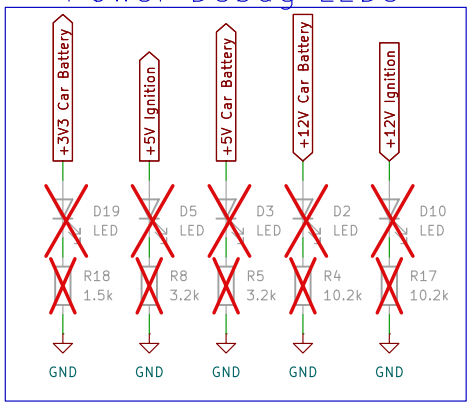
3V3 Volts Power Regulator



Car Battery ADC



Power Debug LEDs



Sheet: /Power/ File: power.kicad_sch		
Title: Peugeot Project PCB		
Size: A4	Date:	**Rev: v1.0**
KiCad E.D.A. 9.0.4		Id: 4/4

P.2 Kompletan izvorni kod firmware-a na ESP32.

Kod 1: main.c

```c
#include "freertos/FreeRTOS.h"
#include <stdio.h>
#include <string.h>

#include "freertos/queue.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "include/global_config.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali_scheme.h"
#include "ulp.h"
#include "ulp_adc.h"
#include "ulp_common.h"
#include "soc/rtc_cntl_reg.h"
#include "driver/rtc_io.h"
#include "esp_sleep.h"
#include "driver/uart.h"

#include "esp_timer.h"

#include "include/mive/common.h"
#include "include/global_init.h"
#include "include/van/tss463c.h"
#include "include/van/van_packet_iden.h"
#include "include/van_structs/van_cdc_emulator_structs.h"
#include "include/van/van_packet_parser.h"
#include "include/mive/psa_packet_defs.h"

#include "include/mive/psa_helper.h"


#define ARRAY_SIZE_OFFSET   5
mive_global_state_t g_global_state = {0};
volatile int g_global_car_state = PSA_STATE_CAR_SLEEP;
volatile int g_global_ext_power_state = 1;
volatile float g_bat_voltage = 0.0f;

static const char *TAG = "MAIN";

static const float ADC_R2 = 523000;
static const float ADC_R1 = 2000000;
static const int audio_menu_duration_ms = 4000;

static int timer_num = 0;

struct psa_output_data_buffers global_libpsa_buffers;

mive_uart_task_packet_t* global_uart_send_buffers;
mive_uart_task_packet_t* global_uart_receive_buffers;

```

```
54
55  RTC_DATA_ATTR int rtc_data_valid = 0;
56  RTC_DATA_ATTR struct psa_preset_data rtc_preset_data[4];
57
58  void main_task(void* params);
59  void stats_task(void *arg);
60
61  static char* car_state_str[] = {
62      [PSA_STATE_CAR_SLEEP] = __STRINGIFY(PSA_STATE_CAR_SLEEP),
63      [PSA_STATE_ECONOMY_MODE] = __STRINGIFY(PSA_STATE_ECONOMY_MODE),
64      [PSA_STATE_CAR_LOCKED] = __STRINGIFY(PSA_STATE_CAR_LOCKED),
65      [PSA_STATE_CAR_OFF] = __STRINGIFY(PSA_STATE_CAR_OFF),
66      [PSA_STATE_RADIO_ON] = __STRINGIFY(PSA_STATE_RADIO_ON),
67      [PSA_STATE_ACCESSORY] = __STRINGIFY(PSA_STATE_ACCESSORY),
68      [PSA_STATE_IGNITION] = __STRINGIFY(PSA_STATE_IGNITION),
69      [PSA_STATE_CRANKING] = __STRINGIFY(PSA_STATE_CRANKING),
70      [PSA_STATE_ENGINE_ON] = __STRINGIFY(PSA_STATE_ENGINE_ON),
71      [PSA_STATE_UNKNOWN] = __STRINGIFY(PSA_STATE_UNKNOWN)
72  };
73
74  IRAM_ATTR static void timer_callback_f(void* user_data)
75  {
76      BaseType_t high_task_wakeup = pdFALSE;
77      QueueHandle_t main_queue = (QueueHandle_t)user_data;
78      struct mive_global_event timer_event = {
79          .ev_data = NULL,
80          .event = MIVE_EVENT_TIMER_1S,
81      };
82      xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
83      if(high_task_wakeup)
84      {
85          esp_timer_isr_dispatch_need_yield();
86      }
87  }
88
89  IRAM_ATTR static void timer_callback_audio_timeout_f(void* user_data)
90  {
91      BaseType_t high_task_wakeup = pdFALSE;
92      QueueHandle_t main_queue = (QueueHandle_t)user_data;
93      struct mive_global_event timer_event = {
94          .ev_data = NULL,
95          .event = MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,
96      };
97      xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
98      if(high_task_wakeup)
99      {
100         esp_timer_isr_dispatch_need_yield();
101     }
102 }
103
104 IRAM_ATTR static void timer_callback_oneshot_f(void* user_data)
105 {
106     BaseType_t high_task_wakeup = pdFALSE;
107     QueueHandle_t main_queue = (QueueHandle_t)user_data;
108     struct mive_global_event timer_event = {
109         .ev_data = NULL,
110         .event = MIVE_EVENT_TIMER_1MS,
111     };
```

```
112     timer_num++;
113     xQueueSendFromISR(main_queue, &timer_event, &high_task_wakeup);
114     if(high_task_wakeup)
115     {
116         esp_timer_isr_dispatch_need_yield();
117     }
118 }
119 
120 static esp_err_t print_real_time_stats(TickType_t xTicksToWait)
121 {
122     TaskStatus_t *start_array = NULL, *end_array = NULL;
123     UBaseType_t start_array_size, end_array_size;
124     configRUN_TIME_COUNTER_TYPE start_run_time, end_run_time;
125     esp_err_t ret;
126 
127     //Allocate array to store current task states
128     start_array_size = uxTaskGetNumberOfTasks() + ARRAY_SIZE_OFFSET;
129     start_array = malloc(sizeof(TaskStatus_t) * start_array_size);
130     if (start_array == NULL) {
131         ret = ESP_ERR_NO_MEM;
132         goto exit;
133     }
134     //Get current task states
135     start_array_size = uxTaskGetSystemState(start_array, start_array_size,
    ↪  &start_run_time);
136     if (start_array_size == 0) {
137         ret = ESP_ERR_INVALID_SIZE;
138         goto exit;
139     }
140 
141     vTaskDelay(xTicksToWait);
142 
143     //Allocate array to store tasks states post delay
144     end_array_size = uxTaskGetNumberOfTasks() + ARRAY_SIZE_OFFSET;
145     end_array = malloc(sizeof(TaskStatus_t) * end_array_size);
146     if (end_array == NULL) {
147         ret = ESP_ERR_NO_MEM;
148         goto exit;
149     }
150     //Get post delay task states
151     end_array_size = uxTaskGetSystemState(end_array, end_array_size, &end_run_time);
152     if (end_array_size == 0) {
153         ret = ESP_ERR_INVALID_SIZE;
154         goto exit;
155     }
156 
157     //Calculate total_elapsed_time in units of run time stats clock period.
158     uint32_t total_elapsed_time = (end_run_time - start_run_time);
159     if (total_elapsed_time == 0) {
160         ret = ESP_ERR_INVALID_STATE;
161         goto exit;
162     }
163 
164     printf("| Task | Run Time | Percentage\n");
165     //Match each task in start_array to those in the end_array
166     for (int i = 0; i < start_array_size; i++) {
167         int k = -1;
168         for (int j = 0; j < end_array_size; j++) {
```

```
            if (start_array[i].xHandle == end_array[j].xHandle) {
                k = j;
                //Mark that task have been matched by overwriting their handles
                start_array[i].xHandle = NULL;
                end_array[j].xHandle = NULL;
                break;
            }
        }
        //Check if matching task found
        if (k >= 0) {
            uint32_t task_elapsed_time = end_array[k].ulRunTimeCounter - start_array[i].ulRunTimeCounter;
            uint32_t percentage_time = (task_elapsed_time * 100UL) / (total_elapsed_time * CONFIG_FREERTOS_NUMBER_OF_CORES);
            printf("| %s | %"PRIu32" | %"PRIu32"%%\n", start_array[i].pcTaskName, task_elapsed_time, percentage_time);
        }
    }

    //Print unmatched tasks
    for (int i = 0; i < start_array_size; i++) {
        if (start_array[i].xHandle != NULL) {
            printf("| %s | Deleted\n", start_array[i].pcTaskName);
        }
    }
    for (int i = 0; i < end_array_size; i++) {
        if (end_array[i].xHandle != NULL) {
            printf("| %s | Created\n", end_array[i].pcTaskName);
        }
    }
    ret = ESP_OK;

exit:    //Common return path
    free(start_array);
    free(end_array);
    return ret;
}

/**
 * @brief Setup the ULP as a wake source to wake the ESP up
 * when the ADC reading goes above `high_adc_treshold`.
 *
 * @param high_adc_treshold
 */
void ulp_adc_wake_up(unsigned int high_adc_treshold)
{
    esp_err_t err;
    adc_oneshot_unit_handle_t adc1_handle = g_global_state.adc_handle;

    ulp_adc_cfg_t adc_cfg = {
        .adc_n = PSA_ADC_UNIT,
        .channel = PSA_ADC_CHANNEL,
        .width = ADC_BITWIDTH_12,
        .atten = ADC_ATTEN_DB_12,
        .ulp_mode = ADC_ULP_MODE_FSM
    };

    const ulp_insn_t program[] = {
```

```
        I_MOVI(R0, 0),                  // Set reg. R0 to initial 0
        I_MOVI(R2, 0),                  // Set reg. R2 to initial 0
        M_LABEL(1),                     // LABEL 1 - Start of measurement
        I_ADDI(R0, R0, 1),              // Increment cycle counter (reg. R0)
        I_ADC(R1, PSA_ADC_UNIT, PSA_ADC_CHANNEL),           // Read ADC value to reg. R1
        I_ADDR(R2, R2, R1),             // Add ADC value from reg R1 to reg. R2
        M_BL(1, 4),                     // If cycle counter is less than 4, go to LABEL 1
        I_RSHI(R0, R2, 2),              // Divide accumulated ADC value in reg. R2 by 4 and
        ↪  save it to reg. R0
        M_BGE(2, high_adc_treshold),  // If average ADC value from reg. R0 is higher or
        ↪  equal than high_adc_treshold, go to LABEL 2
        I_HALT(),                       // Halt the coprocessor, it will be woken up by the
        ↪  timer again
        M_LABEL(2),                     // LABEL 3
        I_WAKE(),                       // Wake up ESP32
        I_END(),                        // Stop ULP program timer
        I_HALT(),                       // Halt the coprocessor
    };

    adc_oneshot_del_unit(adc1_handle);

    ESP_ERROR_CHECK(ulp_adc_init(&adc_cfg));

    rtc_gpio_init(GPIO_NUM_33);


    size_t size = sizeof(program)/sizeof(ulp_insn_t);
    ESP_ERROR_CHECK(ulp_process_macros_and_load(0, program, &size));
    ulp_set_wakeup_period(0, 20000);
    err = ulp_run(0);
    ESP_ERROR_CHECK(err);
    ESP_ERROR_CHECK(esp_sleep_enable_ulp_wakeup());
}

void go_to_sleep(void)
{
    tss_sleep(&global_tss_instance);
    gpio_set_level(32, 0);
    gpio_set_level(27, 0);
    rtc_gpio_isolate(GPIO_NUM_12);
    rtc_gpio_isolate(GPIO_NUM_15);
    printf("Going to sleep\n");
    ulp_adc_wake_up(1000);
    esp_deep_sleep_start();
}

void update_ext_power_state(int power)
{
    // gpio_set_level(PSA_EXT_REG_PIN, power ? 1 : 0);
    // gpio_set_level(TSS_OE_ENABLE_PIN, power ? 1 : 0);

    g_global_ext_power_state = power;
}

void update_car_state(int new_state)
{
    if(new_state == g_global_car_state)
    {
```

```
279         return;
280     }
281 
282     ESP_LOGI(TAG, "[%s] New state = %s", __func__, car_state_str[new_state]);
283     g_global_car_state = new_state;
284     global_libpsa_buffers.status_data->car_state = g_global_car_state;
285     libpsa_send_packet(PSA_IDENT_CAR_STATUS);
286 }
287 
288 void calculate_car_state(void)
289 {
290     // ESP_LOGI(TAG, "[%s]", __func__);
291     int ext_power_status = g_global_ext_power_state;
292     int engine_state = global_libpsa_buffers.dash_data->engine_running;
293     int acc_state = global_libpsa_buffers.dash_data->accesories_on;
294     int ign_state = global_libpsa_buffers.dash_data->ignition_on;
295     int headunit_state = global_libpsa_buffers.headunit_data->unit_powered_on;
296     int lock_state = global_libpsa_buffers.status_data->doors_locked;
297 
298     // PSA_STATE_ENGINE_ON
299     if (engine_state)
300     {
301         update_car_state(PSA_STATE_ENGINE_ON);
302         ext_power_status = 1;
303     }
304     // PSA_STATE_ECONOMY_MODE
305     // else if (dash_packet.packet.data.economy_mode)
306     // {
307     //     update_car_state(PSA_STATE_ECONOMY_MODE);
308     //     ext_power_status = 0;
309     // }
310     // PSA_STATE_CRANKING
311     else if (acc_state == 0 &&
312              ign_state == 1)
313     {
314         update_car_state(PSA_STATE_CRANKING);
315         ext_power_status = 1;
316     }
317     // PSA_STATE_IGNITION
318     else if (acc_state == 1 &&
319              ign_state == 1)
320     {
321         update_car_state(PSA_STATE_IGNITION);
322         ext_power_status = 1;
323     }
324     // PSA_STATE_ACCESSORY
325     else if (acc_state == 1 &&
326              ign_state == 0)
327     {
328         update_car_state(PSA_STATE_ACCESSORY);
329         ext_power_status = 1;
330     }
331     // PSA_STATE_RADIO_ON
332     // PSA_STATE_CAR_LOCKED
333     // PSA_STATE_CAR_OFF
334     else if (acc_state == 0 &&
335              ign_state == 0)
336     {
```

```
        if (headunit_state)
        {
            update_car_state(PSA_STATE_RADIO_ON);
            ext_power_status = 1;
        }
        else if (lock_state)
        {
            update_car_state(PSA_STATE_CAR_LOCKED);
            ext_power_status = 0;
        }
        else
        {
            update_car_state(PSA_STATE_CAR_OFF);
        }
    }

    update_ext_power_state(ext_power_status);
}

void app_main(void)
{
    // To TSS task
    QueueHandle_t tss_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    // To VAN task
    QueueHandle_t van_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    // To main task
    QueueHandle_t main_queue = xQueueCreate(20, sizeof(struct mive_global_event));

    // To uart task
    QueueHandle_t uart_queue = xQueueCreate(10, sizeof(struct mive_global_event));

    g_global_state.global_tss_queue = tss_queue;
    g_global_state.global_van_queue = van_queue;
    g_global_state.global_main_queue = main_queue;
    g_global_state.global_uart_queue = uart_queue;

    init_adc();

    init_libpsa_packets();

    if(rtc_data_valid)
    {
        memcpy(global_libpsa_buffers.presets_data_am, &rtc_preset_data[PSA_PRESET_AM],
        ↪  sizeof(struct psa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_1,
        ↪  &rtc_preset_data[PSA_PRESET_FM_1], sizeof(struct psa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_2,
        ↪  &rtc_preset_data[PSA_PRESET_FM_2], sizeof(struct psa_preset_data));
        memcpy(global_libpsa_buffers.presets_data_fm_ast,
        ↪  &rtc_preset_data[PSA_PRESET_FMAST], sizeof(struct psa_preset_data));
    }

    xTaskCreatePinnedToCore(
        main_task,
        "main_task",
        10240,
```

```
391         NULL, 20, NULL, 0);
392 
393     xTaskCreatePinnedToCore(
394         tss_task,
395         "tss_task",
396         5120,
397         &g_global_state, 10, NULL, 0);
398 
399     xTaskCreatePinnedToCore(
400         van_rmt_task,
401         "van_rx_task",
402         5120,
403         &g_global_state, 15, NULL, 1);
404 
405     xTaskCreatePinnedToCore(
406         uart_task,
407         "uart_task",
408         5120,
409         &g_global_state, 15, NULL, 0);
410 
411     // xTaskCreatePinnedToCore(stats_task, "stats", 4096, NULL, 3, NULL, tskNO_AFFINITY);
412 }
413 
414 
415 void stats_task(void *params)
416 {
417     //Print real time stats periodically
418     while (1) {
419         printf("\n\nGetting real time stats over %"PRIu32" ticks\n",
        ↪  pdMS_TO_TICKS(1000));
420         if (print_real_time_stats(pdMS_TO_TICKS(1000)) == ESP_OK) {
421             printf("Real time stats obtained\n");
422         } else {
423             printf("Error getting real time stats\n");
424         }
425         vTaskDelay(pdMS_TO_TICKS(1000));
426     }
427 }
428 
429 void main_task(void* params)
430 {
431     struct mive_global_event event = {0};
432     struct mive_global_event tss_event = {0};
433     struct mive_global_event tss_trip_event = {0};
434     struct van_cd_changer_packet* cdc_packet;
435     mive_tss_task_packet_t *tss_packet = malloc(sizeof(*tss_packet));
436     mive_tss_task_packet_t *tss_trip_packet = malloc(sizeof(*tss_packet));
437     int ret = 0;
438     int count = 0;
439     int i;
440     int num_timer = 0;
441     int num_timer_ms = 0;
442     int audio_menu_open = 0;
443     int audio_menu_setting = 0;
444     QueueHandle_t main_queue = g_global_state.global_main_queue;
445     QueueHandle_t tss_queue = g_global_state.global_tss_queue;
446     QueueHandle_t van_queue = g_global_state.global_van_queue;
447     QueueHandle_t uart_queue = g_global_state.global_uart_queue;
```

```
448     adc_oneshot_unit_handle_t adc1_handle = g_global_state.adc_handle;
449     adc_cali_handle_t cali_handle = g_global_state.cali_handle;
450 
451     esp_timer_handle_t timer_handle;
452     esp_timer_handle_t timer_handle_oneshot;
453     esp_timer_handle_t timer_handle_audio_timeout;
454 
455     uint32_t ret_num = 0;
456     mive_van_packet_t* van_packet = NULL;
457     mive_uart_queue_packet_t* uart_queue_packet = NULL;
458     mive_uart_task_packet_t* uart_recv_packet = NULL;
459 
460 #if (ESP32_BOARD_TYPE == PEZO)
461     esp_rom_gpio_pad_select_gpio(PSA_EXT_REG_PIN);
462     esp_rom_gpio_pad_select_gpio(TSS_OE_ENABLE_PIN);
463 
464     gpio_set_direction(PSA_EXT_REG_PIN, GPIO_MODE_OUTPUT);
465     gpio_set_direction(TSS_OE_ENABLE_PIN, GPIO_MODE_OUTPUT);
466 
467     gpio_set_level(PSA_EXT_REG_PIN, 1);
468     gpio_set_level(TSS_OE_ENABLE_PIN, 1);
469 
470 #endif
471 
472 
473     tss_packet->iden = PSA_VAN_IDEN_CDCHANGER;
474     tss_packet->packet_size = sizeof(*cdc_packet);
475     tss_packet->message_type = TSS_IMM_REPLY;
476     tss_packet->message_channel = 2;
477 
478     tss_trip_packet->iden = PSA_VAN_IDEN_MFD_STATUS; // 0x5E4
479     tss_trip_packet->packet_size = 2;
480     tss_trip_packet->message_type = TSS_TRANSMIT;
481     tss_trip_packet->message_channel = 1;
482     tss_trip_packet->packet[0] = 0x00;
483     tss_trip_packet->packet[1] = 0xFF;
484 
485     cdc_packet = (struct van_cd_changer_packet*)tss_packet->packet;
486 
487     memset(cdc_packet, 0, sizeof(*cdc_packet));
488 
489     cdc_packet->cd_flag = 1;
490     cdc_packet->cd_num = 1;
491     cdc_packet->cd_present = VAN_CDC_CD_PRESENT;
492     cdc_packet->header = 0x80;
493     cdc_packet->footer = 0x80;
494     cdc_packet->minutes = 1;
495     cdc_packet->seconds = 1;
496     cdc_packet->shuffle = 0;
497     cdc_packet->status = VAN_CDC_ON_PLAYING;
498     cdc_packet->total_tracks = 1;
499     cdc_packet->track_num = 1;
500 
501     esp_timer_create_args_t timer_create_args = {
502         .arg = g_global_state.global_main_queue,
503         .callback = timer_callback_f,
504         .dispatch_method = ESP_TIMER_ISR,
505         .name = NULL,
```

```
        .skip_unhandled_events = true
    };

    esp_timer_create_args_t timer_create_args_oneshot = {
        .arg = g_global_state.global_main_queue,
        .callback = timer_callback_oneshot_f,
        .dispatch_method = ESP_TIMER_ISR,
        .name = NULL,
        .skip_unhandled_events = true
    };

    esp_timer_create_args_t timer_create_args_audio_menu = {
        .arg = g_global_state.global_main_queue,
        .callback = timer_callback_audio_timeout_f,
        .dispatch_method = ESP_TIMER_ISR,
        .name = NULL,
        .skip_unhandled_events = true
    };

    esp_timer_create(&timer_create_args, &timer_handle);
    esp_timer_create(&timer_create_args_oneshot, &timer_handle_oneshot);
    esp_timer_create(&timer_create_args_audio_menu, &timer_handle_audio_timeout);
    esp_timer_start_periodic(timer_handle, 1000000);
    esp_timer_start_once(timer_handle_oneshot, 1000000);

    int adc_raw;
    int voltage_in;
    float voltage_out;
    while(1)
    {
        // vTaskDelay(1000 / portTICK_PERIOD_MS);

        ret = xQueueReceive(main_queue, &event, pdMS_TO_TICKS(1000));

        if(ret == pdPASS)
        {
            // printf("[%s] Received queue item\n", __func__);

            switch (event.event)
            {
            case MIVE_EVENT_RMT_NEW_VAN_FRAME:
                {
                    int stat_idx = 0;
                    van_packet = (mive_van_packet_t*)event.ev_data;

                    ret = psa_parse_van_packet(
                        van_packet->iden,
                        van_packet->packet_size,
                        van_packet->packet,
                        &global_libpsa_buffers);

                    if (ret != MIVE_OK)
                    {
                        ESP_LOGE(TAG, "Error parsing packet 0x%03x", van_packet->iden);
                    }

                    // calculate_car_state();
                    // printf("[%s] Received: 0x%3x ", __func__, van_packet->iden);
```

```
                    // for (int i = 0; i < van_packet->packet_size; i++)
                    // {
                    //     printf("%02x ", van_packet->packet[i]);
                    // }
                    // printf("\n");
                }

                vTaskDelay(pdMS_TO_TICKS(10));
                break;

            case MIVE_EVENT_TIMER_1S:
                {
                    uint8_t temp_val = cdc_packet->header;
                    temp_val = (temp_val >= 0x87) ? 0x80 : temp_val + 1;
                    cdc_packet->header = temp_val;
                    cdc_packet->footer = temp_val;
                    tss_event.event = MIVE_EVENT_TSS_WRITE_FRAME;
                    tss_event.ev_data = tss_packet;
                    xQueueSendToBack(tss_queue, &tss_event, pdMS_TO_TICKS(100));
                }
                break;
            case MIVE_EVENT_TIMER_1MS:

                ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, ADC_CHANNEL_5, &adc_raw));
                ESP_ERROR_CHECK(adc_cali_raw_to_voltage(cali_handle, adc_raw,
                ↪  &voltage_in));

                g_bat_voltage = (voltage_in * 0.001f * (ADC_R1 + ADC_R2)) / ADC_R2;
                global_libpsa_buffers.status_data->voltage = (uint16_t)(g_bat_voltage *
                ↪  1000);
                libpsa_send_packet(PSA_IDENT_CAR_STATUS);
                esp_timer_start_once(timer_handle_oneshot, 50000);

                if (unlikely(g_bat_voltage < 1))
                {
                    struct mive_global_event sleep_event = {
                        .ev_data = NULL,
                        .event = MIVE_EVENT_GLOBAL_SLEEP
                    };

                    xQueueSendToFront(main_queue, &sleep_event, 0);
                }

                break;
            case MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM:
                if(!audio_menu_open)
                {
                    esp_timer_start_once(timer_handle_audio_timeout,
                    ↪  audio_menu_duration_ms * 1000);
                }
                if (audio_menu_setting >= 6)
                {
                    audio_menu_open = 0;
                    audio_menu_setting = 0;
                    esp_timer_stop(timer_handle_audio_timeout);
                }
                else
                {
```

```
619                     audio_menu_open = 1;
620                     audio_menu_setting++;
621                     esp_timer_restart(timer_handle_audio_timeout, audio_menu_duration_ms
                        ↪  * 1000);
622                 }
623                 ESP_LOGI(TAG, "[MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM] menu_open: %d,
                    ↪  menu_setting: %d" , audio_menu_open, audio_menu_setting);
624                 libpsa_update_audio_settings_packet(audio_menu_open, audio_menu_setting);
625                 libpsa_send_packet(PSA_IDENT_HEADUNIT);
626                 break;
627             case MIVE_EVENT_VAN_UPDATE_AUDIO_MENU:
628                 if(audio_menu_open)
629                 {
630                     esp_timer_restart(timer_handle_audio_timeout, audio_menu_duration_ms
                        ↪  * 1000);
631                     ESP_LOGI(TAG, "[MIVE_EVENT_VAN_UPDATE_AUDIO_MENU] menu_open: %d,
                        ↪  menu_setting: %d" , audio_menu_open, audio_menu_setting);
632                     libpsa_update_audio_settings_packet(audio_menu_open,
                        ↪  audio_menu_setting);
633                     libpsa_send_packet(PSA_IDENT_HEADUNIT);
634                 }
635                 break;
636             case MIVE_EVENT_VAN_CLOSE_AUDIO_MENU:
637                 audio_menu_open = 0;
638                 audio_menu_setting = 0;
639                 ESP_LOGI(TAG, "[MIVE_EVENT_VAN_CLOSE_AUDIO_MENU] menu_open: %d,
                    ↪  menu_setting: %d" , audio_menu_open, audio_menu_setting);
640                 libpsa_update_audio_settings_packet(audio_menu_open, audio_menu_setting);
641                 libpsa_send_packet(PSA_IDENT_HEADUNIT);
642                 break;
643             case MIVE_EVENT_UART_NEW_DATA:
644                 uart_queue_packet = (mive_uart_queue_packet_t*)event.ev_data;
645
646                 for(i = 0; i < uart_queue_packet->num_idens; ++i)
647                 {
648                     libpsa_send_packet(uart_queue_packet->idens[i]);
649                 }
650                 break;
651             case MIVE_EVENT_UART_RECEIVE:
652                 uart_recv_packet = (mive_uart_task_packet_t*)event.ev_data;
653
654                 if(uart_recv_packet->iden >= PSA_MSP_MIN_IDENT && uart_recv_packet->iden
                    ↪  < PSA_IDENT_SET_CD_CHANGER_DATA)
655                 {
656                     ESP_LOGI(TAG, "Sending iden 0x%04x", uart_recv_packet->iden);
657                     libpsa_send_packet(uart_recv_packet->iden);
658                 } else if(uart_recv_packet->iden == PSA_IDENT_SET_TRIP_RESET)
659                 {
660                     ESP_LOGI(TAG, "Sending Trip reset");
661                     struct psa_trip_reset_data* data = (struct
                        ↪  psa_trip_reset_data*)uart_recv_packet->data;
662                     tss_trip_event.event = MIVE_EVENT_TSS_WRITE_FRAME;
663                     tss_trip_event.ev_data = tss_trip_packet;
664                     tss_trip_packet->packet_size = 2;
665
666                     switch (data->trip_meter)
667                     {
668                     case PSA_TRIP_A:
```

```
                        tss_trip_packet->packet[0] = 0xA0;
                        xQueueSendToBack(tss_queue, &tss_trip_event, 0);
                        break;
                    case PSA_TRIP_B:
                        tss_trip_packet->packet[0] = 0x60;
                        xQueueSendToBack(tss_queue, &tss_trip_event, 0);
                        break;
                    default:
                        break;
                    }
                }

                break;
            case MIVE_EVENT_GLOBAL_SLEEP:
                memcpy(&rtc_preset_data[PSA_PRESET_AM],
                ↪  global_libpsa_buffers.presets_data_am, sizeof(struct
                ↪  psa_preset_data));
                memcpy(&rtc_preset_data[PSA_PRESET_FM_1],
                ↪  global_libpsa_buffers.presets_data_fm_1, sizeof(struct
                ↪  psa_preset_data));
                memcpy(&rtc_preset_data[PSA_PRESET_FM_2],
                ↪  global_libpsa_buffers.presets_data_fm_2, sizeof(struct
                ↪  psa_preset_data));
                memcpy(&rtc_preset_data[PSA_PRESET_FMAST],
                ↪  global_libpsa_buffers.presets_data_fm_ast, sizeof(struct
                ↪  psa_preset_data));
                rtc_data_valid = 1;
                go_to_sleep();
                break;
            default:
                break;
            }

        }
        calculate_car_state();
    }
}
```

Kod 2: common.c

```
#include <stdint.h>

#include "include/mive/common.h"
#include "esp_attr.h"

static const unsigned char crc8tab[256] = {
  0x00, 0xD5, 0x7F, 0xAA, 0xFE, 0x2B, 0x81, 0x54,
  0x29, 0xFC, 0x56, 0x83, 0xD7, 0x02, 0xA8, 0x7D,
  0x52, 0x87, 0x2D, 0xF8, 0xAC, 0x79, 0xD3, 0x06,
  0x7B, 0xAE, 0x04, 0xD1, 0x85, 0x50, 0xFA, 0x2F,
  0xA4, 0x71, 0xDB, 0x0E, 0x5A, 0x8F, 0x25, 0xF0,
  0x8D, 0x58, 0xF2, 0x27, 0x73, 0xA6, 0x0C, 0xD9,
  0xF6, 0x23, 0x89, 0x5C, 0x08, 0xDD, 0x77, 0xA2,
  0xDF, 0x0A, 0xA0, 0x75, 0x21, 0xF4, 0x5E, 0x8B,
  0x9D, 0x48, 0xE2, 0x37, 0x63, 0xB6, 0x1C, 0xC9,
  0xB4, 0x61, 0xCB, 0x1E, 0x4A, 0x9F, 0x35, 0xE0,
```

```
  0xCF, 0x1A, 0xB0, 0x65, 0x31, 0xE4, 0x4E, 0x9B,
  0xE6, 0x33, 0x99, 0x4C, 0x18, 0xCD, 0x67, 0xB2,
  0x39, 0xEC, 0x46, 0x93, 0xC7, 0x12, 0xB8, 0x6D,
  0x10, 0xC5, 0x6F, 0xBA, 0xEE, 0x3B, 0x91, 0x44,
  0x6B, 0xBE, 0x14, 0xC1, 0x95, 0x40, 0xEA, 0x3F,
  0x42, 0x97, 0x3D, 0xE8, 0xBC, 0x69, 0xC3, 0x16,
  0xEF, 0x3A, 0x90, 0x45, 0x11, 0xC4, 0x6E, 0xBB,
  0xC6, 0x13, 0xB9, 0x6C, 0x38, 0xED, 0x47, 0x92,
  0xBD, 0x68, 0xC2, 0x17, 0x43, 0x96, 0x3C, 0xE9,
  0x94, 0x41, 0xEB, 0x3E, 0x6A, 0xBF, 0x15, 0xC0,
  0x4B, 0x9E, 0x34, 0xE1, 0xB5, 0x60, 0xCA, 0x1F,
  0x62, 0xB7, 0x1D, 0xC8, 0x9C, 0x49, 0xE3, 0x36,
  0x19, 0xCC, 0x66, 0xB3, 0xE7, 0x32, 0x98, 0x4D,
  0x30, 0xE5, 0x4F, 0x9A, 0xCE, 0x1B, 0xB1, 0x64,
  0x72, 0xA7, 0x0D, 0xD8, 0x8C, 0x59, 0xF3, 0x26,
  0x5B, 0x8E, 0x24, 0xF1, 0xA5, 0x70, 0xDA, 0x0F,
  0x20, 0xF5, 0x5F, 0x8A, 0xDE, 0x0B, 0xA1, 0x74,
  0x09, 0xDC, 0x76, 0xA3, 0xF7, 0x22, 0x88, 0x5D,
  0xD6, 0x03, 0xA9, 0x7C, 0x28, 0xFD, 0x57, 0x82,
  0xFF, 0x2A, 0x80, 0x55, 0x01, 0xD4, 0x7E, 0xAB,
  0x84, 0x51, 0xFB, 0x2E, 0x7A, 0xAF, 0x05, 0xD0,
  0xAD, 0x78, 0xD2, 0x07, 0x53, 0x86, 0x2C, 0xF9
};

/* Rounds a number to the nearest multiple */
IRAM_ATTR uint8_t round_to_nearest(uint8_t numToRound, uint8_t multiple)
{
    if (multiple == 0)
        return numToRound;

    uint8_t remainder = numToRound % multiple;

    // If the remainder is greater than half of the multiple, then round up. Otherwise,
    ↪  round down.
    if (remainder > multiple / 2)
        return numToRound + multiple - remainder;
    else
        return numToRound - remainder;
}

IRAM_ATTR uint16_t get_iden_from_bytes(uint8_t const byte1, uint8_t const byte2)
{
    uint16_t const iden = ((uint16_t)byte1 << 8) | (byte2 & 0xf0);
    return iden >> 4;
}

uint8_t dec_to_bcd(uint8_t input)
{
    if (input == 0xff)
    {
        return 0xff;
    }
    else if (input > 99)
    {
        return 0x99;
    }
    else
    {
```

```
74          return ((input / 10 * 16) + (input % 10));
75      }
76  }
77
78  uint8_t bcd_to_dec(uint8_t value)
79  {
80      uint8_t ms_nibble = ((value & 0xF0) >> 4);
81      uint8_t ls_nibble = (value & 0x0F);
82      if (ms_nibble >= 10 || ls_nibble >= 10)
83      {
84          return value;
85      }
86      else
87      {
88          return ms_nibble * 10 + ls_nibble;
89      }
90  }
91
92
93  uint8_t psa_crc8_checksum(const uint8_t * ptr, uint32_t len)
94  {
95    uint8_t crc = 0;
96    for (uint32_t i = 0; i < len; i++) {
97      crc = crc8tab[crc ^ *ptr++];
98    }
99    return crc;
100 }
```

Kod 3: init.c

```
1   #include "freertos/FreeRTOS.h"
2   #include <stdint.h>
3
4   #include "esp_adc/adc_oneshot.h"
5   #include "esp_adc/adc_continuous.h"
6   #include "esp_adc/adc_cali_scheme.h"
7
8   #include "include/global_init.h"
9   #include "include/mive/common.h"
10  #include "include/global_config.h"
11  #include "include/global_state_def.h"
12
13  #include "soc/soc_caps.h"
14
15
16  void init_adc()
17  {
18      adc_oneshot_unit_handle_t adc1_handle;
19      adc_cali_handle_t cali_handle;
20
21      adc_oneshot_unit_init_cfg_t init_config1 = {
22          .unit_id = PSA_ADC_UNIT,
23          .ulp_mode = ADC_ULP_MODE_DISABLE,
24      };
25
```

```
    adc_oneshot_chan_cfg_t config = {
        .bitwidth = ADC_BITWIDTH_12,
        .atten = ADC_ATTEN_DB_12,
    };

    adc_cali_line_fitting_config_t lf_config = {
        .atten = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_12,
        .default_vref = 0,
        .unit_id = PSA_ADC_UNIT
    };

    //ESP_ERROR_CHECK(adc_continuous_new_handle(&adc_config, &adc_c_handle));

    ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config1, &adc1_handle));

    ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL_5, &config));

    ESP_ERROR_CHECK(adc_cali_create_scheme_line_fitting(&lf_config, &cali_handle));

    g_global_state.adc_handle = adc1_handle;
    g_global_state.cali_handle = cali_handle;
}
```

Kod 4: psa_helper.c

```
#include "freertos/FreeRTOS.h"

#include <stdint.h>
#include <string.h>

#include "include/mive/psa_helper.h"
#include "include/van/van_packet_parser.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"

static int uart_send_buffer_num = 0;

static mive_uart_task_packet_t* get_uart_send_buffer(void)
{
    mive_uart_task_packet_t* to_ret = &global_uart_send_buffers[uart_send_buffer_num];

    uart_send_buffer_num = (uart_send_buffer_num >= (PSA_MAIN_UART_SEND_BUFFERS_NUM - 1))
    ↪  ? 0 : uart_send_buffer_num + 1;

    return to_ret;
}

void libpsa_send_packet(uint16_t ident)
{
    mive_uart_task_packet_t* packet = get_uart_send_buffer();
    struct mive_global_event global_event = {
        .event = MIVE_EVENT_UART_SEND,
        .ev_data = packet,
    };
```

```
29
30      switch (ident)
31      {
32      case PSA_IDENT_ENGINE:
33          packet->data_size = sizeof(*global_libpsa_buffers.engine_data);
34          memcpy(
35              packet->data,
36              global_libpsa_buffers.engine_data,
37              packet->data_size);
38          packet->iden = ident;
39          break;
40      case PSA_IDENT_RADIO:
41          packet->data_size = sizeof(*global_libpsa_buffers.radio_data);
42          memcpy(
43              packet->data,
44              global_libpsa_buffers.radio_data,
45              packet->data_size);
46          packet->iden = ident;
47          break;
48      case PSA_IDENT_VIN:
49          packet->data_size = 17;
50          memcpy(
51              packet->data,
52              global_libpsa_buffers.vin_data,
53              packet->data_size);
54          packet->iden = ident;
55          break;
56      case PSA_IDENT_RADIO_PRESETS:
57          packet->data_size = sizeof(*global_libpsa_buffers.presets_data_am);
58          switch(global_libpsa_buffers.radio_data->band)
59          {
60              case PSA_AM_1:
61                  memcpy(
62                      packet->data,
63                      global_libpsa_buffers.presets_data_am,
64                      packet->data_size);
65                  break;
66              case PSA_FM_1:
67                  memcpy(
68                      packet->data,
69                      global_libpsa_buffers.presets_data_fm_1,
70                      packet->data_size);
71                  break;
72              case PSA_FM_2:
73                  memcpy(
74                      packet->data,
75                      global_libpsa_buffers.presets_data_fm_2,
76                      packet->data_size);
77                  break;
78              case PSA_FM_AST:
79                  memcpy(
80                      packet->data,
81                      global_libpsa_buffers.presets_data_fm_ast,
82                      packet->data_size);
83                  break;
84              default:
85                  memset(packet->data, 0, packet->data_size);
86                  break;
```

```
87          }
88          packet->iden = ident;
89          break;
90      case PSA_IDENT_CAR_STATUS:
91          packet->data_size = sizeof(*global_libpsa_buffers.status_data);
92          memcpy(
93              packet->data,
94              global_libpsa_buffers.status_data,
95              packet->data_size);
96          packet->iden = ident;
97          global_libpsa_buffers.status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NONE;
98          break;
99      case PSA_IDENT_HEADUNIT:
100         packet->data_size = sizeof(*global_libpsa_buffers.headunit_data);
101         memcpy(
102             packet->data,
103             global_libpsa_buffers.headunit_data,
104             packet->data_size
105         );
106         packet->iden = ident;
107         break;
108     case PSA_IDENT_CD_PLAYER:
109         packet->data_size = sizeof(*global_libpsa_buffers.cd_player_data);
110         memcpy(
111             packet->data,
112             global_libpsa_buffers.cd_player_data,
113             packet->data_size
114         );
115         packet->iden = ident;
116         break;
117     case PSA_IDENT_DASHBOARD:
118         packet->data_size = sizeof(*global_libpsa_buffers.dash_data);
119         memcpy(
120             packet->data,
121             global_libpsa_buffers.dash_data,
122             packet->data_size
123         );
124         packet->iden = ident;
125         break;
126     case PSA_IDENT_TRIP:
127         packet->data_size = sizeof(*global_libpsa_buffers.trip_data);
128         memcpy(
129             packet->data,
130             global_libpsa_buffers.trip_data,
131             packet->data_size
132         );
133         packet->iden = ident;
134         break;
135     case PSA_IDENT_DOORS:
136         packet->data_size = sizeof(*global_libpsa_buffers.door_data);
137         memcpy(
138             packet->data,
139             global_libpsa_buffers.door_data,
140             packet->data_size
141         );
142         packet->iden = ident;
143         break;
144     default:
```

```
145         packet->iden = 0;
146         packet->data_size = 0;
147         break;
148     }
149 
150     xQueueSendToBack(g_global_state.global_uart_queue, &global_event, 0);
151 }
152 
153 void init_libpsa_packets(void)
154 {
155     memset(&global_libpsa_buffers, 0, sizeof(global_libpsa_buffers));
156 
157     global_libpsa_buffers.engine_data = calloc(1, sizeof(struct psa_engine_data));
158     global_libpsa_buffers.radio_data = calloc(1, sizeof(struct psa_radio_data));
159     global_libpsa_buffers.presets_data_am = calloc(1, sizeof(struct psa_preset_data));
160     global_libpsa_buffers.presets_data_fm_1 = calloc(1, sizeof(struct psa_preset_data));
161     global_libpsa_buffers.presets_data_fm_2 = calloc(1, sizeof(struct psa_preset_data));
162     global_libpsa_buffers.presets_data_fm_ast = calloc(1, sizeof(struct
    ↪  psa_preset_data));
163     global_libpsa_buffers.headunit_data = calloc(1, sizeof(struct psa_headunit_data));
164     global_libpsa_buffers.cd_player_data = calloc(1, sizeof(struct psa_cd_player_data));
165     global_libpsa_buffers.vin_data = calloc(1, 17);
166     global_libpsa_buffers.dash_data = calloc(1, sizeof(struct psa_dash_data));
167     global_libpsa_buffers.door_data = calloc(1, sizeof(struct psa_door_data));
168     global_libpsa_buffers.trip_data = calloc(1, sizeof(struct psa_trip_data));
169     global_libpsa_buffers.status_data = calloc(1, sizeof(struct psa_status_data));
170 
171     global_uart_send_buffers = calloc(PSA_MAIN_UART_SEND_BUFFERS_NUM,
    ↪  sizeof(*global_uart_send_buffers));
172     global_uart_receive_buffers = calloc(PSA_MAIN_UART_SEND_BUFFERS_NUM,
    ↪  sizeof(*global_uart_send_buffers));
173 }
174 
175 void libpsa_update_audio_settings_packet(
176     int audio_menu_open,
177     int audio_menu_setting)
178 {
179     struct psa_headunit_data *audio_settings_output_buffer =
    ↪  global_libpsa_buffers.headunit_data;
180     audio_settings_output_buffer->settings_menu_open = audio_menu_open;
181     switch (audio_menu_setting)
182     {
183     case 0:
184 
185         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
186         audio_settings_output_buffer->settings_changing.balance_changing = 0;
187         audio_settings_output_buffer->settings_changing.bass_changing = 0;
188         audio_settings_output_buffer->settings_changing.fader_changing = 0;
189         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
190         audio_settings_output_buffer->settings_changing.treble_changing = 0;
191         audio_settings_output_buffer->settings_changing.volume_changing = 0;
192 
193         break;
194     case 1: // Bass
195         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
196         audio_settings_output_buffer->settings_changing.balance_changing = 0;
197         audio_settings_output_buffer->settings_changing.bass_changing = 1;
198         audio_settings_output_buffer->settings_changing.fader_changing = 0;
```

```
199         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
200         audio_settings_output_buffer->settings_changing.treble_changing = 0;
201         audio_settings_output_buffer->settings_changing.volume_changing = 0;
202         break;
203 
204     case 2: // Treble
205         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
206         audio_settings_output_buffer->settings_changing.balance_changing = 0;
207         audio_settings_output_buffer->settings_changing.bass_changing = 0;
208         audio_settings_output_buffer->settings_changing.fader_changing = 0;
209         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
210         audio_settings_output_buffer->settings_changing.treble_changing = 1;
211         audio_settings_output_buffer->settings_changing.volume_changing = 1;
212         break;
213 
214     case 3: // Loudness
215         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
216         audio_settings_output_buffer->settings_changing.balance_changing = 0;
217         audio_settings_output_buffer->settings_changing.bass_changing = 0;
218         audio_settings_output_buffer->settings_changing.fader_changing = 0;
219         audio_settings_output_buffer->settings_changing.loudness_changing = 1;
220         audio_settings_output_buffer->settings_changing.treble_changing = 0;
221         audio_settings_output_buffer->settings_changing.volume_changing = 0;
222         break;
223 
224     case 4: // Fader
225         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
226         audio_settings_output_buffer->settings_changing.balance_changing = 0;
227         audio_settings_output_buffer->settings_changing.bass_changing = 0;
228         audio_settings_output_buffer->settings_changing.fader_changing = 1;
229         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
230         audio_settings_output_buffer->settings_changing.treble_changing = 0;
231         audio_settings_output_buffer->settings_changing.volume_changing = 0;
232         break;
233     case 5: // Balance
234 
235         audio_settings_output_buffer->settings_changing.auto_volume_changing = 0;
236         audio_settings_output_buffer->settings_changing.balance_changing = 1;
237         audio_settings_output_buffer->settings_changing.bass_changing = 0;
238         audio_settings_output_buffer->settings_changing.fader_changing = 0;
239         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
240         audio_settings_output_buffer->settings_changing.treble_changing = 0;
241         audio_settings_output_buffer->settings_changing.volume_changing = 0;
242         break;
243     case 6: // Auto-volume
244         audio_settings_output_buffer->settings_changing.auto_volume_changing = 1;
245         audio_settings_output_buffer->settings_changing.balance_changing = 0;
246         audio_settings_output_buffer->settings_changing.bass_changing = 0;
247         audio_settings_output_buffer->settings_changing.fader_changing = 0;
248         audio_settings_output_buffer->settings_changing.loudness_changing = 0;
249         audio_settings_output_buffer->settings_changing.treble_changing = 0;
250         audio_settings_output_buffer->settings_changing.volume_changing = 0;
251         break;
252     default:
253         break;
254     }
255 }
```

Kod 5: tss_task.c

```c
#include "freertos/FreeRTOS.h"

#include <string.h>
#include <stdlib.h>

#include "esp_attr.h"
#include "freertos/queue.h"

#include "include/mive/common.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "include/global_config.h"

#include "include/van/tss463c.h"
#include "include/van/tss463_regs.h"
#include "include/van/van_packet_iden.h"
#include "esp_log.h"

tss_instance_t global_tss_instance = {0};

static const char *TAG = "tss_task";

enum tss_transmission_status
{
    MIVE_TSS_TX_DONE = 0,
    MIVE_TSS_TX_INPROGRESS,
    MIVE_TSS_TX_FAIL,
};

static uint8_t tss_get_memory_addr(uint16_t iden)
{
    switch (iden)
    {
    case PSA_VAN_IDEN_MFD_STATUS:
        return 00;
        break;
    case PSA_VAN_IDEN_CDCHANGER:
        return 10;
        break;
    default:
        return 90;
        break;
    }
}

static uint8_t tss_get_channel_to_use(uint16_t iden)
{
    switch (iden)
    {
    case PSA_VAN_IDEN_MFD_STATUS:
        return 1;
        break;
    case PSA_VAN_IDEN_CDCHANGER:
        return 2;
        break;
```

```
    default:
        return 7;
        break;
    }
}

static int tss_send_frame(tss_instance_t* instance, mive_tss_task_packet_t* packet)
{
    int retval = MIVE_OK;
    uint16_t iden = packet->iden;
    uint8_t memory_addr = tss_get_memory_addr(iden);
    uint8_t channel = tss_get_channel_to_use(iden);
    enum tss_message_type msg_type = packet->message_type;

    switch (msg_type)
    {
    case TSS_TRANSMIT_NOACK:
        retval = tss_transmit_message(
            instance, channel, iden, packet->packet, packet->packet_size,
            memory_addr, 0);
        break;

    case TSS_TRANSMIT:
        retval = tss_transmit_message(
            instance, channel, iden, packet->packet, packet->packet_size,
            memory_addr, 1);
        break;

    case TSS_IMM_REPLY:
        retval = tss_immediate_reply_message(
            instance, channel, iden, packet->packet, packet->packet_size,
            memory_addr);
        break;

    case TSS_DEF_REPLY:
    case TSS_REPLY_REQUEST:
        retval = -MIVE_ERR_NOT_IMPLEMENTED;
        break;
    case TSS_NONE:
    default:
        retval = -MIVE_ERR_INVALID_ARGUMENT;
        break;
    }

    ESP_LOGD(TAG, "[%s] Retval = %d", __func__, retval);

    if(retval == MIVE_OK)
    {
        packet->message_channel = channel;
    }
    else if(retval == -MIVE_ERR_CHANNEL_BUSY)
    {
        // Do nothing
        retval = MIVE_OK;
    }
    else
    {
        packet->message_channel = 0xff;
```

```
    }

    return retval;
}

static int tss_check_channel_status(tss_instance_t* instance, mive_tss_task_packet_t*
↪  packet)
{
    int retval = -MIVE_ERR_INVALID_ARGUMENT;

    uint8_t channel = packet->message_channel;
    message_length_and_status_register_t reg_value = {0};

    reg_value.Value = tss_get_channel_status_register(instance, channel);

    switch (packet->message_type)
    {
    case TSS_TRANSMIT:
    case TSS_TRANSMIT_NOACK:
    case TSS_IMM_REPLY:
    case TSS_DEF_REPLY:
        if(reg_value.data.CHTx)
        {
            retval = MIVE_TSS_TX_DONE;
        }
        else
        {
            retval = MIVE_TSS_TX_INPROGRESS;
        }
        break;
    case TSS_REPLY_REQUEST:
        if(reg_value.data.CHRx)
        {
            retval = MIVE_TSS_TX_DONE;
        }
        else
        {
            retval = MIVE_TSS_TX_INPROGRESS;
        }
        break;
    default:
        retval = -MIVE_ERR_INVALID_ARGUMENT;
        break;
    }

    if(retval == MIVE_TSS_TX_DONE)
    {
        tss_free_channel(instance, channel);
    }

    return retval;
}

void tss_task(void* params)
{
    int task_running = true;
    int ret = 0;
    mive_global_state_t* state = (mive_global_state_t*) params;
```

```
171     tss_instance_t* instance = &global_tss_instance;
172     struct mive_global_event event_data = {0};
173     mive_tss_task_packet_t *task_packet = NULL;
174     QueueHandle_t tss_queue = state->global_tss_queue;
175 
176     tss_create(instance);
177 
178     tss_start(instance);
179 
180     while (task_running)
181     {
182         ret = xQueueReceive(tss_queue, &event_data, pdMS_TO_TICKS(100));
183         if(ret == pdPASS)
184         {
185             ESP_LOGD(TAG, "[%s] Got event from queue: %d", __func__, event_data.event);
186             switch (event_data.event)
187             {
188             case MIVE_EVENT_TSS_WRITE_FRAME:
189                 ret = tss_send_frame(instance,
                    ↪  (mive_tss_task_packet_t*)event_data.ev_data);
190                 // if(ret == MIVE_OK)
191                 // {
192                 //     event_data.event = MIVE_EVENT_TSS_MONITOR_CHANNEL;
193                 //     xQueueSendToBack(tss_queue, &event_data, 0);
194                 // }
195                 break;
196 
197 
198             case MIVE_EVENT_TSS_MONITOR_CHANNEL:
199                 ret = tss_check_channel_status(instance,
                    ↪  (mive_tss_task_packet_t*)event_data.ev_data);
200                 if(ret == MIVE_TSS_TX_INPROGRESS)
201                 {
202                     // Send back to queue
203                     xQueueSendToBack(tss_queue, &event_data, 0);
204                 }
205                 break;
206 
207             case MIVE_EVENT_TSS_RESET:
208                 tss_start(instance);
209                 break;
210 
211             case MIVE_EVENT_TSS_ACTIVATE:
212                 tss_activate(instance);
213                 break;
214 
215             case MIVE_EVENT_TSS_IDLE:
216                 tss_idle(instance);
217                 break;
218 
219             case MIVE_EVENT_TSS_SLEEP:
220                 tss_sleep(instance);
221                 break;
222             default:
223                 break;
224             }
225         }
226     }
```

```
227 }
```

Kod 6: tss463c.c

```
1  #include "freertos/FreeRTOS.h"
2
3  #include <unistd.h>
4  #include <string.h>
5  #include <stdbool.h>
6
7  #include "include/van/tss463_regs.h"
8  #include "include/van/tss463c.h"
9  #include "esp_log.h"
10
11 #include "include/mive/common.h"
12
13 #include "driver/spi_master.h"
14 #include "driver/gpio.h"
15
16 #include "include/global_config.h"
17
18 static const char *TAG = "tss463c";
19
20 static portMUX_TYPE tss_spinlock = portMUX_INITIALIZER_UNLOCKED;
21
22 #define get_bytes_from_iden(iden, byte1, byte2) \
23     byte1 = (uint8_t)(((iden << 4) & 0xff00) >> 8); \
24     byte2 = (uint8_t)(iden & 0xF);
25
26 IRAM_ATTR static void pre_transaction_cb(spi_transaction_t *trans)
27 {
28     return;
29 }
30
31 IRAM_ATTR static void post_transaction_cb(spi_transaction_t *trans)
32 {
33     return;
34 }
35
36 // IRAM_ATTR static void get_bytes_from_iden(uint16_t iden, uint8_t* byte1, uint8_t*
    ↪  byte2)
37 // {
38 //     *byte1 = (uint8_t)(((iden << 4) & 0xff00) >> 8);
39 //     *byte2 = (uint8_t)(iden & 0xF);
40 // }
41
42 IRAM_ATTR static int is_channel_valid(tss_instance_t* instance, uint8_t channel, uint16_t
    ↪  iden)
43 {
44     tss_channel_t *chan = NULL;
45
46     if(channel > TSS_MAX_CHANNEL)
47     {
48         return -MIVE_ERR_INVALID_ARGUMENT;
49     }
```

```
50
51      // chan = &instance->channels[channel];
52
53      // if((!chan->is_busy) && ((chan->is_occupied == 0) || (chan->is_occupied &&
        ↪  chan->identifier == iden)))
54      // {
55      //     return MIVE_OK;
56      // }
57
58      // return -MIVE_ERR_CHANNEL_BUSY;
59      return MIVE_OK;
60  }
61
62  static int tss_spi_master_init(tss_instance_t *instance)
63  {
64      spi_device_handle_t tss_handle;
65
66      #if (ESP32_BOARD_TYPE == DEVKIT)
67
68      spi_bus_config_t spi_config = {
69          .mosi_io_num = 23,
70          .miso_io_num = 19,
71          .sclk_io_num = 18,
72          .max_transfer_sz = 0,
73          .data2_io_num = -1,
74          .data3_io_num = -1,
75          .data4_io_num = -1,
76          .data5_io_num = -1,
77          .data6_io_num = -1,
78          .data7_io_num = -1
79      };
80
81      #else
82
83      spi_bus_config_t spi_config = {
84          .mosi_io_num = 13,
85          .miso_io_num = 12,
86          .sclk_io_num = 14,
87          .max_transfer_sz = 0,
88          .data2_io_num = -1,
89          .data3_io_num = -1,
90          .data4_io_num = -1,
91          .data5_io_num = -1,
92          .data6_io_num = -1,
93          .data7_io_num = -1
94      };
95
96      #endif
97
98      spi_device_interface_config_t device_config = {
99          .clock_speed_hz = 4000000,
100         .mode = 3,
101         .spics_io_num = -1,
102         .queue_size = 1,
103         .pre_cb = pre_transaction_cb,
104         .post_cb = post_transaction_cb,
105
106     };
```

```
107
108     ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &spi_config, SPI_DMA_CH_AUTO));
109
110     ESP_ERROR_CHECK(spi_bus_add_device(SPI2_HOST, &device_config, &tss_handle));
111
112     instance->tss_handle = tss_handle;
113
114     return 0;
115 }
116
117 /**
118  * @brief Performs an Asynchronous Reset and puts the TSS into motorolla mode
119  *
120  * @param instance
121  * @return int
122  */
123 static int tss_motorolla_mode(tss_instance_t *instance)
124 {
125     int return_val = 0;
126     // gpio_set_level(instance->cs_pin, 0);
127     spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
128     TSS_SELECT();
129
130     usleep(1);
131     spi_transaction_t transaction;
132     memset(&transaction, 0, sizeof(transaction));
133     transaction.user = instance;
134     transaction.tx_data[0] = 0x00;
135     transaction.length = 8;
136     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
137
138     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
139
140     if (transaction.rx_data[0] != 0xAA)
141     {
142         printf("Error setting motorolla mode: Invalid response 0x%02X / 0xAA\n",
            ↪  transaction.rx_data[0]);
143         return_val = 1;
144         // goto error;
145     }
146
147     usleep(4);
148
149     transaction.user = instance;
150     transaction.tx_data[0] = 0x00;
151     transaction.length = 8;
152     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
153
154     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
155
156     if (transaction.rx_data[0] != 0x55)
157     {
158         printf("Error setting motorolla mode: Invalid response 0x%02X / 0x55\n",
            ↪  transaction.rx_data[0]);
159         return_val = 1;
160         // goto error;
161     }
162
```

```
163     usleep(2);
164 
165     // error:
166     // gpio_set_level(instance->cs_pin, 1);
167     TSS_DESELECT();
168     spi_device_release_bus(instance->tss_handle);
169 
170     instance->chip_mode = TSS_MODE_IDLE;
171 
172     return return_val;
173 }
174 
175 static int tss_disable_channel(tss_instance_t *instance, uint8_t channel_id)
176 {
177     ESP_LOGI(TAG, "[%s] Disabling channel %d", __func__, channel_id);
178 
179     int return_val = 0;
180     if(channel_id > TSS_MAX_CHANNEL)
181     {
182         return -MIVE_ERR_INVALID_ARGUMENT;
183     }
184 
185     /*
186      *  Datasheet pg. 37
187      *  The easiest way to disable an channel register is to set the
188      *  received and transmitted bits to 1 in the Message Length and Status Registe
189      */
190     static const uint8_t data[] = {0x00, 0x00, 0x00, 0x0F, 0x00, 0x00, 0x00, 0x00};
191 
192     return_val = tss_registers_set(instance, TSS_CHANNEL_ADDR(channel_id), data,
    ↪  sizeof(data));
193 
194     memset(&instance->channels[channel_id], 0, sizeof(instance->channels[channel_id]));
195 
196     return return_val;
197 }
198 
199 /**
200  * @brief Put the TSS in active mode
201  *
202  * @param instance
203  * @return int
204  */
205 int tss_activate(tss_instance_t *instance)
206 {
207     // Sending the Activate command
208     command_register_t comm_register = {0};
209     comm_register.data.ACTI = 1;
210 
211     if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value) != MIVE_OK)
212     {
213         return 1;
214     }
215 
216     instance->chip_mode = TSS_MODE_ACTIVE;
217 
218     return 0;
219 }
```

```
220
221 /**
222  * @brief Put the TSS in sleep mode
223  *
224  * @param instance
225  * @return int
226  */
227 int tss_sleep(tss_instance_t *instance)
228 {
229     // Sending the Sleep command
230     command_register_t comm_register = {0};
231     comm_register.data.SLEEP = 1;
232
233     if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value))
234     {
235         return 1;
236     }
237
238     instance->chip_mode = TSS_MODE_SLEEP;
239
240     return 0;
241 }
242
243 /**
244  * @brief Put the TSS in idle mode
245  *
246  * @param instance
247  * @return int
248  */
249 int tss_idle(tss_instance_t *instance)
250 {
251     // Sending the Idle command
252     command_register_t comm_register = {0};
253     comm_register.data.IDLE = 1;
254
255     if (tss_register_set(instance, TSS_COMMANDREGISTER, comm_register.Value))
256     {
257         return 1;
258     }
259
260     instance->chip_mode = TSS_MODE_IDLE;
261
262     return 0;
263 }
264
265 IRAM_ATTR int tss_register_set(
266     tss_instance_t *instance,
267     uint8_t reg_addr,
268     uint8_t reg_value)
269 {
270     int return_val = 0;
271     spi_transaction_t transaction = {0};
272
273     if(unlikely(instance->chip_mode == TSS_MODE_SLEEP))
274     {
275         ESP_LOGE(TAG, "[%s] Chip in sleep mode", __func__);
276         return -1;
277     }
```

```
278
279     xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
280     spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
281
282     // gpio_set_level(instance->cs_pin, 0);
283     TSS_SELECT();
284     transaction.user = instance;
285     transaction.tx_data[0] = reg_addr;
286     transaction.length = 8;
287     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
288
289     usleep(1);
290
291     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
292
293     if (transaction.rx_data[0] != 0xAA)
294     {
295         printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n",
            ↪  transaction.rx_data[0]);
296         return_val = 1;
297         goto error;
298     }
299
300     transaction.tx_data[0] = TSS_WRITE;
301     transaction.length = 8;
302     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
303
304     usleep(2);
305     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
306
307     if (transaction.rx_data[0] != 0x55)
308     {
309         printf("Error setting regiser: Invalid response 0x%02x / 0x55\n",
            ↪  transaction.rx_data[0]);
310         return_val = 1;
311         goto error;
312     }
313
314     usleep(4);
315
316     transaction.tx_data[0] = reg_value;
317     transaction.length = 8;
318     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
319
320     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
321
322 error:
323     TSS_DESELECT();
324     spi_device_release_bus(instance->tss_handle);
325     xSemaphoreGive(instance->tss_spi_semaphore);
326
327     return return_val;
328 }
329
330 IRAM_ATTR int tss_registers_set(
331     tss_instance_t *const instance,
332     uint8_t const reg_addr,
333     uint8_t const *const values,
```

```
334     uint8_t const count)
335 {
336     unsigned int i = 0;
337     int return_val = 0;
338     spi_transaction_t transaction = {0};
339 
340     xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
341     // printf("Setting %d registers starting from addr %d\n", count, reg_addr);
342     spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
343     TSS_SELECT();
344 
345     transaction.user = instance;
346     transaction.tx_data[0] = reg_addr;
347     transaction.length = 8;
348     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
349 
350     usleep(1);
351     spi_device_transmit(instance->tss_handle, &transaction);
352     if (transaction.rx_data[0] != 0xAA)
353     {
354         printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n",
        ↪  transaction.rx_data[0]);
355         return_val = 1;
356         goto error;
357     }
358 
359     transaction.tx_data[0] = TSS_WRITE;
360 
361     usleep(2);
362     spi_device_transmit(instance->tss_handle, &transaction);
363 
364     if (transaction.rx_data[0] != 0x55)
365     {
366         printf("Error setting regiser: Invalid response 0x%02x / 0x55\n",
        ↪  transaction.rx_data[0]);
367         return_val = 1;
368         goto error;
369     }
370 
371     usleep(4);
372 
373     // printf("Transmitting: ");
374     for (i = 0; i < count; ++i)
375     {
376         transaction.tx_data[0] = values[i];
377         spi_device_transmit(instance->tss_handle, &transaction);
378         printf("%02x ", transaction.tx_data[0]);
379         usleep(3);
380     }
381 
382     printf("\n");
383 
384     usleep(3);
385 
386 error:
387 
388     TSS_DESELECT();
389     spi_device_release_bus(instance->tss_handle);
```

```
390
391     xSemaphoreGive(instance->tss_spi_semaphore);
392
393     return return_val;
394 }
395
396 IRAM_ATTR int tss_register_get(
397     tss_instance_t *const instance,
398     uint8_t const reg_addr,
399     uint8_t *const reg_value)
400 {
401     int return_val = 0;
402     spi_transaction_t transaction = {0};
403
404     xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
405     spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
406
407     // gpio_set_level(instance->cs_pin, 0);
408     TSS_SELECT();
409     transaction.user = instance;
410     transaction.tx_data[0] = reg_addr;
411     transaction.length = 8;
412     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
413
414     usleep(1);
415
416     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
417
418     if (transaction.rx_data[0] != 0xAA)
419     {
420         printf("Error getting regiser: Invalid response 0x%02x / 0xAA\n",
        ↪  transaction.rx_data[0]);
421         return_val = 1;
422         goto error;
423     }
424
425     transaction.tx_data[0] = TSS_READ;
426
427     usleep(2);
428     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
429
430     if (transaction.rx_data[0] != 0x55)
431     {
432         printf("Error getting regiser: Invalid response 0x%02x / 0x55\n",
        ↪  transaction.rx_data[0]);
433         return_val = 1;
434         goto error;
435     }
436
437     usleep(4);
438
439     transaction.tx_data[0] = 0xFF;
440     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
441
442     *reg_value = transaction.rx_data[0];
443 error:
444     TSS_DESELECT();
445     spi_device_release_bus(instance->tss_handle);
```

```
446
447     xSemaphoreGive(instance->tss_spi_semaphore);
448
449     return return_val;
450 }
451
452 IRAM_ATTR int tss_registers_get(
453     tss_instance_t *const instance,
454     uint8_t const reg_addr,
455     uint8_t *const buffer,
456     uint8_t count)
457 {
458     int return_val = 0;
459     spi_transaction_t transaction = {0};
460
461     if (unlikely(buffer == NULL))
462     {
463         return 1;
464     }
465
466     if(unlikely(instance->chip_mode == TSS_MODE_SLEEP))
467     {
468         ESP_LOGE(TAG, "[%s] Chip in sleep mode", __func__);
469         return -1;
470     }
471
472     xSemaphoreTake(instance->tss_spi_semaphore, portMAX_DELAY);
473     spi_device_acquire_bus(instance->tss_handle, portMAX_DELAY);
474
475     // gpio_set_level(instance->cs_pin, 0);
476     TSS_SELECT();
477     memset(&transaction, 0, sizeof(transaction));
478     transaction.user = instance;
479     transaction.tx_data[0] = reg_addr;
480     transaction.length = 8;
481     transaction.flags = SPI_TRANS_USE_TXDATA | SPI_TRANS_USE_RXDATA;
482
483     usleep(1);
484
485     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
486
487     if (transaction.rx_data[0] != 0xAA)
488     {
489         printf("Error setting regiser: Invalid response 0x%02x / 0xAA\n",
        ↪  transaction.rx_data[0]);
490         return_val = 1;
491         goto error;
492     }
493
494     transaction.tx_data[0] = TSS_READ;
495
496     usleep(2);
497     ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
498
499     if (transaction.rx_data[0] != 0x55)
500     {
501         printf("Error setting regiser: Invalid response 0x%02x / 0x55\n",
        ↪  transaction.rx_data[0]);
```

```
502         return_val = 1;
503         goto error;
504     }
505 
506     usleep(4);
507 
508     for (int i = 0; i < count; ++i)
509     {
510 
511         transaction.tx_data[0] = 0xFF;
512         ESP_ERROR_CHECK(spi_device_transmit(instance->tss_handle, &transaction));
513         buffer[i] = transaction.rx_data[0];
514         usleep(3);
515     }
516 
517 error:
518     TSS_DESELECT();
519     spi_device_release_bus(instance->tss_handle);
520 
521     xSemaphoreGive(instance->tss_spi_semaphore);
522 
523     return return_val;
524 }
525 
526 uint8_t tss_get_channel_status_register(
527     tss_instance_t* const instance,
528     uint8_t const channel_num)
529 {
530     uint8_t reg_value;
531     if(channel_num > TSS_MAX_CHANNEL)
532     {
533         return -MIVE_ERR_INVALID_ARGUMENT;
534     }
535     tss_register_get(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num), &reg_value);
536 
537     instance->channels[channel_num].message_len_and_status_reg_value = reg_value;
538 
539     return reg_value;
540 }
541 
542 uint8_t tss_set_channel_status_register(
543     tss_instance_t* const instance,
544     uint8_t const channel_num,
545     uint8_t const reg_value)
546 {
547     uint8_t new_reg_value;
548     if(channel_num > TSS_MAX_CHANNEL)
549     {
550         return -MIVE_ERR_INVALID_ARGUMENT;
551     }
552     tss_register_set(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num), reg_value);
553 
554     tss_register_get(instance, TSS_CHANNEL_MESS_LEN_AND_STATUS(channel_num),
    ↪  &new_reg_value);
555 
556     instance->channels[channel_num].message_len_and_status_reg_value = new_reg_value;
557 
558     return new_reg_value;
```

```
559 }
560
561 int tss_abort_channel(
562     tss_instance_t* const instance,
563     uint8_t const channel_num)
564 {
565     message_length_and_status_register_t reg;
566
567     reg.Value = tss_get_channel_status_register(instance, channel_num);
568     reg.data.CHER = 1;
569
570     tss_set_channel_status_register(instance, channel_num, reg.Value);
571
572     return MIVE_OK;
573 }
574
575 int tss_free_channel(
576     tss_instance_t* const instance,
577     uint8_t const channel_num)
578 {
579     tss_channel_t* channel = &instance->channels[channel_num];
580
581     channel->is_busy = false;
582     tss_disable_channel(instance, channel_num);
583     return MIVE_OK;
584 }
585
586 int tss_create(tss_instance_t* instance)
587 {
588     if (unlikely(instance == NULL))
589     {
590         printf("tss_start -> instance is NULL\n");
591         return 1;
592     }
593
594     memset(instance, 0, sizeof(*instance));
595
596     if (tss_spi_master_init(instance))
597     {
598         return 1;
599     }
600
601     instance->cs_pin = TSS_CS_PIN;
602     instance->buffers_size = 256;
603     instance->interrupt_pin = TSS_INT_PIN;
604     instance->tss_spi_semaphore = xSemaphoreCreateMutex();
605     instance->chip_mode = TSS_MODE_IDLE;
606
607     xSemaphoreGive(instance->tss_spi_semaphore);
608
609     gpio_config_t io_conf = {
610         .intr_type = GPIO_INTR_DISABLE,
611         .mode = GPIO_MODE_OUTPUT_OD,
612         .pin_bit_mask = (1 << instance->cs_pin),
613         .pull_up_en = GPIO_PULLUP_ENABLE,
614     };
615
616     // Configure CS pin as output
```

```
617     gpio_config(&io_conf);
618 
619     TSS_DESELECT();
620 
621     instance->tx_buffer = heap_caps_calloc(1, instance->buffers_size, MALLOC_CAP_DMA);
622     instance->rx_buffer = heap_caps_calloc(1, instance->buffers_size, MALLOC_CAP_DMA);
623 
624     return 0;
625 }
626 
627 int tss_start(tss_instance_t *instance)
628 {
629 
630     if (tss_motorolla_mode(instance))
631     {
632         // return 1;
633     }
634     usleep(3);
635 
636     for (uint8_t i = 0; i < TSS_MAX_CHANNEL; i++)
637     {
638         tss_disable_channel(instance, i);
639     }
640 
641     if (tss_register_set(instance, TSS_LINECONTROL, TSS_8MHz_125kBPS))
642     {
643         return 1;
644     }
645 
646     transmit_control_register_t tc_register;
647     tc_register.Value = 0;
648     tc_register.data.max_retries = 5;
649     tc_register.data.VER = 1;
650     tc_register.data.module_type = 1;
651 
652     if (tss_register_set(instance, TSS_TRANSMITCONTROL, tc_register.Value))
653     {
654         return 1;
655     }
656 
657     // Select interrupts to enable
658     interrupt_register_t it_register;
659     it_register.Value = 0;
660     it_register.data.RESET = 1; // Must be 1
661     it_register.data.TOK = 0;   // Change this to enable interrupt on successful
    ↪  transmissions
662     it_register.data.ROK = 0;   // Change this to enable interrupt on reception with RAK
663     it_register.data.RNOK = 0;  // Change this to enable interrupt on reception without
    ↪  RAK
664     it_register.data.RE = 0;    // Change this to enable interrupt on reception error
665     it_register.data.TE = 0;    // Change this to enable interrupt on transmission error
666 
667     if (tss_register_set(instance, TSS_INTERRUPTENABLE, it_register.Value))
668     {
669         return 1;
670     }
671 
672     tss_register_get(instance, TSS_INTERRUPTSTATUS, &(it_register.Value));
```

```
673
674     printf("\tRNOK: %d\n", it_register.data.RNOK);
675     printf("\tROK: %d\n", it_register.data.ROK);
676     printf("\tRE: %d\n", it_register.data.RE);
677     printf("\tTOK: %d\n", it_register.data.TOK);
678     printf("\tTE: %d\n", it_register.data.TE);
679     printf("\tRESET: %d\n", it_register.data.RESET);
680
681     // Reset all interrupt statuses
682     it_register.data.RESET = 1;
683     it_register.data.TOK = 1;
684     it_register.data.ROK = 1;
685     it_register.data.RNOK = 1;
686     it_register.data.RE = 1;
687     it_register.data.TE = 1;
688
689     if (tss_register_set(instance, TSS_INTERRUPTRESET, it_register.Value))
690     {
691         return 1;
692     }
693
694     // Memsetting the TSS's RAM to zeros
695     uint8_t zeroarray[128] = {0};
696     if (tss_registers_set(instance, TSS_GETMAIL(0), zeroarray, 128))
697     {
698         return 1;
699     }
700
701     it_register.Value = 0;
702     it_register.data.RESET = 1; // Must be 1
703     it_register.data.TOK = 1;   // Change this to enable interrupt on successful
    ↪  transmissions
704     it_register.data.ROK = 1;   // Change this to enable interrupt on reception with RAK
705     it_register.data.RNOK = 1;  // Change this to enable interrupt on reception without
    ↪  RAK
706     it_register.data.RE = 1;    // Change this to enable interrupt on reception error
707     it_register.data.TE = 1;    // Change this to enable interrupt on transmission error
708
709     if (tss_register_set(instance, TSS_INTERRUPTENABLE, it_register.Value))
710     {
711         return 1;
712     }
713
714     if (tss_activate(instance))
715     {
716         return 1;
717     }
718
719     return 0;
720 }
721
722 int tss_transmit_message(
723     tss_instance_t* const instance,
724     uint8_t channel,
725     uint16_t const iden,
726     uint8_t *const data,
727     uint8_t const data_size,
728     uint8_t const memory_offset,
```

```
    uint8_t const rak)
{
    uint8_t id1 = 0;
    uint8_t id2 = 0;
    uint8_t memory_address;
    id2_and_command_register_t id2_reg = {0};
    message_pointer_register_t mp_reg = {0};
    message_length_and_status_register_t mls_reg = {0};
    uint8_t channel_data[8] = {0};
    tss_channel_t* chan = NULL;
    int retval;

    retval = is_channel_valid(instance, channel, iden);

    if(retval != MIVE_OK)
    {
        printf("[%s] Invalid channel %d (%d)\n", __func__, channel, retval);
        return retval;
    }
    if(memory_offset & 0x80)
    {
        memory_address = memory_offset;
    }
    else
    {
        memory_address = TSS_GETMAIL(memory_offset);
    }

    ESP_LOGD(TAG, "[%s] Using memory addr 0x%02x", __func__, memory_address);

    get_bytes_from_iden(iden, id1, id2);


    id2_reg.data.ID = id2;
    id2_reg.data.RNW = 0;
    id2_reg.data.RTR = 0;
    id2_reg.data.EXT = 1;
    id2_reg.data.RAK = rak;

    mp_reg.data.DRAK = 0;
    mp_reg.data.message_pointer = memory_address & 0x7F;

    mls_reg.data.CHTx = 0;
    mls_reg.data.M_L = data_size + 1;

    // When writing to the bus, the first element in the buffer is ignored,
    // so we skip one (memory_address + 1)
    tss_registers_set(instance, memory_address + 1, data, data_size);

    channel_data[0] = id1;
    channel_data[1] = id2_reg.Value;
    channel_data[2] = mp_reg.Value;
    channel_data[3] = mls_reg.Value;
    channel_data[6] = id1;
    channel_data[7] = id2;

    tss_registers_set(instance, TSS_CHANNEL_ADDR(channel), channel_data, 8);

```

```
787     chan = &instance->channels[channel];
788     chan->data_size = data_size + 1;
789     chan->ram_address = memory_address;
790     chan->identifier = iden;
791     chan->is_occupied = true;
792     chan->is_busy = true;
793     chan->message_len_and_status_reg_value = mls_reg.Value;
794     chan->tx_attempt = 1;
795     chan->message_type = TSS_TRANSMIT;
796 
797     return MIVE_OK;
798 }
799 
800 int tss_immediate_reply_message(
801     tss_instance_t* const instance,
802     uint8_t channel,
803     uint16_t const iden,
804     uint8_t *const data,
805     uint8_t const data_size,
806     uint8_t const memory_offset)
807 {
808     uint8_t id1 = 0;
809     uint8_t id2 = 0;
810     uint8_t memory_address;
811     id2_and_command_register_t id2_reg = {0};
812     message_pointer_register_t mp_reg = {0};
813     message_length_and_status_register_t mls_reg = {0};
814     uint8_t channel_data[8] = {0};
815     tss_channel_t* chan = NULL;
816     int retval;
817 
818     retval = is_channel_valid(instance, channel, iden);
819 
820     if(retval != MIVE_OK)
821     {
822         printf("[%s] Invalid channel %d (%d)\n", __func__, channel, retval);
823         return retval;
824     }
825 
826     if(memory_offset & 0x80)
827     {
828         memory_address = memory_offset;
829     }
830     else
831     {
832         memory_address = TSS_GETMAIL(memory_offset);
833     }
834 
835     ESP_LOGD(TAG, "[%s] Using memory addr 0x%02x", __func__, memory_address);
836 
837     get_bytes_from_iden(iden, id1, id2);
838 
839 
840     id2_reg.data.ID = id2;
841     id2_reg.data.RNW = 1;
842     id2_reg.data.RTR = 0;
843     id2_reg.data.EXT = 1;
844     id2_reg.data.RAK = 0;
```

```
845
846     mp_reg.data.DRAK = 0;
847     mp_reg.data.message_pointer = memory_address & 0x7F;
848
849     mls_reg.data.CHTx = 0;
850     mls_reg.data.CHRx = 0;
851     mls_reg.data.M_L = data_size + 1;
852
853     // When writing to the bus, the first element in the buffer is ignored,
854     // so we skip one (memory_address + 1)
855     tss_registers_set(instance, memory_address + 1, data, data_size);
856
857     channel_data[0] = id1;
858     channel_data[1] = id2_reg.Value;
859     channel_data[2] = mp_reg.Value;
860     channel_data[3] = mls_reg.Value;
861     channel_data[6] = id1;
862     channel_data[7] = id2;
863
864     tss_registers_set(instance, TSS_CHANNEL_ADDR(channel), channel_data, 8);
865
866     chan = &instance->channels[channel];
867     chan->data_size = data_size + 1;
868     chan->ram_address = memory_address;
869     chan->identifier = iden;
870     chan->is_occupied = true;
871     chan->is_busy = true;
872     chan->message_len_and_status_reg_value = mls_reg.Value;
873     chan->tx_attempt = 1;
874     chan->message_type = TSS_IMM_REPLY;
875
876     return MIVE_OK;
877 }
```

Kod 7: uart_task.c

```
1  #include "freertos/FreeRTOS.h"
2
3  #include <stdint.h>
4  #include <string.h>
5  #include <stdlib.h>
6
7  #include "include/global_config.h"
8  #include "include/global_tasks.h"
9  #include "include/global_state_def.h"
10
11 #include "include/mive/common.h"
12 #include "include/mive/psa_packet_defs.h"
13 #include "include/mive/psa_helper.h"
14
15 #include "driver/uart.h"
16 #include "esp_timer.h"
17 #include "esp_log.h"
18
19 static const char *TAG = "UART";
```

```
20  static int uart_recv_buffer_num = 0;
21
22  IRAM_ATTR static void uart_timer_callback_f(void* user_data)
23  {
24      BaseType_t high_task_wakeup = pdFALSE;
25      QueueHandle_t uart_queue = (QueueHandle_t)user_data;
26      struct mive_global_event timer_event = {
27          .ev_data = NULL,
28          .event = MIVE_EVENT_TIMER_1MS,
29      };
30      xQueueSendFromISR(uart_queue, &timer_event, &high_task_wakeup);
31      if(high_task_wakeup)
32      {
33          esp_timer_isr_dispatch_need_yield();
34      }
35  }
36
37
38  static mive_uart_task_packet_t* get_uart_recv_buffer(void)
39  {
40      mive_uart_task_packet_t* to_ret = &global_uart_receive_buffers[uart_recv_buffer_num];
41
42      uart_recv_buffer_num = (uart_recv_buffer_num >= (PSA_MAIN_UART_SEND_BUFFERS_NUM - 1))
        ↪  ? 0 : uart_recv_buffer_num + 1;
43
44      return to_ret;
45  }
46
47  static void uart_rx_task(void* params);
48
49  void uart_task(void* params)
50  {
51      int ret = 0;
52      const int uart_buffer_size = 2048;
53      struct mive_global_event event = {0};
54      QueueHandle_t main_queue = g_global_state.global_main_queue;
55      QueueHandle_t tss_queue = g_global_state.global_tss_queue;
56      QueueHandle_t van_queue = g_global_state.global_van_queue;
57      QueueHandle_t uart_queue = g_global_state.global_uart_queue;
58      QueueHandle_t uart_driver_queue;
59      esp_timer_handle_t uart_timer_handle;
60      mive_uart_task_packet_t* uart_packet = NULL;
61      uint8_t* uart_buffer = malloc(256);
62      uint8_t* uart_data_buffer = NULL;
63      uint8_t uart_crc;
64      struct psa_header* psa_packet_header = (struct psa_header*)uart_buffer;
65
66
67      esp_timer_create_args_t uart_timer_create_args = {
68          .arg = uart_queue,
69          .callback = uart_timer_callback_f,
70          .dispatch_method = ESP_TIMER_ISR,
71          .name = NULL,
72          .skip_unhandled_events = true
73      };
74
75      uart_config_t uart_config = {
76          .baud_rate = 500000,
```

```
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };

    ESP_ERROR_CHECK(uart_driver_install(UART_NUM_2, uart_buffer_size, 0, 10,
    ↪  &uart_driver_queue, 0));
    ESP_ERROR_CHECK(uart_param_config(UART_NUM_2, &uart_config));
    ESP_ERROR_CHECK(uart_set_pin(UART_NUM_2, 4, 5, -1, -1));

    xTaskCreatePinnedToCore(
        uart_rx_task,
        "uart_rx_task",
        5120,
        &g_global_state, 11, NULL, xPortGetCoreID());

    esp_timer_create(&uart_timer_create_args, &uart_timer_handle);
    esp_timer_start_once(uart_timer_handle, 1000000);

    while(true)
    {
        ret = xQueueReceive(uart_queue, &event, pdMS_TO_TICKS(500));
        if(ret == pdPASS)
        {
            switch (event.event)
            {
            case MIVE_EVENT_TIMER_UART_SEND:
                // Send data
                esp_timer_start_once(uart_timer_handle, 1000);
                break;

            case MIVE_EVENT_UART_SEND:
                uart_packet = (mive_uart_task_packet_t*)event.ev_data;
                if(uart_packet->iden > PSA_MSP_MIN_IDENT && uart_packet->data_size <
                ↪  PSA_MSP_MAX_SIZE)
                {
                    uart_data_buffer = uart_buffer + sizeof(*psa_packet_header);

                    psa_packet_header->ident = uart_packet->iden;
                    psa_packet_header->direction = '>';
                    psa_packet_header->size = uart_packet->data_size;
                    psa_packet_header->start = PSA_MSP_START_MAGIC;

                    memcpy(uart_data_buffer, uart_packet->data, uart_packet->data_size);

                    uart_crc = psa_crc8_checksum(uart_buffer + 2u, uart_packet->data_size
                    ↪  + 3u);

                    uart_data_buffer[psa_packet_header->size] = uart_crc;

                    uart_write_bytes(
                        UART_NUM_2,
                        uart_buffer,
                        uart_packet->data_size + sizeof(*psa_packet_header) + 1u);
                }
                vTaskDelay(1);
                break;
```

```
            default:
                break;
            }
        }
    }

    vTaskDelete(NULL);
    return;
}

void uart_rx_task(void* params)
{
    QueueHandle_t main_queue = g_global_state.global_main_queue;
    QueueHandle_t tss_queue = g_global_state.global_tss_queue;
    QueueHandle_t van_queue = g_global_state.global_van_queue;
    QueueHandle_t uart_queue = g_global_state.global_uart_queue;
    struct mive_global_event global_event = {
        .event = MIVE_EVENT_UART_RECEIVE,
        .ev_data = NULL,
    };
    uint8_t* rx_buffer = malloc(2048);
    uint8_t* data;
    uint8_t calc_crc;
    mive_uart_task_packet_t* task_packet;

    struct psa_header* header = (struct psa_header*)rx_buffer;
    data = rx_buffer + sizeof(*header);

    while(1)
    {
        int rxBytes = uart_read_bytes(UART_NUM_2, rx_buffer, 5, pdMS_TO_TICKS(1000));
        if(rxBytes > 0)
        {
            ESP_LOGD(TAG, "[%s] Got %d bytes", __func__, rxBytes);

            if(rxBytes == sizeof(*header))
            {
                rxBytes = uart_read_bytes(UART_NUM_2, data, header->size + 1,
                ↪  pdMS_TO_TICKS(5));
                calc_crc = psa_crc8_checksum(rx_buffer + 2u, header->size + 3u);
                if(calc_crc == data[header->size])
                {
                    ESP_LOGD(TAG, "[%s] Valid packet\n", __func__);
                    task_packet = get_uart_recv_buffer();
                    task_packet->iden = header->ident;
                    task_packet->data_size = header->size;
                    memcpy(task_packet->data, data, header->size);
                    global_event.ev_data = task_packet;
                    xQueueSendToBack(main_queue, &global_event, 0);
                }
            }
        }

    }

}
```

Kod 8: van_packet_parser.c

```c
#include "freertos/FreeRTOS.h"
#include <stdio.h>
#include <stdint.h>
#include <endian.h>
#include <string.h>

#include "include/mive/common.h"
#include "include/van/van_packet_iden.h"
#include "include/van/van_packet_parser.h"
#include "include/mive/common.h"
#include "include/mive/psa_packet_defs.h"
#include "include/van_structs.h"
#include "include/global_state_def.h"

#define PACKET_BUFFER_NUM 20

static int audio_menu_open = 0;
static int audio_menu_setting = 0; // 0, 1, 2, 3, 4, 5, 6

// static const int audio_menu_duration_ms = 4000;

mive_uart_queue_packet_t packet_buffers[PACKET_BUFFER_NUM];

static int packet_buffer_num = 0;

union dash_mileage
{
    uint32_t number : 24;
    uint8_t buffer[3];
} __attribute__((packed));

static uint16_t swap_endian_uint16(uint16_t const x)
{
    return (x << 8) | (x >> 8);
}

static uint32_t swap_endian_uint32(uint32_t const x)
{
    uint32_t val = ((x << 8) & 0xFF00FF00) | ((x >> 8) & 0xFF00FF);
    return (val << 16) | (val >> 16);
}

static uint32_t mileage_swap_bytes(union dash_mileage const mileage)
{
    uint32_t temp_mileage = (uint32_t)mileage.buffer[0] << 16 |
    ↪  (uint32_t)mileage.buffer[1] << 8 | mileage.buffer[2];
    return temp_mileage;
}

static mive_uart_queue_packet_t* get_packet_buffer(void)
{
    mive_uart_queue_packet_t* to_ret = &packet_buffers[packet_buffer_num];

    packet_buffer_num = (packet_buffer_num >= (PACKET_BUFFER_NUM - 1)) ? 0 :
    ↪  packet_buffer_num + 1;
```

```
54
55      return to_ret;
56  }
57
58  static int psa_parse_buttons_iden(
59      uint8_t const *const data,
60      struct psa_output_data_buffers *data_buffers)
61  {
62      struct mive_global_event event = {0};
63
64      if (data_buffers->headunit_data == NULL)
65      {
66          return -MIVE_ERR_INVALID_ARGUMENT;
67      }
68
69      if ((data[1] & 0x0F) == 0x02)
70      {
71          // Refresh the timer when audio up or down buttons are pressed
72          if (((data[2] & 0x1F) == 0x11 || (data[2] & 0x1F) == 0x12))
73          {
74              event.event = MIVE_EVENT_VAN_UPDATE_AUDIO_MENU;
75          }
76          // Change the menu when the audio button is let go. Will probably go tits up when
            ↪  the button is held down for more than 2 secs
77          if (data[2] == 0x56)
78          {
79              event.event = MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM;
80          }
81      }
82
83      if(event.event)
84      {
85          xQueueSendToBack(g_global_state.global_main_queue, &event, 0);
86      }
87
88      return MIVE_OK;
89  }
90
91  static int psa_parse_rpm_iden(
92      uint8_t const *const data,
93      struct psa_output_data_buffers *data_buffers)
94  {
95      struct mive_global_event queue_event = {
96          .event = MIVE_EVENT_UART_NEW_DATA
97      };
98      mive_uart_queue_packet_t* queue_packet;
99
100     if (data_buffers->engine_data == NULL)
101     {
102         return -MIVE_ERR_INVALID_ARGUMENT;
103     }
104
105     struct psa_engine_data *engine_data = (struct psa_engine_data
        ↪  *)(data_buffers->engine_data);
106
107     struct psa_van_rpm_speed_struct const *const rpm_data = (struct
        ↪  psa_van_rpm_speed_struct *)data;
108
```

```
    engine_data->rpm = be16toh(rpm_data->rpm);
    engine_data->speed = be16toh(rpm_data->speed);

    queue_packet = get_packet_buffer();

    queue_packet->idens[0] = PSA_IDENT_ENGINE;
    queue_packet->num_idens = 1;

    queue_event.ev_data = (void*)queue_packet;

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_dash_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    uint32_t mileage;
    union dash_mileage temp_mileage;
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_DASHBOARD, PSA_IDENT_ENGINE},
        .num_idens = 2,
    };


    if (data_buffers->dash_data == NULL || data_buffers->engine_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_dash_data *dash_data = (struct psa_dash_data *)(data_buffers->dash_data);
    struct psa_engine_data *engine_data = (struct psa_engine_data
    ↪  *)(data_buffers->engine_data);
    // struct psa_door_data *door_data = (struct psa_door_data
    ↪  *)(data_buffers->door_data);

    struct psa_van_dashboard *van_data = (struct psa_van_dashboard *)data;

    dash_data->accesories_on = van_data->accesories_on;
    dash_data->backlight_off = van_data->is_backlight_off;
    dash_data->brightness = van_data->brightness;
    dash_data->door_open = van_data->door_open;
    dash_data->economy_mode = van_data->economy_mode;
    dash_data->engine_running = van_data->engine_running;
    dash_data->external_temp = van_data->external_temp;
```

```
    dash_data->ignition_on = van_data->ignition_on;

    dash_data->backlight_off = van_data->is_backlight_off;
    dash_data->reverse_gear = van_data->reverse_gear;
    dash_data->water_temp = van_data->water_temp;
    engine_data->engine_temperature = van_data->water_temp;
    engine_data->reverse = van_data->reverse_gear;
    mileage = van_data->mileage;

    temp_mileage.number = van_data->mileage;

    dash_data->mileage = mileage_swap_bytes(temp_mileage);

    xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);

    return MIVE_OK;
}

static int psa_parse_trip_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_DOORS, PSA_IDENT_TRIP},
        .num_idens = 2,
    };

    if (data_buffers->door_data == NULL || data_buffers->trip_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_trip_data *trip_data = (struct psa_trip_data *)(data_buffers->trip_data);
    struct psa_door_data *door_data = (struct psa_door_data *)(data_buffers->door_data);

    struct psa_van_trip_computer *van_data = (struct psa_van_trip_computer *)data;

    door_data->open_doors = (van_data->door_status.packed);

    if (door_data->open_doors)
    {
        door_data->door_open = 1;
    }
    else
    {
        door_data->door_open = 0;
    }
```

```
223
224     trip_data->current_fuel_consumption = be16toh(van_data->current_fuel_consumption);
225     trip_data->distance_to_empty = be16toh(van_data->distance_to_empty);
226     trip_data->trip_1_distance = be16toh(van_data->trip_1_distance);
227     trip_data->trip_1_fuel_consumption = be16toh(van_data->trip_1_fuel_consumption);
228     trip_data->trip_1_speed = van_data->trip_1_speed;
229
230     trip_data->trip_2_distance = be16toh(van_data->trip_2_distance);
231     trip_data->trip_2_fuel_consumption = be16toh(van_data->trip_2_fuel_consumption);
232     trip_data->trip_2_speed = van_data->trip_2_speed;
233     trip_data->trip_button_pressed = van_data->trip_button.trip_button;
234
235     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
236
237     return MIVE_OK;
238 }
239
240 // 0x554 len 22
241 static int psa_parse_radio_tuner_iden(
242     uint8_t const *const data,
243     struct psa_output_data_buffers *data_buffers)
244 {
245     mive_uart_queue_packet_t* queue_packet = NULL;
246
247     mive_uart_queue_packet_t queue_packet_c = {
248         .idens = {PSA_IDENT_RADIO, PSA_IDENT_RADIO_PRESETS},
249         .num_idens = 2,
250     };
251
252     if (data_buffers->radio_data == NULL)
253     {
254         return -MIVE_ERR_INVALID_ARGUMENT;
255     }
256
257     if (data[1] != 0xD1) // D1 is frequency info
258     {
259         return -MIVE_ERR_VAN_UNKNOWN_IDEN;
260     }
261
262     queue_packet = get_packet_buffer();
263
264     *queue_packet = queue_packet_c;
265
266     struct mive_global_event queue_event = {
267         .event = MIVE_EVENT_UART_NEW_DATA,
268         .ev_data = queue_packet,
269     };
270
271     struct psa_van_radio_freq_info *van_data = (struct psa_van_radio_freq_info *)data;
272     struct psa_radio_data *radio_data = (struct psa_radio_data
    ↪  *)data_buffers->radio_data;
273
274     struct psa_preset_data *preset_data = NULL;
275
276     memset(radio_data->station, 0, 9);
277     strncpy(radio_data->station, (const char *)van_data->station, 8);
278
279     uint16_t freq = (uint16_t)(van_data->frequency[1]) << 8 | van_data->frequency[0];
```

```
280
281     radio_data->freq = freq * 5 + 5000;
282
283     radio_data->signal_strength = van_data->signal_info & 0x0F;
284     // radio_data->freq = uint16_t(van_data->frequency[1]) << 8 | van_data->frequency[0];
        ↪  // swap_endian_uint16(van_data->frequency);
285
286     radio_data->preset = van_data->memory_position;
287
288     switch (van_data->band)
289     {
290     case PSA_VAN_BAND_FM1:
291         radio_data->band = PSA_FM_1;
292         preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_1;
293         break;
294     case PSA_VAN_BAND_FM2:
295         radio_data->band = PSA_FM_2;
296         preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_2;
297         break;
298     case PSA_VAN_BAND_FM3:
299         radio_data->band = PSA_FM_3;
300         break;
301     case PSA_VAN_BAND_FMAST:
302         radio_data->band = PSA_FM_AST;
303         preset_data = (struct psa_preset_data *)data_buffers->presets_data_fm_ast;
304
305         break;
306     case PSA_VAN_BAND_AM:
307         radio_data->band = PSA_AM_1;
308         preset_data = (struct psa_preset_data *)data_buffers->presets_data_am;
309         break;
310     case PSA_VAN_BAND_NONE:
311     case PSA_VAN_BAND_PTY_SELECT:
312     default:
313         radio_data->band = PSA_NONE;
314         break;
315     }
316
317     if (radio_data->preset > 0 && radio_data->preset <= 6 && preset_data != NULL)
318     {
319         strncpy(preset_data->presets[radio_data->preset - 1].preset_name, (const char
            ↪  *)van_data->station, 8);
320         preset_data->presets->preset_num = radio_data->preset;
321     }
322
323     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
324
325     return MIVE_OK;
326 }
327
328 // 0x554 len 10
329 static int psa_parse_radio_cd_short_iden(
330     uint8_t const *const data,
331     struct psa_output_data_buffers *data_buffers)
332 {
333     mive_uart_queue_packet_t* queue_packet = NULL;
334
335     mive_uart_queue_packet_t queue_packet_c = {
```

```
        .idens = {PSA_IDENT_CD_PLAYER},
        .num_idens = 1,
    };

    if (data_buffers->cd_player_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    if (data[1] != 0xD6) // D6 is CD info
    {
        return -MIVE_ERR_VAN_UNKNOWN_IDEN;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_van_radio_cd_info_len_10 *van_data = (struct psa_van_radio_cd_info_len_10
    ↪  *)data;
    struct psa_cd_player_data *cd_data = (struct psa_cd_player_data
    ↪  *)data_buffers->cd_player_data;

    cd_data->current_minute = bcd_to_dec(van_data->minutes);
    cd_data->current_second = bcd_to_dec(van_data->seconds);
    cd_data->current_track = bcd_to_dec(van_data->current_track);
    cd_data->total_tracks = bcd_to_dec(van_data->total_tracks);

    cd_data->status = 0;

    switch (van_data->status)
    {
    case 0x10:
        cd_data->status |= PSA_CD_PLAYER_ERROR;
        break;
    case 0x11:
        cd_data->status |= PSA_CD_PLAYER_LOADING;
        break;
    case 0x12:
        cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PAUSED;
        break;
    case 0x13:
        cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PLAYING;
        break;
    case 0x02:
        cd_data->status |= PSA_CD_PLAYER_PAUSED;
        break;
    case 0x03:
        cd_data->status |= PSA_CD_PLAYER_PLAYING;
        break;
    case 0x04:
        cd_data->status |= PSA_CD_PLAYER_FAST_FORWARDING;
        break;
```

```
392     case 0x05:
393         cd_data->status |= PSA_CD_PLAYER_REWINDING;
394         break;
395     default:
396         cd_data->status = 0;
397         break;
398     }
399 
400     cd_data->total_minutes = 0xff;
401     cd_data->total_seconds = 0xff;
402 
403     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
404 
405     return MIVE_OK;
406 }
407 
408 // 0x554 len 19
409 static int psa_parse_radio_cd_long_iden(
410     uint8_t const *const data,
411     struct psa_output_data_buffers *data_buffers)
412 {
413     mive_uart_queue_packet_t* queue_packet = NULL;
414 
415     mive_uart_queue_packet_t queue_packet_c = {
416         .idens = {PSA_IDENT_CD_PLAYER},
417         .num_idens = 1,
418     };
419 
420     if (data_buffers->cd_player_data == NULL)
421     {
422         return -MIVE_ERR_INVALID_ARGUMENT;
423     }
424 
425     if (data[1] != 0xD6) // D6 is CD info
426     {
427         return -MIVE_ERR_VAN_UNKNOWN_IDEN;
428     }
429 
430     queue_packet = get_packet_buffer();
431 
432     *queue_packet = queue_packet_c;
433 
434     struct mive_global_event queue_event = {
435         .event = MIVE_EVENT_UART_NEW_DATA,
436         .ev_data = queue_packet,
437     };
438 
439     struct psa_van_radio_cd_info_len_19 *van_data = (struct psa_van_radio_cd_info_len_19
    ↪  *)data;
440     struct psa_cd_player_data *cd_data = (struct psa_cd_player_data
    ↪  *)data_buffers->cd_player_data;
441 
442     cd_data->current_minute = bcd_to_dec(van_data->minutes);
443     cd_data->current_second = bcd_to_dec(van_data->seconds);
444     cd_data->current_track = bcd_to_dec(van_data->current_track);
445     cd_data->total_tracks = bcd_to_dec(van_data->total_tracks);
446     cd_data->total_minutes = bcd_to_dec(van_data->total_minutes);
447     cd_data->total_seconds = bcd_to_dec(van_data->total_seconds);
```

```
448
449     cd_data->status = 0;
450
451     switch (van_data->status)
452     {
453     case 0x10:
454         cd_data->status |= PSA_CD_PLAYER_ERROR;
455         break;
456     case 0x11:
457         cd_data->status |= PSA_CD_PLAYER_LOADING;
458         break;
459     case 0x12:
460         cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PAUSED;
461         break;
462     case 0x13:
463         cd_data->status |= PSA_CD_PLAYER_LOADING | PSA_CD_PLAYER_PLAYING;
464         break;
465     case 0x02:
466         cd_data->status |= PSA_CD_PLAYER_PAUSED;
467         break;
468     case 0x03:
469         cd_data->status |= PSA_CD_PLAYER_PLAYING;
470         break;
471     case 0x04:
472         cd_data->status |= PSA_CD_PLAYER_FAST_FORWARDING;
473         break;
474     case 0x05:
475         cd_data->status |= PSA_CD_PLAYER_REWINDING;
476         break;
477     default:
478         cd_data->status = 0;
479         break;
480     }
481
482     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
483
484     return MIVE_OK;
485 }
486
487 static int psa_parse_radio_preset_iden(
488     uint8_t const *const data,
489     struct psa_output_data_buffers *data_buffers)
490 {
491     // if (data_buffers == NULL)
492     // {
493     //     return -MIVE_ERR_INVALID_ARGUMENT;
494     // }
495
496     // if (data_buffers->presets_data == NULL)
497     // {
498     //     return -MIVE_ERR_INVALID_ARGUMENT;
499     // }
500
501     // if (data[1] != 0xD3) // D3 is preset info
502     // {
503     //     return -MIVE_ERR_VAN_UNKNOWN_IDEN;
504     // }
505
```

```
    // struct psa_van_radio_preset_info *van_data = (struct psa_van_radio_preset_info
    ↪  *)data;

    // struct psa_preset_data *preset_data = (struct psa_preset_data
    ↪  *)data_buffers->presets_data;

    // if (van_data->position > 0 && van_data->position < 7)
    // {
    //     memset(preset_data->presets[van_data->position].preset_name, 0, 10);
    //     strncpy(preset_data->presets[van_data->position].preset_name, (const char
    ↪  *)van_data->station_name, 8);

    //     preset_data->presets[van_data->position].preset_num = van_data->position;
    // }
    // else
    // {
    //     return PSA_INVALID_VAN_PACKET;
    // }

    return MIVE_OK;
}

static int psa_parse_headunit_iden(
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    mive_uart_queue_packet_t* queue_packet = NULL;

    mive_uart_queue_packet_t queue_packet_c = {
        .idens = {PSA_IDENT_HEADUNIT},
        .num_idens = 1,
    };

    if (data_buffers->headunit_data == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    queue_packet = get_packet_buffer();

    *queue_packet = queue_packet_c;

    struct mive_global_event queue_event = {
        .event = MIVE_EVENT_UART_NEW_DATA,
        .ev_data = queue_packet,
    };

    struct psa_van_radio_settings *van_data = (struct psa_van_radio_settings *)data;

    struct psa_headunit_data *headunit_data = (struct psa_headunit_data
    ↪  *)data_buffers->headunit_data;

    headunit_data->unit_powered_on = van_data->power.power_on;
    headunit_data->auto_volume = van_data->audio_properties.auto_volume;
    headunit_data->balance = ((int8_t)0x3f - van_data->balance.value);
    headunit_data->fader = ((int8_t)0x3f - van_data->fader.value);
    headunit_data->bass = ((int8_t)van_data->bass.value - 0x3f);
    headunit_data->treble = ((int8_t)van_data->treble.value - 0x3f);
```

```
560     headunit_data->loudness_on = van_data->audio_properties.loudness_on;
561     headunit_data->volume = van_data->volume.value;
562 
563     headunit_data->muted = van_data->audio_properties.external_mute ||
        ↪  van_data->audio_properties.stalk_mute;
564 
565     switch (van_data->source.source)
566     {
567     case 0x01: // Tuner
568         headunit_data->source = PSA_RADIO_TUNER;
569         break;
570     case 0x02: // Tape or CD
571         headunit_data->source = PSA_RADIO_INTERNAL;
572         break;
573     case 0x03: // Cd changer
574         headunit_data->source = PSA_RADIO_EXTERNAL;
575         break;
576     case 0x05: // Navigation
577         headunit_data->source = PSA_RADIO_NAVIGATION;
578         break;
579     default:
580         headunit_data->source = PSA_RADIO_NONE;
581         break;
582     }
583 
584     headunit_data->cd_present = van_data->source.cd_present;
585     headunit_data->tape_present = van_data->source.tape_present;
586     // headunit_data->settings_menu_open =
        ↪  van_data->audio_properties.audio_properties_menu_open;
587 
588     // headunit_data->settings_changing.balance_changing = van_data->balance.updating;
589     // headunit_data->settings_changing.bass_changing = van_data->bass.updating;
590     // headunit_data->settings_changing.fader_changing = van_data->fader.updating;
591     // headunit_data->settings_changing.treble_changing = van_data->treble.updating;
592     // headunit_data->settings_changing.volume_changing = van_data->volume.updating;
593 
594     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
595 
596     return MIVE_OK;
597 }
598 
599 static int psa_parse_vin_iden(
600     uint8_t const *const data,
601     struct psa_output_data_buffers *data_buffers)
602 {
603     mive_uart_queue_packet_t* queue_packet = NULL;
604 
605     mive_uart_queue_packet_t queue_packet_c = {
606         .idens = {PSA_IDENT_VIN},
607         .num_idens = 1,
608     };
609 
610     if (data_buffers->vin_data == NULL)
611     {
612         return -MIVE_ERR_INVALID_ARGUMENT;
613     }
614 
615     queue_packet = get_packet_buffer();
```

```
616
617     *queue_packet = queue_packet_c;
618
619     struct mive_global_event queue_event = {
620         .event = MIVE_EVENT_UART_NEW_DATA,
621         .ev_data = queue_packet,
622     };
623     // char *vin = (char *)data_buffers->vin_data;
624     int const vin_size = 17;
625
626     memcpy(data_buffers->vin_data, data, vin_size);
627
628     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
629
630     return MIVE_OK;
631 }
632
633 // 0x4FC len 11
634 static int psa_parse_instruments_short_iden(
635     uint8_t const *const data,
636     struct psa_output_data_buffers *data_buffers)
637 {
638     mive_uart_queue_packet_t* queue_packet = NULL;
639     mive_uart_queue_packet_t queue_packet_c = {
640         .idens = {PSA_IDENT_ENGINE},
641         .num_idens = 1,
642     };
643
644     if (data_buffers->engine_data == NULL)
645     {
646         return -MIVE_ERR_INVALID_ARGUMENT;
647     }
648
649     queue_packet = get_packet_buffer();
650
651     *queue_packet = queue_packet_c;
652
653     struct mive_global_event queue_event = {
654         .event = MIVE_EVENT_UART_NEW_DATA,
655         .ev_data = queue_packet,
656     };
657
658     struct psa_engine_data *engine_data = (struct psa_engine_data
    ↪  *)data_buffers->engine_data;
659     struct psa_van_instrument_cluster_short *van_data = (struct
    ↪  psa_van_instrument_cluster_short *)data;
660
661     engine_data->fuel_level = van_data->fuel_level;
662     engine_data->oil_temperature = van_data->oil_temp;
663
664     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
665
666     return MIVE_OK;
667 }
668
669 // 0x4FC len 14
670 static int psa_parse_instruments_long_iden(
671     uint8_t const *const data,
```

```
672     struct psa_output_data_buffers *data_buffers)
673 {
674     mive_uart_queue_packet_t* queue_packet = NULL;
675     mive_uart_queue_packet_t queue_packet_c = {
676         .idens = {PSA_IDENT_ENGINE},
677         .num_idens = 1,
678     };
679 
680     if (data_buffers->engine_data == NULL)
681     {
682         return -MIVE_ERR_INVALID_ARGUMENT;
683     }
684 
685     queue_packet = get_packet_buffer();
686 
687     *queue_packet = queue_packet_c;
688 
689     struct mive_global_event queue_event = {
690         .event = MIVE_EVENT_UART_NEW_DATA,
691         .ev_data = queue_packet,
692     };
693 
694     struct psa_engine_data *engine_data = (struct psa_engine_data
    ↪  *)data_buffers->engine_data;
695     struct psa_van_instrument_cluster_long *van_data = (struct
    ↪  psa_van_instrument_cluster_long *)data;
696 
697     engine_data->fuel_level = van_data->fuel_level;
698     engine_data->oil_temperature = van_data->oil_temp;
699 
700     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
701 
702     return MIVE_OK;
703 }
704 
705 // 0x524 len 14
706 static int psa_parse_car_status_2_short(
707     uint8_t const *const data,
708     struct psa_output_data_buffers *data_buffers)
709 {
710     mive_uart_queue_packet_t* queue_packet = NULL;
711     mive_uart_queue_packet_t queue_packet_c = {
712         .idens = {PSA_IDENT_CAR_STATUS},
713         .num_idens = 1,
714     };
715 
716     if (data_buffers->status_data == NULL)
717     {
718         return -MIVE_ERR_INVALID_ARGUMENT;
719     }
720 
721     queue_packet = get_packet_buffer();
722 
723     *queue_packet = queue_packet_c;
724 
725     struct mive_global_event queue_event = {
726         .event = MIVE_EVENT_UART_NEW_DATA,
727         .ev_data = queue_packet,
```

```
728     };
729
730
731     struct psa_status_data *status_data = (struct psa_status_data
        ↪  *)data_buffers->status_data;
732     struct psa_van_car_status_2_short *van_data = (struct psa_van_car_status_2_short
        ↪  *)data;
733
734     status_data->doors_locked = van_data->Field8.doors_locked;
735     status_data->deadlocking_active = van_data->Field8.deadlocking_active;
736
737     status_data->key_in_ignition = van_data->Field5.ignition_key_left_in;
738
739     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
740
741     return MIVE_OK;
742 }
743
744 // 0x524 len 16
745 static int psa_parse_car_status_2_long(
746     uint8_t const *const data,
747     struct psa_output_data_buffers *data_buffers)
748 {
749     mive_uart_queue_packet_t* queue_packet = NULL;
750     mive_uart_queue_packet_t queue_packet_c = {
751         .idens = {PSA_IDENT_CAR_STATUS},
752         .num_idens = 1,
753     };
754
755     if (data_buffers->status_data == NULL)
756     {
757         return -MIVE_ERR_INVALID_ARGUMENT;
758     }
759
760     queue_packet = get_packet_buffer();
761
762     *queue_packet = queue_packet_c;
763
764     struct mive_global_event queue_event = {
765         .event = MIVE_EVENT_UART_NEW_DATA,
766         .ev_data = queue_packet,
767     };
768
769     struct psa_status_data *status_data = (struct psa_status_data
        ↪  *)data_buffers->status_data;
770     struct psa_van_car_status_2_long *van_data = (struct psa_van_car_status_2_long
        ↪  *)data;
771
772     status_data->doors_locked = van_data->Field8.doors_locked;
773     status_data->deadlocking_active = van_data->Field8.deadlocking_active;
774
775     status_data->key_in_ignition = van_data->Field5.ignition_key_left_in;
776
777     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
778
779     return MIVE_OK;
780 }
781
```

```
782 static int psa_parse_cdc_command(
783     uint8_t const *const data,
784     struct psa_output_data_buffers *data_buffers)
785 {
786     mive_uart_queue_packet_t* queue_packet = NULL;
787     mive_uart_queue_packet_t queue_packet_c = {
788         .idens = {PSA_IDENT_CAR_STATUS},
789         .num_idens = 1,
790     };
791 
792     if (data_buffers->status_data == NULL)
793     {
794         return -MIVE_ERR_INVALID_ARGUMENT;
795     }
796 
797     queue_packet = get_packet_buffer();
798 
799     *queue_packet = queue_packet_c;
800 
801     struct mive_global_event queue_event = {
802         .event = MIVE_EVENT_UART_NEW_DATA,
803         .ev_data = queue_packet,
804     };
805 
806     struct psa_status_data *status_data = (struct psa_status_data
    ↪  *)data_buffers->status_data;
807     struct psa_van_cd_changer_command_struct *van_data = (struct
    ↪  psa_van_cd_changer_command_struct *)data;
808 
809     uint16_t command = be16toh(van_data->command);
810 
811     switch (command)
812     {
813     case PSA_VAN_CD_CHANGER_PLAY:
814         status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PLAY;
815         break;
816     case PSA_VAN_CD_CHANGER_PAUSE:
817         status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PAUSE;
818         break;
819     case PSA_VAN_CD_CHANGER_NEXT_TRACK:
820         status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NEXT_TRACK;
821         break;
822     case PSA_VAN_CD_CHANGER_PREVIOUS_TRACK:
823         status_data->cd_changer_command = PSA_CD_CHANGER_COMM_PREV_TRACK;
824         break;
825     default:
826         status_data->cd_changer_command = PSA_CD_CHANGER_COMM_NONE;
827         break;
828     }
829 
830     xQueueSendToBack(g_global_state.global_main_queue, &queue_event, 0);
831 
832     return MIVE_OK;
833 }
834 
835 int psa_parse_van_packet(
836     uint16_t iden,
837     uint8_t size,
```

```
    uint8_t const *const data,
    struct psa_output_data_buffers *data_buffers)
{
    if (data == NULL || data_buffers == NULL)
    {
        return -MIVE_ERR_INVALID_ARGUMENT;
    }

    switch (iden)
    {
    case PSA_VAN_IDEN_RPM:
        if (size != sizeof(struct psa_van_rpm_speed_struct))
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_rpm_iden(data, (struct psa_output_data_buffers *)data_buffers);
        break;

    case PSA_VAN_IDEN_CAR_STATUS1:
        if (size != sizeof(struct psa_van_trip_computer))
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_trip_iden(data, (struct psa_output_data_buffers *)data_buffers);
        break;

    case PSA_VAN_IDEN_ENGINE:
        if (size != sizeof(struct psa_van_dashboard))
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_dash_iden(data, (struct psa_output_data_buffers *)data_buffers);
        break;
    case PSA_VAN_IDEN_VIN:
        if (size != 17)
        {
            return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
        }

        return psa_parse_vin_iden(data, (struct psa_output_data_buffers *)data_buffers);

        break;

    case PSA_VAN_IDEN_HEAD_UNIT:
        if (size == 22)
        {
            return psa_parse_radio_tuner_iden(data, (struct psa_output_data_buffers
            ↪  *)data_buffers);
        }
        else if (size == 10)
        {
            return psa_parse_radio_cd_short_iden(data, (struct psa_output_data_buffers
            ↪  *)data_buffers);
        }
        else if (size == 19)
```

```
894         {
895             return psa_parse_radio_cd_long_iden(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
896         }
897         // else if (size == 12)
898         // {
899         //     return psa_parse_radio_preset_iden(data, (struct psa_output_data_buffers
            ↪  *)data_buffers);
900         // }
901         else
902         {
903             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
904         }
905 
906         break;
907     case PSA_VAN_IDEN_AUDIO_SETTINGS:
908         if (size != 11)
909         {
910             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
911         }
912 
913         return psa_parse_headunit_iden(data, (struct psa_output_data_buffers
            ↪  *)data_buffers);
914         break;
915 
916     case PSA_VAN_IDEN_LIGHTS_STATUS:
917         if (size == 11)
918         {
919             return psa_parse_instruments_short_iden(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
920         }
921         else if (size == 14)
922         {
923             return psa_parse_instruments_long_iden(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
924         }
925         else
926         {
927             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
928         }
929         break;
930 
931     case PSA_VAN_IDEN_CAR_STATUS2:
932         if (size == 14)
933         {
934             return psa_parse_car_status_2_short(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
935         }
936         else if (size == 16)
937         {
938             return psa_parse_car_status_2_long(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
939         }
940         else
941         {
942             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
943         }
944 
```

```
945     case PSA_VAN_IDEN_CDCHANGER_COMMAND:
946         if (size == 2)
947         {
948             return psa_parse_cdc_command(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
949         }
950         else
951         {
952             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
953         }
954 
955     case PSA_VAN_IDEN_DEVICE_REPORT:
956         if (size == 3)
957         {
958             return psa_parse_buttons_iden(data, (struct psa_output_data_buffers
                ↪  *)data_buffers);
959         }
960         else
961         {
962             return -MIVE_ERR_VAN_INVALID_PACKET_SIZE;
963         }
964 
965     default:
966         return -MIVE_ERR_VAN_UNKNOWN_IDEN;
967         break;
968     }
969 
970     return MIVE_OK;
971 }
```

Kod 9: van_rmt_rx.c

```
1  #include "freertos/FreeRTOS.h"
2  
3  #include <stdint.h>
4  #include <stdbool.h>
5  #include <string.h>
6  
7  #include "include/mive/common.h"
8  #include "include/van/van_rmt_rx.h"
9  #include "freertos/task.h"
10 #include "freertos/queue.h"
11 #include "esp_attr.h"
12 #include "driver/gpio.h"
13 #include "driver/rmt_rx.h"
14 
15 #include "esp_log.h"
16 
17 static const char *TAG = "RMT_RX";
18 
19 #define VAN_CRC_POLYNOM = 0x0F9D;
20 
21 #define VAN_CRC_TABLE_SIZE 256
22 static const uint16_t crcTable[VAN_CRC_TABLE_SIZE] =
23 {
```

```
    0x0000, 0x0F9D, 0x1F3A, 0x10A7, 0x3E74, 0x31E9, 0x214E, 0x2ED3,
    0x7CE8, 0x7375, 0x63D2, 0x6C4F, 0x429C, 0x4D01, 0x5DA6, 0x523B,
    0x764D, 0x79D0, 0x6977, 0x66EA, 0x4839, 0x47A4, 0x5703, 0x589E,
    0x0AA5, 0x0538, 0x159F, 0x1A02, 0x34D1, 0x3B4C, 0x2BEB, 0x2476,
    0x6307, 0x6C9A, 0x7C3D, 0x73A0, 0x5D73, 0x52EE, 0x4249, 0x4DD4,
    0x1FEF, 0x1072, 0x00D5, 0x0F48, 0x219B, 0x2E06, 0x3EA1, 0x313C,
    0x154A, 0x1AD7, 0x0A70, 0x05ED, 0x2B3E, 0x24A3, 0x3404, 0x3B99,
    0x69A2, 0x663F, 0x7698, 0x7905, 0x57D6, 0x584B, 0x48EC, 0x4771,
    0x4993, 0x460E, 0x56A9, 0x5934, 0x77E7, 0x787A, 0x68DD, 0x6740,
    0x357B, 0x3AE6, 0x2A41, 0x25DC, 0x0B0F, 0x0492, 0x1435, 0x1BA8,
    0x3FDE, 0x3043, 0x20E4, 0x2F79, 0x01AA, 0x0E37, 0x1E90, 0x110D,
    0x4336, 0x4CAB, 0x5C0C, 0x5391, 0x7D42, 0x72DF, 0x6278, 0x6DE5,
    0x2A94, 0x2509, 0x35AE, 0x3A33, 0x14E0, 0x1B7D, 0x0BDA, 0x0447,
    0x567C, 0x59E1, 0x4946, 0x46DB, 0x6808, 0x6795, 0x7732, 0x78AF,
    0x5CD9, 0x5344, 0x43E3, 0x4C7E, 0x62AD, 0x6D30, 0x7D97, 0x720A,
    0x2031, 0x2FAC, 0x3F0B, 0x3096, 0x1E45, 0x11D8, 0x017F, 0x0EE2,
    0x1CBB, 0x1326, 0x0381, 0x0C1C, 0x22CF, 0x2D52, 0x3DF5, 0x3268,
    0x6053, 0x6FCE, 0x7F69, 0x70F4, 0x5E27, 0x51BA, 0x411D, 0x4E80,
    0x6AF6, 0x656B, 0x75CC, 0x7A51, 0x5482, 0x5B1F, 0x4BB8, 0x4425,
    0x161E, 0x1983, 0x0924, 0x06B9, 0x286A, 0x27F7, 0x3750, 0x38CD,
    0x7FBC, 0x7021, 0x6086, 0x6F1B, 0x41C8, 0x4E55, 0x5EF2, 0x516F,
    0x0354, 0x0CC9, 0x1C6E, 0x13F3, 0x3D20, 0x32BD, 0x221A, 0x2D87,
    0x09F1, 0x066C, 0x16CB, 0x1956, 0x3785, 0x3818, 0x28BF, 0x2722,
    0x7519, 0x7A84, 0x6A23, 0x65BE, 0x4B6D, 0x44F0, 0x5457, 0x5BCA,
    0x5528, 0x5AB5, 0x4A12, 0x458F, 0x6B5C, 0x64C1, 0x7466, 0x7BFB,
    0x29C0, 0x265D, 0x36FA, 0x3967, 0x17B4, 0x1829, 0x088E, 0x0713,
    0x2365, 0x2CF8, 0x3C5F, 0x33C2, 0x1D11, 0x128C, 0x022B, 0x0DB6,
    0x5F8D, 0x5010, 0x40B7, 0x4F2A, 0x61F9, 0x6E64, 0x7EC3, 0x715E,
    0x362F, 0x39B2, 0x2915, 0x2688, 0x085B, 0x07C6, 0x1761, 0x18FC,
    0x4AC7, 0x455A, 0x55FD, 0x5A60, 0x74B3, 0x7B2E, 0x6B89, 0x6414,
    0x4062, 0x4FFF, 0x5F58, 0x50C5, 0x7E16, 0x718B, 0x612C, 0x6EB1,
    0x3C8A, 0x3317, 0x23B0, 0x2C2D, 0x02FE, 0x0D63, 0x1DC4, 0x1259,
};

IRAM_ATTR static bool rmt_rx_done_callback(rmt_channel_handle_t channel, const
↪  rmt_rx_done_event_data_t *edata, void *user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    van_rmt_rx_instance_t* van_instance = (van_rmt_rx_instance_t*)user_data;
    // send the received RMT symbols to the parser task
    rmt_receive(
        van_instance->rx_chan,
        van_instance->rmt_symbol_buffer,
        van_instance->rmt_symbol_buffer_size,
        &van_instance->rx_recv_config);
    xQueueSendFromISR(van_instance->receive_queue, edata, &high_task_wakeup);
    return high_task_wakeup == pdTRUE;
}

IRAM_ATTR bool van_rmt_rx_parse_byte(
    van_rmt_rx_instance_t* instance,
    uint8_t level,
    uint32_t duration,
    uint8_t *bitCounter,
    uint8_t *tempByte,
    uint8_t *mask,
    uint8_t *finalByte)
{
```

```
81      bool result = false;
82      if (instance->rmt_rx_van_line_level == RX_VAN_LINE_LEVEL_LOW)
83      {
84          level = !level;
85      }
86
87      // on the bus the time slices are a little off from the multiple of the TS
        ↪  microseconds, so we round it to the nearest multiple of the TS before dividing
88      uint8_t countOfTimeSlices = round_to_nearest(duration,
        ↪  instance->rmt_rx_time_slice_divisor) / instance->rmt_rx_time_slice_divisor;
89
90      for (int i = 0; i < countOfTimeSlices; i++)
91      {
92          // every 5th bit is a manchester bit, we must skip them while we are building our
            ↪  byte
93          bool isManchesterBit = (*bitCounter + 1) % 5 == 0;
94          if (!isManchesterBit)
95          {
96              if (level == 1)
97              {
98                  *tempByte |= *mask;
99              }
100             *mask = *mask >> 1;
101         }
102         else
103         {
104             // if we found the second manchester bit, then we have the full byte in the
                ↪  tempByte variable, so we place it in the finalByte and we can start
                ↪  building the next byte
105             if (*bitCounter == 9)
106             {
107                 *bitCounter = -1;
108                 *mask = 1 << 7;
109                 *finalByte = *tempByte;
110                 *tempByte = 0;
111                 result = true;
112             }
113         }
114         *bitCounter = *bitCounter + 1;
115     }
116
117     return result;
118 }
119
120 IRAM_ATTR uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData)
121 {
122     uint16_t crc15 = 0x7FFF;
123
124     for (int i = 0; i < lengthOfData; i++)  // Skip first byte (SOF, 0x0E) and last 2
        ↪  (CRC)
125     {
126         uint8_t byte = data[i];
127
128         // XOR-in next input byte into MSB of crc, that's our new intermediate divident
129         uint8_t pos = (uint8_t)( (crc15 >> 7) ^ byte);
130
131         // Shift out the MSB used for division per lookup table and XOR with the
            ↪  remainder
```

```
        crc15 = (uint16_t)((crc15 << 8) ^ (uint16_t)(crcTable[pos]));
    } // for

    crc15 ^= 0x7FFF;
    crc15 <<= 1;  // Shift left 1 bit to turn 15 bit result into 16 bit representation

    return crc15;
}

// IRAM_ATTR uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData)
// {
//     const uint8_t order = 15;
//     const uint16_t polynom = 0xF9D;
//     const uint16_t xorValue = 0x7FFF;
//     const uint16_t mask = 0x7FFF;

//     uint16_t crc = 0x7FFF;

//     for (uint8_t i = 0; i < lengthOfData; i++)
//     {
//         uint8_t currentByte = data[i];

//         // rotate one data byte including crcmask
//         for (uint8_t j = 0; j < 8; j++)
//         {
//             bool bit = (crc & (1 << (order - 1))) != 0;
//             if ((currentByte & 0x80) != 0)
//             {
//                 bit = !bit;
//             }
//             currentByte <<= 1;

//             crc = ((crc << 1) & mask) ^ (-bit & polynom);
//         }
//     }

//     // perform xor and multiply result by 2 to turn 15 bit result into 16 bit
↪  representation
//     return (crc ^ xorValue) << 1;
// }

IRAM_ATTR bool van_rmt_rx_is_crc_ok(uint8_t vanMessage[], int vanMessageLength)
{
    bool retval = false;
    uint8_t crcByte1 = 0;
    uint8_t crcByte2 = 0;
    uint16_t crcValueInMessage = 0;

    // Check if message length is even valid (Start byte + iden + crc)
    if(vanMessageLength < 5)
    {
        return false;
    }

    crcByte1 = vanMessage[vanMessageLength - 2];
    crcByte2 = vanMessage[vanMessageLength - 1];
    crcValueInMessage = crcByte1 << 8 | crcByte2;

```

```
    if (vanMessageLength - 3 <= 32)
    {
        uint16_t calculatedCrc = van_rmt_rx_crc15(vanMessage + 1, vanMessageLength - 3);
        if(crcValueInMessage == calculatedCrc)
        {
            retval = true;
        }
        else
        {
            ESP_LOGE(TAG, "[%s] Calculated: 0x%04x, Received: 0x%04x", __func__,
            ↪  calculatedCrc, crcValueInMessage);
            retval = false;
        }
    }
    return retval;
}

/* Uninstall the RMT driver */
void van_rmt_rx_channel_stop_new(van_rmt_rx_instance_t* instance)
{
    rmt_disable(instance->rx_chan);
}

void van_rmt_rx_channel_start_new(van_rmt_rx_instance_t* instance)
{
    rmt_enable(instance->rx_chan);
}

void van_rmt_rx_channel_init_new(
    van_rmt_rx_instance_t* instance,
    uint8_t channel,
    int rxPin,
    int ledPin,
    RX_VAN_LINE_LEVEL vanLineLevel,
    RX_VAN_NETWORK_TYPE vanNetworkType)
{
    rmt_rx_channel_config_t rx_chan_config = {0};
    rmt_receive_config_t rx_recv_config = {0};
    rmt_channel_handle_t rx_chan;
    uint32_t timeslot_interval_ns = 0;
    uint8_t _rmt_van_rx_time_slice_divisor;
    QueueHandle_t receive_queue = NULL;
    rmt_symbol_word_t* rmt_symbol_buffer = NULL;

    // TS = 8us = 125k
    // TS = 16us = 62.5k

    if(rxPin < 0)
    {
        ESP_LOGE(TAG, "[%s] Invalid rxPin: %d", __func__, rxPin);
        return;
    }

    rx_chan_config.gpio_num = rxPin;
    rx_chan_config.clk_src = RMT_CLK_SRC_DEFAULT;
    rx_chan_config.resolution_hz = 1000000; // 1 MHz
    rx_chan_config.mem_block_symbols = 512;
    // RMT DMA not supported on ESP32
```

```
    rx_chan_config.flags.with_dma = 0;

    if (vanNetworkType == RX_VAN_NETWORK_COMFORT)
    {
        _rmt_van_rx_time_slice_divisor = 8;
        timeslot_interval_ns = 8 * 1000;
    }
    else
    {
        _rmt_van_rx_time_slice_divisor = 16;
        timeslot_interval_ns = 16 * 1000;
    }

    rx_recv_config.signal_range_max_ns = 10 * timeslot_interval_ns;
    rx_recv_config.signal_range_min_ns = timeslot_interval_ns /
    ↪  _rmt_van_rx_time_slice_divisor;

    rmt_new_rx_channel(&rx_chan_config, &rx_chan);

    receive_queue = xQueueCreate(30, sizeof(rmt_rx_done_event_data_t));

    rmt_rx_event_callbacks_t cbs = {
        .on_recv_done = rmt_rx_done_callback
    };
    rmt_rx_register_event_callbacks(rx_chan, &cbs, instance);

    rmt_symbol_buffer = heap_caps_malloc(2048, MALLOC_CAP_DMA);

    instance->rmt_rx_channel = channel;
    instance->rmt_rx_rxPin = rxPin;
    instance->rmt_rx_van_line_level = vanLineLevel;
    instance->rmt_rx_time_slice_divisor = _rmt_van_rx_time_slice_divisor;
    instance->rx_chan = rx_chan;
    instance->rx_recv_config = rx_recv_config;
    instance->receive_queue = receive_queue;
    instance->rmt_symbol_buffer = rmt_symbol_buffer;
    instance->rmt_symbol_buffer_size = 2048;
}
```

Kod 10: van_rmt_task.c

```
#include "freertos/FreeRTOS.h"

#include <string.h>
#include <stdlib.h>

#include "esp_attr.h"
#include "freertos/queue.h"

#include "driver/gpio.h"
#include "include/mive/common.h"
#include "include/global_tasks.h"
#include "include/global_state_def.h"
#include "include/global_config.h"
#include "include/van/van_rmt_rx.h"
```

```
15
16  van_rmt_rx_instance_t global_van_instance;
17
18  #define VAN_MAX_NUM_PACKETS 50
19
20  static uint8_t* buffers[VAN_MAX_NUM_PACKETS];
21
22  static int van_parse_bytes(van_rmt_rx_instance_t* instance, mive_van_packet_t* packet,
    ↪  rmt_rx_done_event_data_t rx_data, int packet_index)
23  {
24      rmt_symbol_word_t* items;
25
26      uint8_t bitCounter = 0;
27      uint8_t mask = 1 << 7;
28      uint8_t tempByte = 0;
29      uint8_t finalByte = 0;
30      int van_message_length = 0;
31      size_t i = 0;
32      bool isCompleteByte = false;
33      int retval = 0;
34
35      uint8_t temp_array[VAN_MAX_PACKET_LEN + 10] = {0};
36
37      items = rx_data.received_symbols;
38      // printf("Got %d symbols\n", rx_data.num_symbols);
39      for (i = 0; i < rx_data.num_symbols; i++)
40      {
41          if(van_message_length >= VAN_MAX_PACKET_LEN)
42          {
43              break;
44          }
45          isCompleteByte = van_rmt_rx_parse_byte(instance, items[i].level0,
            ↪  items[i].duration0, &bitCounter, &tempByte, &mask, &finalByte);
46          if (isCompleteByte)
47          {
48              temp_array[van_message_length] = finalByte;
49              van_message_length++;
50          }
51
52          isCompleteByte = van_rmt_rx_parse_byte(instance, items[i].level1,
            ↪  items[i].duration1, &bitCounter, &tempByte, &mask, &finalByte);
53          if (isCompleteByte)
54          {
55              temp_array[van_message_length] = finalByte;
56              van_message_length++;
57          }
58      }
59
60      // printf("[%s] Received: ", __func__);
61      // for(int i = 0; i < van_message_length; ++i)
62      // {
63      //     printf("0x%02x ", temp_array[i]);
64      // }
65
66      // printf("\n");
67
68      if((van_message_length > 5) &&
69         (van_message_length < VAN_MAX_PACKET_LEN) &&
```

```
70          (van_rmt_rx_is_crc_ok(temp_array, van_message_length)))
71      {
72          packet->iden = (((uint16_t)temp_array[1] << 8) | ((uint16_t)temp_array[2] &
            ↪  0xf0)) >> 4;
73          packet->packet_size = van_message_length - 5;
74          packet->packet = buffers[packet_index];
75          memset(packet->packet, 0, packet->packet_size);
76          memcpy(packet->packet, temp_array + 3, packet->packet_size);
77          retval = MIVE_OK;
78      }
79      else
80      {
81          // printf("[%s] CRC error\n", __func__);
82          retval = -MIVE_ERR_VAN_CRC_ERROR;
83      }
84      //  0e 5e 48 a0 20 11 d4
85
86      return retval;
87  }
88
89  void van_rmt_task(void* params)
90  {
91      int ret = 0;
92      unsigned int packet_num = 0;
93      mive_global_state_t *state = (mive_global_state_t*)params;
94      state->van_rmt_instance = &global_van_instance;
95      van_rmt_rx_instance_t* van_instance = &global_van_instance;
96      mive_van_packet_t* van_packets = NULL;
97      QueueHandle_t receive_queue;
98      QueueHandle_t global_queue_to_main;
99      rmt_rx_done_event_data_t rx_data;
100     struct mive_global_event event_item_to_send = {.event =
        ↪  MIVE_EVENT_RMT_NEW_VAN_FRAME};
101
102     printf("[%s] Starting...\n", __func__);
103
104     van_rmt_rx_channel_init_new(
105         van_instance,
106         0,
107         VAN_RX_PIN,
108         VAN_RX_LED_PIN,
109         RX_VAN_LINE_LEVEL_HIGH,
110         RX_VAN_NETWORK_COMFORT);
111
112
113     van_rmt_rx_channel_start_new(van_instance);
114
115     rmt_receive(
116         van_instance->rx_chan,
117         van_instance->rmt_symbol_buffer,
118         van_instance->rmt_symbol_buffer_size,
119         &van_instance->rx_recv_config);
120
121     receive_queue = van_instance->receive_queue;
122     global_queue_to_main = state->global_main_queue;
123
124     van_packets = calloc(VAN_MAX_NUM_PACKETS, sizeof(*van_packets));
125
```

```
126     for(int i = 0; i < VAN_MAX_NUM_PACKETS; ++i)
127     {
128         buffers[i] = calloc(1, VAN_MAX_PACKET_LEN);
129     }
130 
131     while (true)
132     {
133         if(xQueueReceive(receive_queue, &rx_data, pdMS_TO_TICKS(1000)) == pdPASS)
134         {
135             ret = van_parse_bytes(van_instance, &van_packets[packet_num], rx_data,
                ↪  packet_num);
136             if(ret == MIVE_OK)
137             {
138                 event_item_to_send.ev_data = &van_packets[packet_num];
139                 // printf("[%s] Sending to queue...\n", __func__);
140                 xQueueSendToBack(global_queue_to_main, &event_item_to_send, 0);
141 
142                 packet_num = (packet_num == VAN_MAX_NUM_PACKETS - 1) ? 0 : packet_num +
                    ↪  1;
143             }
144         }
145     }
146 
147 }
```

Kod 11: include/global_config.h

```
1  #ifndef MIVE_GLOBAL_CONFIG_H
2  #define MIVE_GLOBAL_CONFIG_H
3  
4  // #include "soc/gpio_num.h"
5  
6  
7  #define DEVKIT 1
8  #define PEZO 2
9  
10 // #define ESP32_BOARD_TYPE PEZO
11 #define ESP32_BOARD_TYPE PEZO
12 
13 #if (ESP32_BOARD_TYPE == PEZO)
14 
15 #define VAN_RX_PIN GPIO_NUM_21
16 #define VAN_RX_LED_PIN -1
17 #define TSS_CS_PIN 22
18 #define TSS_INT_PIN 19
19 #define PSA_ADC_UNIT 0
20 #define PSA_ADC_CHANNEL 5
21 #define PSA_EXT_REG_PIN 27
22 #define TSS_OE_ENABLE_PIN 32
23 
24 #elif (ESP32_BOARD_TYPE == DEVKIT)
25 
26 #define VAN_RX_PIN GPIO_NUM_21
27 #define VAN_RX_LED_PIN GPIO_NUM_2
28 #define TSS_CS_PIN 5
```

```
#define TSS_INT_PIN 27
#define PSA_ADC_UNIT 0
#define PSA_ADC_CHANNEL 5
#define PSA_EXT_REG_PIN -1
#define TSS_OE_ENABLE_PIN -1

#else

#define VAN_RX_PIN -1
#define VAN_RX_LED_PIN -1
#define TSS_CS_PIN -1
#define TSS_INT_PIN -1
#define PSA_ADC_UNIT -1
#define PSA_ADC_CHANNEL -1
#define PSA_EXT_REG_PIN -1
#define TSS_OE_ENABLE_PIN -1

#endif

#endif // GLOBAL_CONFIG_H
```

Kod 12: include/global_init.h

```
#ifndef _PSA_GLOBAL_INIT_H
#define _PSA_GLOBAL_INIT_H

void init_adc(void);

#endif
```

Kod 13: include/global_state_def.h

```
#ifndef MIVE_GLOBAL_STATE_H
#define MIVE_GLOBAL_STATE_H

#include "freertos/queue.h"

#include "global_tasks.h"

#include "van/tss463c.h"
#include "van/van_rmt_rx.h"

#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali_scheme.h"

struct mive_global_state_t
{
    // Tasks to VAN
    QueueHandle_t global_van_queue;
    // Tasks to tss
    QueueHandle_t global_tss_queue;
    // Tasks to uart
```

```
    QueueHandle_t global_uart_queue;
    // Tasks to main
    QueueHandle_t global_main_queue;

    tss_instance_t *tss_instance;
    van_rmt_rx_instance_t *van_rmt_instance;

    adc_oneshot_unit_handle_t adc_handle;
    adc_cali_handle_t cali_handle;

};

extern struct mive_global_state_t g_global_state;
extern volatile int g_global_car_state;
extern volatile float g_bat_voltage;

#endif // MIVE_GLOBAL_STATE_H
```

Kod 14: include/global_tasks.h

```
#ifndef MIVE_GLOBAL_TASKS_H
#define MIVE_GLOBAL_TASKS_H

typedef struct mive_global_state_t mive_global_state_t;

enum mive_global_event_t
{
    MIVE_EVENT_NONE = 0,
    MIVE_EVENT_GLOBAL_SLEEP,
    MIVE_EVENT_RMT_NEW_VAN_FRAME,
    MIVE_EVENT_TSS_RESET,
    MIVE_EVENT_TSS_ACTIVATE,
    MIVE_EVENT_TSS_IDLE,
    MIVE_EVENT_TSS_SLEEP,
    MIVE_EVENT_TSS_WRITE_FRAME,
    MIVE_EVENT_TSS_MONITOR_CHANNEL,

    MIVE_EVENT_VAN_UPDATE_AUDIO_MENU,
    MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM,
    MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,

    MIVE_EVENT_UART_NEW_DATA, // New VAN Data to send
    MIVE_EVENT_UART_RECEIVE, // Received data from UART
    MIVE_EVENT_UART_SEND, // Send data to UART

    MIVE_EVENT_TIMER_UART_SEND,
    MIVE_EVENT_TIMER_1MS,
    MIVE_EVENT_TIMER_1S,

    MIVE_EVENT_ADC,
    MIVE_EVENT_STATE_CHANGE,
    MIVE_EVENT_POWER,
};

struct mive_global_event
```

```
{
    enum mive_global_event_t event;
    void* ev_data;
};

/* VAN Task type definitions */

#define VAN_MAX_PACKET_LEN 40

// ev_data for MIVE_EVENT_RMT_NEW_VAN_FRAME
typedef struct mive_van_packet
{
    uint16_t iden;
    uint8_t packet_size;
    uint8_t* packet;

} mive_van_packet_t;

/* TSS Task type definitions */

typedef struct mive_tss_task_packet
{
    uint16_t iden;
    uint8_t message_type; // enum tss_message_type in tss463.h
    uint8_t message_channel;
    uint8_t packet[VAN_MAX_PACKET_LEN];
    uint8_t packet_size;
} mive_tss_task_packet_t;

/*
UART Task event data
    Main -> UART task = Send packet
    UART task -> Main = Received packet
*/
typedef struct mive_uart_task_packet
{
    uint16_t iden; // uart message identifier
    uint8_t data_size; // Size of the data
    uint8_t data[128]; // Data portion of the packet
} mive_uart_task_packet_t;

/*
UART Queue event data.
New VAN data parsed -> queue uart send

*/
typedef struct mive_uart_queue_packet
{
    uint16_t num_idens; // Number of idens to send
    uint16_t idens[10]; // uart message identifiers
} mive_uart_queue_packet_t;

/* Task function definitions */

void van_rmt_task(void* params);
void tss_task(void* params);
void uart_task(void* params);

```

```
94 #endif // MIVE_GLOBAL_TASKS_H
```

Kod 15: include/mive/common.h

```
1  #ifndef MIVE_COMMON_H
2  #define MIVE_COMMON_H
3
4  #include <stdint.h>
5
6  typedef enum {
7      MIVE_OK = 0,
8      MIVE_ERR,
9      MIVE_ERR_INVALID_ARGUMENT,
10     MIVE_ERR_NOT_IMPLEMENTED,
11     MIVE_ERR_OUTOFMEMORY,
12     MIVE_ERR_VAN_CRC_ERROR,
13     MIVE_ERR_TSS_BUSY,
14     MIVE_ERR_CHANNEL_BUSY,
15     MIVE_ERR_VAN_UNKNOWN_IDEN,
16     MIVE_ERR_VAN_INVALID_PACKET_SIZE,
17 } mive_error;
18
19 uint8_t round_to_nearest(uint8_t num_to_round, uint8_t multiple);
20 uint16_t get_iden_from_bytes(uint8_t const byte1, uint8_t const byte2);
21 uint8_t dec_to_bcd(uint8_t input);
22 uint8_t bcd_to_dec(uint8_t value);
23 uint8_t psa_crc8_checksum(const uint8_t * ptr, uint32_t len);
24
25
26 #endif // MIVE_COMMON_H
```

Kod 16: include/mive/psa_helper.h

```
1  #ifndef _PSA_HELPER_H
2  #define _PSA_HELPER_H
3
4  #include <stdint.h>
5  #include "include/global_tasks.h"
6  #include "include/van/van_packet_parser.h"
7
8  #define PSA_MAIN_UART_SEND_BUFFERS_NUM 20
9
10 enum psa_radio_preset_band
11 {
12     PSA_PRESET_AM = 0,
13     PSA_PRESET_FM_1 = 1,
14     PSA_PRESET_FM_2 = 2,
15     PSA_PRESET_FMAST = 3,
16     PSA_PRESET_MAX = 4,
17 };
18
19 extern struct psa_output_data_buffers global_libpsa_buffers;
```

```
20
21 extern mive_uart_task_packet_t* global_uart_send_buffers;
22 extern mive_uart_task_packet_t* global_uart_receive_buffers;
23
24 extern void libpsa_send_packet(uint16_t ident);
25 extern void init_libpsa_packets(void);
26 extern void libpsa_update_audio_settings_packet(int audio_menu_open, int
   ↪  audio_menu_setting);
27
28 #endif // _PSA_HELPER_H
```

Kod 17: include/mive/psa_packet_defs.h

```
1  #ifndef PSA_PACKET_DEFS_H
2  #define PSA_PACKET_DEFS_H
3
4  #include <stdint.h>
5
6  #define PSA_MSP_START_MAGIC 0x69
7  #define PSA_MSP_MAX_SIZE 100
8
9
10 enum psa_idents
11 {
12     PSA_IDENT_VERSIONS = 0x4000,      // Send API and firmware version numbers
13     PSA_IDENT_VIN = 0x4001,           // Send the VIN packet
14     PSA_IDENT_ENGINE = 0x4002,        // Send the Engine packet
15     PSA_IDENT_RADIO = 0x4003,         // Send the Radio packet
16     PSA_IDENT_RADIO_PRESETS = 0x4004, // Sent the Presets packet
17     PSA_IDENT_DOORS = 0x4005,         // Send the Doors packet
18     PSA_IDENT_DASHBOARD = 0x4006,     // Send the Dashboard packet
19     PSA_IDENT_TRIP = 0x4007,          // Send the Trip packet
20     PSA_IDENT_HEADUNIT = 0x4008,      // Send the Headunit packet
21     PSA_IDENT_CAR_STATUS = 0x4009,    // Send the Car status packet
22     PSA_IDENT_CD_PLAYER = 0x4010,     // Send the CD player packet
23
24     PSA_IDENT_SET_CD_CHANGER_DATA = 0x4501, // Send CD changer player data
25     PSA_IDENT_SET_TRIP_RESET = 0x4502,      // Send trip reset request
26
27     PSA_IDENT_TIME = 0x5000,       // Send the ESP RTC time
28     PSA_IDENT_ESP32_TEMP = 0x5100, // Send the ESP temperature, borked
29
30     PSA_IDENT_MAIN_APP = 0x6000, // Special ident used for my app, sends all packets as
       ↪  one packet. UNUSED
31 };
32
33 #define PSA_MSP_MIN_IDENT PSA_IDENT_VERSIONS
34
35 enum psa_msp_response
36 {
37     PSA_MSP_OK = 0x0E,                // Incoming packet parsed successfully
38     PSA_MSP_CRC_ERROR = 0xAA,         // Incoming packet has invalid CRC
39     PSA_MSP_UNKNOWN_IDENT = 0xBB,     // Incoming packet has unknown ident
40     PSA_MSP_INVALID_DATA_SIZE = 0xCC, // Received data size is not valid for sent ident
41     PSA_MSP_INVALID_DATA = 0xDD,      // Received data is invalid, possibly out of bounds
```

```
42      PSA_MSP_UNKNOWN_ERROR = 0xFF,     // Worst case scenario error, shouldn't appear
        ↪  normally
43  };
44
45  enum psa_radio_band
46  {
47      PSA_NONE = 0,
48      PSA_AM_1 = 1, // AM 1 band
49      PSA_AM_2,     // AM 2 band
50      PSA_AM_3,     // AM 3 band
51      PSA_FM_1,     // FM 1 band
52      PSA_FM_2,     // FM 2 band
53      PSA_FM_3,     // FM 3 band
54      PSA_FM_AST    // FMast band
55  };
56
57  enum psa_radio_source
58  {
59      PSA_RADIO_NONE = (uint8_t)0,
60      PSA_RADIO_TUNER = (uint8_t)1,      // Radio tuner
61      PSA_RADIO_INTERNAL = (uint8_t)2,   // Internal CD or tape player
62      PSA_RADIO_EXTERNAL = (uint8_t)3,   // CD changer
63      PSA_RADIO_NAVIGATION = (uint8_t)4, // Navigation audio
64  };
65
66  /**
67   * @brief Check the Documentation for an explanation
68   *
69   */
70  enum psa_car_state
71  {
72      PSA_STATE_CAR_SLEEP = 0, // Car in sleep, no power to VAN+
73      PSA_STATE_ECONOMY_MODE,  // Car is in Battery-saving mode
74      PSA_STATE_CAR_LOCKED,    // Car off and doors locked
75      PSA_STATE_CAR_OFF,       // Car off, radio off
76      PSA_STATE_RADIO_ON,      // Car off, radio on
77      PSA_STATE_ACCESSORY,     // Key in accessory position
78      PSA_STATE_IGNITION,      // Key in ignition position
79      PSA_STATE_CRANKING,      // Engine cranking
80      PSA_STATE_ENGINE_ON,     // Engine ON.
81      PSA_STATE_UNKNOWN,       // Some logic is wrong, this should not appear
82  };
83
84  enum psa_trip_meter
85  {
86      PSA_TRIP_A = 1,
87      PSA_TRIP_B = 2,
88  };
89
90  /**
91   * @brief Doesnt work at the moment
92   *
93   */
94  enum psa_cd_player_status
95  {
96      PSA_CD_PLAYER_PLAYING = (uint8_t)(1 << 0),
97      PSA_CD_PLAYER_PAUSED = (uint8_t)(1 << 1),
98      PSA_CD_PLAYER_FAST_FORWARDING = (uint8_t)(1 << 2),
```

```
    PSA_CD_PLAYER_REWINDING = (uint8_t)(1 << 3),
    PSA_CD_PLAYER_LOADING = (uint8_t)(1 << 4),
    PSA_CD_PLAYER_EJECTING = (uint8_t)(1 << 5),
    PSA_CD_PLAYER_ERROR = (uint8_t)(1 << 6),
};

enum psa_cd_changer_status
{
    PSA_CD_CHANGER_OFF = (uint8_t)0x41,
    PSA_CD_CHANGER_ON = (uint8_t)0xC1,
    PSA_CD_CHANGER_BUSY = (uint8_t)0xD3,
    PSA_CD_CHANGER_PLAYING = (uint8_t)0xC3,
    PSA_CD_CHANGER_FAST_FORWARD = (uint8_t)0xC4,
    PSA_CD_CHANGER_FAST_REWIND = (uint8_t)0xC5,
};

enum psa_cd_changer_commands
{
    PSA_CD_CHANGER_COMM_NONE = 0x00,       // No new command
    PSA_CD_CHANGER_COMM_PLAY = 0x10,       // Command to start playback
    PSA_CD_CHANGER_COMM_PAUSE = 0x11,      // Command to pause playback
    PSA_CD_CHANGER_COMM_PREV_TRACK = 0x20, // Command to go to previous track
    PSA_CD_CHANGER_COMM_NEXT_TRACK = 0x21, // Command to go to next track
};

struct psa_header
{
    /* Packet start, 0x69 */
    uint8_t start;
    /* < when sending to the esp, > when receiving from the esp */
    uint8_t direction;
    /* Packet ident. Taken into CRC calc */
    uint16_t ident;
    /* Payload size. Taken into CRC calc */
    uint8_t size;
} __attribute__((packed));

struct psa_version_data
{
    uint8_t firmware_version_major;
    uint8_t firmware_version_minor;
    uint8_t api_version;
} __attribute__((packed));

struct psa_version_packet
{
    struct psa_header header;
    struct psa_version_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_engine_data
{
    uint16_t rpm;               // RPM * 8
    uint16_t speed;             // km/h * 100
    uint8_t fuel_level;         // Fuel level in %. Does not appear on all cars
    uint8_t engine_temperature; // Engine water temperature + 40 in Celcius. To display,
    ↪  subtract 40. 0xFF when invalid
```

```
    uint8_t reverse;            // Is the car in reverse
    uint8_t oil_temperature;    // Oil temperature + 40 in Celcius. 0xFF when invalid
    uint8_t reserved;           // reserved
} __attribute__((packed));

struct psa_engine_packet
{
    struct psa_header header;
    struct psa_engine_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_radio_data
{
    uint16_t freq;           // freq * 100
    uint8_t band;            // Radio band, enum psa_radio_band
    uint8_t preset;          // Preset number 1 - 6
    uint8_t signal_strength; // Signal Strength 0 - 15, probably. The values are random
    char station[9];         // ASCII station name. Do not treat as NULL-terminated.
} __attribute__((packed));

struct psa_radio_packet
{
    struct psa_header header;
    struct psa_radio_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_preset_info
{
    uint8_t preset_num;   // Preset number 1 - 6
    char preset_name[10]; // Preset station name
} __attribute__((packed));

struct psa_preset_data
{
    struct psa_preset_info presets[6];
} __attribute__((packed));

struct psa_preset_packet
{
    struct psa_header header;
    struct psa_preset_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_cd_player_data
{
    uint8_t status; // enum psa_cd_player_status

    uint8_t current_minute; // Current minute in decimal
    uint8_t total_minutes;  // Total minuts in decimal
    uint8_t current_second; // Current second in decimal
    uint8_t total_seconds;  // Not total_minutes*60, but ss in mm:ss. In decimal
    uint8_t current_track;  // Current track number in decimal
    uint8_t total_tracks;   // Total number of tracks in decimal
} __attribute__((packed));

```

```
struct psa_cd_player_packet
{
    struct psa_header header;
    struct psa_cd_player_data data;
    uint8_t crc;
} __attribute__((packed));

/**
 * @brief What settings are being changed (selected).
 * Used for displaying audio settings display
 *
 */
struct psa_headunit_settings
{
    uint8_t volume_changing : 1;      // The volume is changing. Not used
    uint8_t bass_changing : 1;        // The bass value is changing / selected
    uint8_t treble_changing : 1;      // The treble value is changing / selected
    uint8_t fader_changing : 1;       // The fader value is changing / selected
    uint8_t balance_changing : 1;     // The balance value is changing / selected
    uint8_t auto_volume_changing : 1; // The auto-volume value is changing / selected
    uint8_t loudness_changing : 1;    // The LOUDNESS value is changing / selected
    uint8_t : 1;                      // Unused
} __attribute__((packed));

struct psa_headunit_data
{
    uint8_t unit_powered_on;    // 1 if powered on
    uint8_t source;             // enum psa_radio_source
    uint8_t muted;              // is muted
    uint8_t cd_present;         // is CD present
    uint8_t tape_present;       // is TAPE present
    uint8_t volume;             // Current volume
    int8_t bass;                // Current bass
    int8_t treble;              // Current treble
    int8_t fader;               // Current fader
    int8_t balance;             // Current balance
    uint8_t auto_volume;        // Current auto volume setting
    uint8_t loudness_on;        // Current loudness setting
    uint8_t settings_menu_open; // Is settings menu open. Not always accurate.

    struct psa_headunit_settings settings_changing; // What settings are being updated

} __attribute__((packed));

struct psa_headunit_packet
{
    struct psa_header header;
    struct psa_headunit_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_door_data
{
    uint8_t door_open; // Is a door open?

    uint8_t open_doors; // What doors are open. Check below comment for clarification

    /*
```

```
272         Open doors bitfields:
273 
274         uint8_t fuel_flap : 1;   // bit 0
275         uint8_t sunroof : 1;     // bit 1
276         uint8_t hood : 1;        // bit 2
277         uint8_t boot_lid : 1;    // bit 3
278         uint8_t rear_left : 1;   // bit 4
279         uint8_t rear_right : 1;  // bit 5
280         uint8_t front_left : 1;  // bit 6
281         uint8_t front_right : 1; // bit 7
282     */
283 } __attribute__((packed));
284 
285 struct psa_door_packet
286 {
287     struct psa_header header;
288     struct psa_door_data data;
289     uint8_t crc;
290 } __attribute__((packed));
291 
292 struct psa_dash_data
293 {
294     uint8_t brightness;        // 0 to 15. Dashboard brightness
295     uint8_t backlight_off : 1; // 1 when interior backlight is off
296 
297     uint8_t accesories_on : 1;  // Key in ACC
298     uint8_t ignition_on : 1;    // Key in IGN
299     uint8_t engine_running : 1; // Engine on
300     uint8_t door_open : 1;      // A door is open
301     uint8_t economy_mode : 1;   // ECO MODE for the radio.
302     uint8_t reverse_gear : 1;   // Gearbox in reverse
303 
304     uint8_t water_temp;    // Subtract by 40
305     uint32_t mileage;      // Divide by 10
306     uint8_t external_temp; // 0xFF if not present, otherwise divide by 2 and subtract 40.
307 
308 } __attribute__((packed));
309 
310 struct psa_dash_packet
311 {
312     struct psa_header header;
313     struct psa_dash_data data;
314     uint8_t crc;
315 
316 } __attribute__((packed));
317 
318 /**
319  * @brief Data from the trip computer.
320  *
321  *
322  */
323 struct psa_trip_data
324 {
325     uint16_t trip_1_distance;
326     uint16_t trip_2_distance;
327     uint16_t trip_1_fuel_consumption;
328     uint16_t trip_2_fuel_consumption;
329     uint16_t current_fuel_consumption;
```

```
330     uint16_t distance_to_empty;
331 
332     uint8_t trip_1_speed;
333     uint8_t trip_2_speed;
334     uint8_t trip_button_pressed;
335 
336 } __attribute__((packed));
337 
338 struct psa_trip_packet
339 {
340     struct psa_header header;
341     struct psa_trip_data data;
342     uint8_t crc;
343 
344 } __attribute__((packed));
345 
346 /**
347  * @brief Random assortment of data that doesnt fit anywhere else.
348  * Also send the car state, and cd changer command.
349  *
350  */
351 struct psa_status_data
352 {
353     uint8_t doors_locked;
354     uint8_t deadlocking_active;
355     uint8_t car_state;
356     uint8_t cd_changer_command;
357     uint16_t voltage;
358     uint8_t key_in_ignition;
359     uint8_t reserved[11];
360 } __attribute__((packed));
361 
362 struct psa_status_packet
363 {
364     struct psa_header header;
365     struct psa_status_data data;
366     uint8_t crc;
367 } __attribute__((packed));
368 
369 /**
370  * @brief The current time as reported by the ESP
371  *
372  */
373 struct psa_time_data
374 {
375     uint8_t valid_time; // 1 if time is set
376     uint8_t day;        // 1 - 31
377     uint8_t month;      // 0 - 11
378     uint8_t year;       // years since 1900
379     uint8_t hour;       // 0 - 24
380     uint8_t minutes;    // 0 - 59
381     uint8_t seconds;    // 0 - 61
382 } __attribute__((packed));
383 
384 struct psa_time_packet
385 {
386     struct psa_header header;
387     struct psa_time_data data;
```

```
    uint8_t crc;
} __attribute__((packed));

struct psa_esp32_temp_data
{
    uint16_t temperature;
    uint16_t cpu_freq;
} __attribute__((packed));

struct psa_esp32_temp_packet
{
    struct psa_header header;
    struct psa_esp32_temp_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_vin_packet
{
    struct psa_header header;
    uint8_t vin[17]; // VIN in ASCII. NOT null-terminated
    uint8_t crc;

} __attribute__((packed));

// SETTER STRUCTS

struct psa_cd_changer_data
{
    uint8_t status;             // enum psa_cd_changer_status
    uint8_t track_time_seconds; // Track seconds in decimal
    uint8_t track_time_minutes; // Track minutes in decimal
    uint8_t track_number;       // Track number in decimal
    uint8_t cd_number;          // CD number in decimal
    uint8_t total_tracks;       // Total tracks in decimal
    uint8_t cds_present;        // Bitwise presentation of present cds
} __attribute__((packed));

struct psa_cd_changer_packet
{
    struct psa_header header;
    struct psa_cd_changer_data data;
    uint8_t crc;
} __attribute__((packed));

struct psa_trip_reset_data
{
    uint8_t trip_meter; // enum psa_trip_meter, trip meter to reset
} __attribute__((packed));

struct psa_trip_reset_packet
{
    struct psa_header header;
    struct psa_trip_reset_data data;
    uint8_t crc;
} __attribute__((packed));

#endif
```

Kod 18: include/van/tss463_regs.h

```
#ifndef TSS463C_REGS_H
#define TSS463C_REGS_H

#include <stdint.h>

// Register addresses

#define TSS_LINECONTROL 0x00       // r/w   - 0x00
#define TSS_TRANSMITCONTROL 0x01   // r/w   - 0x02
#define TSS_DIAGNOSISCONTROL 0x02  // r/w   - 0x00
#define TSS_COMMANDREGISTER 0x03   // w     - 0x00
#define TSS_LINESTATUS 0x04        // r     - 0bx01xxx00
#define TSS_TRANSMITSTATUS 0x05    // r     - 0x00
#define TSS_LASTMESSAGESTATUS 0x06 // r     - 0x00
#define TSS_LASTERRORSTATUS 0x07   // r     - 0x00

/* Interrupt registers */

#define TSS_INTERRUPTSTATUS 0x09   // r     - 0x80
#define TSS_INTERRUPTENABLE 0x0A   // r/w   - 0x80
#define TSS_INTERRUPTRESET 0x0B    // w

// Used with Interrupt Status regiter, Interrupt enable register and Interrupt reset register
typedef union
{
    struct
    {
        // Receive "with no RAK (RAK=0)" OK Status Flag
        uint8_t RNOK : 1;
        // Receive "with RAK (RAK=1)" OK Status Flag
        uint8_t ROK : 1;
        // Receive Error Status Flag
        uint8_t RE : 1;
        // Transmit OK Status Flag
        uint8_t TOK : 1;
        // Transmit Error Status Flag (or Exceeded Retry)
        uint8_t TE : 1;
        // Must be 0
        uint8_t ZERO : 2;
        // Interrupt on chip reset, must be 1 in interrupt enable
        uint8_t RESET : 1;
    } data;
    uint8_t Value;
} interrupt_register_t;

typedef union
{
    struct
    {
        uint8_t CHRx : 1; // Channel message received
        uint8_t CHTx : 1; // Channel message transmitted
        /*
        As status, this bit is set by the TSS463C when error occurs in transmission or on a received frame. The user must reset it
```

```
        */
        uint8_t CHER : 1;
        /*
        The 5 high bits of this register allows the user to specify either the length of the message to be transmitted, or the maximum length of a message receivable in the pointed reception buffer
        The first byte in this register does not contain data, but the length of the message received. This implies that the length value has to be equal to or greater than the maximum length of a message
        to be received in this buffer (or the length of a message to be transmitted) plus 1
        */
        uint8_t M_L : 5;
    } data;
    uint8_t Value;
} message_length_and_status_register_t;

typedef union
{
    struct
    {
        // Length of the received message
        uint8_t received_message_len : 5;
        // Status of the received RTR bit
        uint8_t RTR : 1;
        // Status of the received RNW bit
        uint8_t RNW : 1;
        // Status of the received RAK bit
        uint8_t RRAK : 1;
    } data;
    uint8_t Value;
} received_message_and_status_register_t;

typedef union
{
    struct
    {
        uint8_t RTR : 1;
        uint8_t RNW : 1;
        uint8_t RAK : 1;
        uint8_t EXT : 1;
        uint8_t ID : 4;
    } data;
    uint8_t Value;
} id2_and_command_register_t;

typedef union
{
    struct
    {
        // Message Pointer the value in this register is the offset from 0x80
        uint8_t message_pointer : 7;
        /*Disable RAK (Used in 'Spy Mode')
        In reception: whatever is the RAK bit of the incoming valid frame, no ACK answer will be set. If the message was successfully received, an IT is set (ROK or RNOK).
        In transmission: no action.
        One: disable active, 'spy mode'.
```

```
        Zero: disable inactive, normal operation
        */
        uint8_t DRAK : 1;
    } data;
    uint8_t Value;
} message_pointer_register_t;

typedef union
{
    struct
    {
        // Module type:
        // 1 - Master/autonomous module (rank 0,1, and 16)
        // 0 - Synchronous/slave module (rank 1 and 16)
        uint8_t module_type : 1;
        // Must be 0b001
        uint8_t VER : 3;
        // Maximum retries
        uint8_t max_retries : 4;
    } data;
    uint8_t Value;
} transmit_control_register_t;

typedef union
{
    struct
    {
        uint8_t ESDC : 1;
        uint8_t ETIP : 1;
        uint8_t Mb : 1;
        uint8_t Ma : 1;
        uint8_t SDC : 4;
    } data;
    uint8_t Value;
} diagnosis_control_register_t;

// Page 30
typedef union
{
    struct
    {
        uint8_t MSDC : 1;
        uint8_t ZERO : 2; // Must be 0
        // Rearbitrate -> Reset the retry counter and rearbitrate all messages
        uint8_t REAR : 1;
        // Activate command -> Page 51
        uint8_t ACTI : 1;
        // Idle command -> Page 51
        uint8_t IDLE : 1;
        // Sleep command -> puts the chip to sleep
        uint8_t SLEEP : 1;
        // General reset -> resets the chip
        uint8_t GRES : 1;
    } data;
    uint8_t Value;
} command_register_t;

// Page 31
```

```
typedef union
{
    struct
    {
        // Receiving status -> 1 when there is activity on the bus
        uint8_t RXG : 1;
        // Transmitting status -> TSS is transimitting a message
        uint8_t TXG : 1;
        // System diagnostics bits -> Table 8
        uint8_t Sa : 1;
        uint8_t Sb : 1;
        uint8_t Sc : 1;
        // Chip is in Idle state
        uint8_t IDG : 1;
        // Chip is in sleep state
        uint8_t SPG : 1;
        uint8_t : 1;
    } data;
    uint8_t Value;
} line_status_register_t;

// Page 32
typedef union
{
    struct
    {
        // Identifier currently transmitting
        uint8_t IDT : 4;
        // Number of retries done
        uint8_t NRT : 4;
    } data;
    uint8_t Value;
} transmission_status_register_t;

// Page 32
typedef union
{
    struct
    {
        // Identifier transmitted, received, or exceeded the retry count
        uint8_t IDTr : 4;
        // Number of retries done
        uint8_t NRTr : 4;
    } data;
    uint8_t Value;
} last_message_status_register_t;

// Page 32
typedef union
{
    struct
    {
        uint8_t frame_violation : 1;
        uint8_t code_violation : 1;
        uint8_t ack_error : 1;
        uint8_t crc_error : 1;
        uint8_t : 1;
        uint8_t buffer_overflow : 1;
```

```
        uint8_t buffer_occupied : 1;
        uint8_t : 1;
    } data;
    uint8_t Value;
} last_error_status_register_t;

#endif
```

Kod 19: include/van/tss463c.h

```
#ifndef TSS463C_H
#define TSS463C_H

#include <stdint.h>
#include "driver/spi_master.h"
#include "freertos/semphr.h"

#define TSS_CHANNEL_ADDR(x) (0x10 + (0x08 * x))
#define TSS_CHANNEL_MESS_PTR(x) TSS_CHANNEL_ADDR(x) + 2
#define TSS_CHANNEL_MESS_LEN_AND_STATUS(x) TSS_CHANNEL_ADDR(x) + 3
#define TSS_MAILBOX_OFFSET 0x80
#define TSS_GETMAIL(x) (TSS_MAILBOX_OFFSET + x)

#define TSS_8MHz_62k5BPS 0x30
#define TSS_8MHz_125kBPS 0x20

#define TSS_SELECT() gpio_set_level(instance->cs_pin, 0)
// #define TSS_SELECT()
#define TSS_DESELECT() gpio_set_level(instance->cs_pin, 1)
// #define TSS_DESELECT()


#define TSS_WRITE 0xE0
#define TSS_READ 0x60

#define TSS_MAX_CHANNEL 14
#define TSS_MAX_MEMORY_ADDR 0x7f

enum tss_message_type {
    TSS_NONE = 0,
    TSS_TRANSMIT,
    TSS_TRANSMIT_NOACK,
    TSS_REPLY_REQUEST,
    TSS_IMM_REPLY,
    TSS_DEF_REPLY,
};

enum tss_chip_mode {
    TSS_MODE_IDLE = 0,
    TSS_MODE_ACTIVE,
    TSS_MODE_SLEEP,
};

struct tss_channel_t
{
```

```
46      uint16_t identifier; // Ident registered to the channel
47      uint8_t ram_address; // The RAM address in use
48      uint8_t data_size;   // Memory size allocated
49      uint8_t message_type; // Message type, see enum tss_message_type
50      uint8_t is_occupied; // Is the channel registered?
51      uint8_t is_busy;     // Is the channel doing something?
52      uint8_t tx_attempt;  // Number of attempts to transmit said channel (starts at 1)
53      uint8_t message_len_and_status_reg_value;
54  };
55
56  typedef struct tss_channel_t tss_channel_t;
57
58  typedef struct
59  {
60      uint8_t start;
61      uint8_t end;
62  } tss_ringbuffer_t;
63
64  struct tss_instance_t
65  {
66      spi_device_handle_t tss_handle;
67      uint8_t *tx_buffer;
68      uint8_t *rx_buffer;
69      uint32_t buffers_size;
70      uint8_t cs_pin;
71      uint8_t interrupt_pin;
72      enum tss_chip_mode chip_mode;
73
74      SemaphoreHandle_t tss_spi_semaphore;
75      tss_ringbuffer_t tss_ringbuffer;
76      tss_channel_t channels[TSS_MAX_CHANNEL];
77  };
78
79  typedef struct tss_instance_t tss_instance_t;
80
81  typedef enum
82  {
83      TSS_EVENT_TRANSMIT_CHANNEL = 0, // Queue specified channel for transmission (first
        ↪  time)
84      TSS_EVENT_CHECK_CHANNEL, // Check on the channel's status
85      TSS_EVENT_RETRANSMIT_CHANNEL, // Retransmit specified channel
86      TSS_EVENT_CLEAN_CHANNEL, // Cleanup channel after transmission
87      TSS_EVENT_CANCEL_CHANNEL, // Abort sending channel
88
89      TSS_EVENT_MAX,
90  } tss_event_t;
91
92  typedef struct
93  {
94      tss_event_t event_type;
95      int channel_num;
96  } tss_event_command;
97
98
99  extern tss_instance_t global_tss_instance;
100
101 /* ============================================================ */
102
```

```
103 int tss_register_get(
104     tss_instance_t *const instance,
105     uint8_t const reg_addr,
106     uint8_t *const reg_value);
107
108 int tss_register_set(
109     tss_instance_t *instance,
110     uint8_t reg_addr,
111     uint8_t reg_value);
112
113 int tss_registers_set(
114     tss_instance_t *const instance,
115     uint8_t const reg_addr,
116     uint8_t const *const values,
117     uint8_t const count);
118
119 int tss_registers_get(
120     tss_instance_t *const instance,
121     uint8_t const reg_addr,
122     uint8_t *const buffer,
123     uint8_t count);
124
125 /* ================================================================== */
126
127 /**
128  * @brief Get the next available mailbox address to use for channel
129  *
130  *
131  * @param instance
132  * @param buf_size
133  * @param addr
134  * @return MIVE_OK or MIVE_ERR_OUTOFMEMORY or MIVE_ERR_INVALID_ARGUMENT
135  */
136 int tss_get_memory_address(
137     tss_instance_t* const instance,
138     uint8_t const channel_num,
139     uint8_t const buf_size,
140     uint8_t *const addr);
141
142 /* ================================================================== */
143
144 int tss_abort_channel(
145     tss_instance_t* const instance,
146     uint8_t const channel_num);
147
148 int tss_free_channel(
149     tss_instance_t* const instance,
150     uint8_t const channel_num);
151
152 int tss_retransmit_channel(
153     tss_instance_t* const instance,
154     uint8_t const channel_num);
155
156 /* ================================================================== */
157 /* Convinience functions to get certain register values */
158
159 uint8_t tss_get_channel_status_register(
160     tss_instance_t* const instance,
```

```
161     uint8_t const channel_num);
162
163 /* ================================================================== */
164 /* Only transmissions are handled by the TSS, the receiving is done */
165 /* by the RMT peripheral                                              */
166
167 int tss_transmit_message(
168     tss_instance_t* const instance,
169     uint8_t channel,
170     uint16_t const iden,
171     uint8_t *const data,
172     uint8_t const data_size,
173     uint8_t const memory_offset,
174     uint8_t const rak);
175
176 int tss_reply_request_message(
177     tss_instance_t* const instance,
178     uint8_t channel,
179     uint16_t const iden,
180     uint8_t const data_size,
181     uint8_t const memory_offset);
182
183 int tss_immediate_reply_message(
184     tss_instance_t* const instance,
185     uint8_t channel,
186     uint16_t const iden,
187     uint8_t *const data,
188     uint8_t const data_size,
189     uint8_t const memory_offset);
190
191 int tss_deferred_reply_message(
192     tss_instance_t* const instance,
193     uint8_t channel,
194     uint16_t const iden,
195     uint8_t *const data,
196     uint8_t const data_size,
197     uint8_t const memory_offset);
198
199 /* ================================================================== */
200
201 /**
202  * @brief Create and allocate resources for use with the TSS463C.
203  * Initializes the SPI driver.
204  *
205  * @param instance
206  * @return int
207  */
208 int tss_create(tss_instance_t* instance);
209
210 /**
211  * @brief Reset the TSS463C and put it in active mode.
212  *
213  * @param instance
214  * @return int
215  */
216 int tss_start(
217     tss_instance_t* const instance);
218
```

```
219 void tss_task(void *params);
220
221 /* ================================================================ */
222
223 int tss_activate(tss_instance_t *instance);
224 int tss_sleep(tss_instance_t *instance);
225 int tss_idle(tss_instance_t *instance);
226
227 #endif
```

Kod 20: include/van/van_packet_iden.h

```
1  #ifndef PSA_VAN_PACKET_IDEN_H
2  #define PSA_VAN_PACKET_IDEN_H
3
4  // VAN IDENtifiers
5
6  enum psa_van_iden
7  {
8      PSA_VAN_IDEN_VIN = 0xE24,
9      PSA_VAN_IDEN_ENGINE = 0x8A4,
10     PSA_VAN_IDEN_HEAD_UNIT_STALK = 0x9C4,
11     PSA_VAN_IDEN_LIGHTS_STATUS = 0x4FC,
12     PSA_VAN_IDEN_DEVICE_REPORT = 0x8C4,
13     PSA_VAN_IDEN_CAR_STATUS1 = 0x564,
14     PSA_VAN_IDEN_CAR_STATUS2 = 0x524,
15     PSA_VAN_IDEN_RPM = 0x824,
16     PSA_VAN_IDEN_DASHBOARD_BUTTONS = 0x664,
17     PSA_VAN_IDEN_HEAD_UNIT = 0x554,
18     PSA_VAN_IDEN_TIME = 0x984,
19     PSA_VAN_IDEN_AUDIO_SETTINGS = 0x4D4,
20     PSA_VAN_IDEN_MFD_STATUS = 0x5E4,
21     PSA_VAN_IDEN_AIRCON1 = 0x464,
22     PSA_VAN_IDEN_AIRCON2 = 0x4DC,
23     PSA_VAN_IDEN_CDCHANGER = 0x4EC,
24     PSA_VAN_IDEN_SATNAV_STATUS_1 = 0x54E,
25     PSA_VAN_IDEN_SATNAV_STATUS_2 = 0x7CE,
26     PSA_VAN_IDEN_SATNAV_STATUS_3 = 0x8CE,
27     PSA_VAN_IDEN_SATNAV_GUIDANCE_DATA = 0x9CE,
28     PSA_VAN_IDEN_SATNAV_GUIDANCE = 0x64E,
29     PSA_VAN_IDEN_SATNAV_REPORT = 0x6CE,
30     PSA_VAN_IDEN_MFD_TO_SATNAV = 0x94E,
31     PSA_VAN_IDEN_SATNAV_TO_MFD = 0x74E,
32     PSA_VAN_IDEN_SATNAV_DOWNLOADING = 0x6F4,
33     PSA_VAN_IDEN_SATNAV_DOWNLOADED1 = 0xA44,
34     PSA_VAN_IDEN_SATNAV_DOWNLOADED2 = 0xAC4,
35     PSA_VAN_IDEN_WHEEL_SPEED = 0x744,
36     PSA_VAN_IDEN_ODOMETER = 0x8FC,
37     PSA_VAN_IDEN_COM2000 = 0x450,
38     PSA_VAN_IDEN_CDCHANGER_COMMAND = 0x8EC,
39     PSA_VAN_IDEN_MFD_TO_HEAD_UNIT = 0x8D4,
40     PSA_VAN_IDEN_AIR_CONDITIONER_DIAG = 0xADC,
41     PSA_VAN_IDEN_AIR_CONDITIONER_DIAG_COMMAND = 0xA5C,
42     PSA_VAN_IDEN_ECU = 0xB0E,
43 };
```

```
44
45 #endif
```

Kod 21: include/van/van_packet_parser.h

```
1  #ifndef VAN_PACKET_PARSER_H
2  #define VAN_PACKET_PARSER_H
3
4  #include <stdint.h>
5
6  #include "../mive/psa_packet_defs.h"
7
8  struct psa_output_data_buffers
9  {
10     struct psa_engine_data *engine_data; // PSA_IDENT_ENGINE
11     struct psa_radio_data *radio_data; // PSA_IDENT_RADIO
12     struct psa_preset_data *presets_data_am; // PSA_IDENT_RADIO_PRESETS
13     struct psa_preset_data *presets_data_fm_1; // PSA_IDENT_RADIO_PRESETS
14     struct psa_preset_data *presets_data_fm_2; // PSA_IDENT_RADIO_PRESETS
15     struct psa_preset_data *presets_data_fm_ast; // PSA_IDENT_RADIO_PRESETS
16     struct psa_cd_player_data *cd_player_data; // PSA_IDENT_CD_PLAYER
17     struct psa_headunit_data *headunit_data; // PSA_IDENT_HEADUNIT
18     struct psa_door_data *door_data; // PSA_IDENT_DOORS
19     uint8_t *vin_data; // PSA_IDENT_VIN
20     struct psa_dash_data *dash_data; // PSA_IDENT_DASHBOARD
21     struct psa_trip_data *trip_data; // PSA_IDENT_TRIP
22     struct psa_status_data *status_data; // PSA_IDENT_CAR_STATUS
23 };
24
25 extern int psa_parse_van_packet(
26     uint16_t iden,
27     uint8_t size,
28     uint8_t const *const data,
29     struct psa_output_data_buffers *data_buffers);
30
31 #endif // VAN_PACKET_PARSER_H
```

Kod 22: include/van/van_rmt_rx.h

```
1  #ifndef VAN_RMT_RX_H
2  #define VAN_RMT_RX_H
3
4  #include <stdint.h>
5  #include <stdbool.h>
6
7  #include "driver/rmt_rx.h"
8  #include "freertos/queue.h"
9
10 typedef enum
11 {
12     RX_VAN_LINE_LEVEL_LOW = 0,
13     RX_VAN_LINE_LEVEL_HIGH = 1,
```

```
} RX_VAN_LINE_LEVEL;

typedef enum
{
    RX_VAN_NETWORK_BODY = 0,
    RX_VAN_NETWORK_COMFORT = 1,
} RX_VAN_NETWORK_TYPE;

typedef struct {
    int rmt_rx_ledPin;
    int rmt_rx_rxPin;
    uint8_t rmt_rx_channel;
    uint8_t rmt_rx_time_slice_divisor;
    RX_VAN_LINE_LEVEL rmt_rx_van_line_level;

    rmt_channel_handle_t rx_chan;
    rmt_receive_config_t rx_recv_config;
    QueueHandle_t receive_queue;
    rmt_symbol_word_t *rmt_symbol_buffer;
    size_t rmt_symbol_buffer_size;

} van_rmt_rx_instance_t;

void van_rmt_rx_channel_stop_new(van_rmt_rx_instance_t* instance);
void van_rmt_rx_channel_init_new(
    van_rmt_rx_instance_t* instance,
    uint8_t channel,
    int rxPin,
    int ledPin,
    RX_VAN_LINE_LEVEL vanLineLevel,
    RX_VAN_NETWORK_TYPE vanNetworkType);
void van_rmt_rx_channel_start_new(van_rmt_rx_instance_t* instance);

bool van_rmt_rx_parse_byte(van_rmt_rx_instance_t* instance, uint8_t level, uint32_t
↪  duration, uint8_t *bitCounter, uint8_t *tempByte, uint8_t *mask, uint8_t *finalByte);
uint16_t van_rmt_rx_crc15(uint8_t data[], uint8_t lengthOfData);
bool van_rmt_rx_is_crc_ok(uint8_t vanMessage[], int vanMessageLength);


#endif // VAN_RMT_RX_H
```

Kod 23: include/van_structs/van_car_status_structs.h

```
#ifndef PSA_VAN_CAR_STATUS_STRUCT_H
#define PSA_VAN_CAR_STATUS_STRUCT_H

#include <stdint.h>

// Iden 0x524

typedef struct {
    uint8_t tyre_pressure_low                     : 1; // bit 0
    uint8_t no_door_open                          : 1; // bit 1
    uint8_t automatic_gearbox_temperature_too_high : 1; // bit 2
    uint8_t brake_system_fault                    : 1; // bit 3
```

```
    uint8_t hydraulic_suspension_pressure_faulty   : 1; // bit 4
    uint8_t suspension_faulty                      : 1; // bit 5
    uint8_t engine_oil_temperature_too_high        : 1; // bit 6
    uint8_t engine_overheating                     : 1; // bit 7
} VanDisplayMsgV2Byte0Struct;

typedef struct {
    uint8_t unblock_diesel_filter     : 1; // bit 0
    uint8_t stop_car_icon             : 1; // bit 1
    uint8_t diesel_additive_too_low   : 1; // bit 2
    uint8_t fuel_tank_access_open     : 1; // bit 3
    uint8_t tyre_punctured            : 1; // bit 4
    uint8_t coolant_level_low         : 1; // bit 5
    uint8_t oil_pressure_too_low      : 1; // bit 6
    uint8_t oil_level_too_low         : 1; // bit 7
} VanDisplayMsgV2Byte1Struct;

typedef struct {
    uint8_t mil                          : 1; // bit 0  displays Antipollution fault
    uint8_t brake_pads_worn              : 1; // bit 1
    uint8_t diagnosis_ok                 : 1; // bit 2
    uint8_t automatic_gearbox_faulty     : 1; // bit 3
    uint8_t esp                          : 1; // bit 4  displays ESP faulty
    uint8_t abs                          : 1; // bit 5  displays ABS fault
    uint8_t suspension_or_steering_fault : 1; // bit 6
    uint8_t braking_system_faulty        : 1; // bit 7
} VanDisplayMsgV2Byte2Struct;

typedef struct {
    uint8_t side_airbag_faulty             : 1; // bit 0
    uint8_t airbags_faulty                 : 1; // bit 1
    uint8_t cruise_control_faulty          : 1; // bit 2
    uint8_t engine_temperature_too_high    : 1; // bit 3
    uint8_t fault_load_shedding_in_progress : 1; // bit 4
    uint8_t ambient_brightness_sensor_fault : 1; // bit 5
    uint8_t rain_sensor_fault              : 1; // bit 6
    uint8_t water_in_diesel_fuel_filter    : 1; // bit 7
} VanDisplayMsgV2Byte3Struct;

typedef struct {
    uint8_t left_rear_sliding_door_faulty  : 1; // bit 0
    uint8_t headlight_corrector_fault      : 1; // bit 1
    uint8_t right_rear_sliding_door_faulty : 1; // bit 2
    uint8_t no_broken_lamp                 : 1; // bit 3
    uint8_t battery_low                    : 1; // bit 4
    uint8_t battery_charge_fault           : 1; // bit 5
    uint8_t diesel_particle_filter_faulty  : 1; // bit 6
    uint8_t catalytic_converter_fault      : 1; // bit 7
} VanDisplayMsgV2Byte4Struct;

typedef struct {
    uint8_t handbrake                          : 1; // bit 0
    uint8_t seatbelt_warning                   : 1; // bit 1  lights the instrument
    ↪  cluster also
    uint8_t passenger_airbag_deactivated       : 1; // bit 2
    uint8_t screen_washer_liquid_level_too_low : 1; // bit 3
    uint8_t current_speed_too_high             : 1; // bit 4
    uint8_t ignition_key_left_in               : 1; // bit 5
```

```
    uint8_t sidelights_left_on                : 1; // bit 6
    uint8_t hill_holder_active                : 1; // bit 7  on RT3 monochrome: Driver's
    ↪  seatbelt not fastened
} VanDisplayMsgV2Byte5Struct;

typedef struct {
    uint8_t shock_sensor_faulty                 : 1; // bit 0
    uint8_t seatbelt_warning_                   : 1; // bit 1 not sure if it is seatbelt
    ↪  warning
    uint8_t check_and_reinitialize_tyre_pressure : 1; // bit 2
    uint8_t remote_control_battery_low          : 1; // bit 3
    uint8_t left_stick_button                   : 1; // bit 4
    uint8_t put_automatic_gearbox_in_P_position  : 1; // bit 5
    uint8_t stop_lights_test_brake_gently       : 1; // bit 6
    uint8_t fuel_level_low                      : 1; // bit 7  lights the instrument
    ↪  cluster also
} VanDisplayMsgV2Byte6Struct;

typedef struct {
    uint8_t automatic_headlamp_lighting_deactivated : 1; // bit 0
    uint8_t rear_lh_passenger_seatbelt_not_fastened : 1; // bit 1
    uint8_t rear_rh_passenger_seatbelt_not_fastened : 1; // bit 2
    uint8_t front_passenger_seatbelt_not_fastened   : 1; // bit 3
    uint8_t driving_school_pedals_indication        : 1; // bit 4
    uint8_t number_of_tpms_missing                  : 3; // bit 5-7  RT3 displays: X tyre
    ↪  pressure sensors missing where x equals this number
} VanDisplayMsgV2Byte7Struct;

typedef struct {
    uint8_t doors_locked                : 1; // bit 0
    uint8_t esp_asr_deactivated         : 1; // bit 1
    uint8_t child_safety_activated      : 1; // bit 2
    uint8_t deadlocking_active          : 1; // bit 3
    uint8_t automatic_lighting_active   : 1; // bit 4
    uint8_t automatic_wiping_active     : 1; // bit 5
    uint8_t engine_immobilizer_fault    : 1; // bit 6
    uint8_t sport_suspension_mode_active : 1; // bit 7
} VanDisplayMsgV2Byte8Struct;

typedef struct {
    uint8_t                             : 1; // bit 0
    uint8_t                             : 1; // bit 1
    uint8_t                             : 1; // bit 2
    uint8_t change_of_fuel_used_in_progress : 1; // bit 3
    uint8_t lpg_fuel_refused            : 1; // bit 4
    uint8_t lpg_system_faulty           : 1; // bit 5
    uint8_t lpg_in_use                  : 1; // bit 6
    uint8_t min_level_lpg               : 1; // bit 7
} VanDisplayMsgV2Byte10Struct;

typedef struct {
    uint8_t adin_fault                  : 1; // bit 0
    uint8_t use_stop_start              : 1; // bit 1
    uint8_t stop_start_available        : 1; // bit 2
    uint8_t stop_start_activated        : 1; // bit 3
    uint8_t stop_start_deactivated      : 1; // bit 4
    uint8_t stop_start_deferred         : 1; // bit 5
```

```
123     uint8_t unknown6                        : 1; // bit 6 displays empty messagebox on RT3
        ↪  monochrome when the Message byte is 0x5E
124     uint8_t unknown7                        : 1; // bit 7 displays empty messagebox on RT3
        ↪  monochrome when the Message byte is 0x5F
125 } VanDisplayMsgV2Byte11Struct;
126
127 typedef struct {
128     uint8_t                                 : 1; // bit 0
129     uint8_t                                 : 1; // bit 1
130     uint8_t                                 : 1; // bit 2
131     uint8_t                                 : 1; // bit 3
132     uint8_t                                 : 1; // bit 4
133     uint8_t                                 : 1; // bit 5
134     uint8_t                                 : 1; // bit 6
135     uint8_t change_to_neutral               : 1; // bit 7
136 } VanDisplayMsgV2Byte12Struct;
137
138
139 //Read left to right in documentation
140 struct psa_van_car_status_2_long {
141     VanDisplayMsgV2Byte0Struct Field0;
142     VanDisplayMsgV2Byte1Struct Field1;
143     VanDisplayMsgV2Byte2Struct Field2;
144     VanDisplayMsgV2Byte3Struct Field3;
145     VanDisplayMsgV2Byte4Struct Field4;
146     VanDisplayMsgV2Byte5Struct Field5;
147     VanDisplayMsgV2Byte6Struct Field6;
148     VanDisplayMsgV2Byte7Struct Field7;
149     VanDisplayMsgV2Byte8Struct Field8;
150     uint8_t Message;
151     VanDisplayMsgV2Byte10Struct Field10;
152     VanDisplayMsgV2Byte11Struct Field11;
153     VanDisplayMsgV2Byte12Struct Field12;
154     uint8_t Field13;
155     uint8_t Field14;
156     uint8_t Field15;
157 } __attribute__((packed));
158
159 struct psa_van_car_status_2_short {
160     VanDisplayMsgV2Byte0Struct Field0;
161     VanDisplayMsgV2Byte1Struct Field1;
162     VanDisplayMsgV2Byte2Struct Field2;
163     VanDisplayMsgV2Byte3Struct Field3;
164     VanDisplayMsgV2Byte4Struct Field4;
165     VanDisplayMsgV2Byte5Struct Field5;
166     VanDisplayMsgV2Byte6Struct Field6;
167     VanDisplayMsgV2Byte7Struct Field7;
168     VanDisplayMsgV2Byte8Struct Field8;
169     uint8_t Message;
170     VanDisplayMsgV2Byte10Struct Field10;
171     VanDisplayMsgV2Byte11Struct Field11;
172     VanDisplayMsgV2Byte12Struct Field12;
173     uint8_t Field13;
174 } __attribute__((packed));
175
176 #endif // PSA_VAN_CAR_STATUS_STRUCT_H
```

Kod 24: include/van_structs/van_cd_changer_commands_struct.h

```
#ifndef VAN_CD_CHANGER_COMMANDS_H
#define VAN_CD_CHANGER_COMMANDS_H

#include <stdint.h>

enum psa_van_cd_changer_commands
{
    PSA_VAN_CD_CHANGER_POWER_OFF = 0x1101,
    PSA_VAN_CD_CHANGER_POWER_OFF_2 = 0x2101,
    PSA_VAN_CD_CHANGER_PAUSE = 0x1181,
    PSA_VAN_CD_CHANGER_PLAY = 0x1183,
    PSA_VAN_CD_CHANGER_PREVIOUS_TRACK = 0x31FE,
    PSA_VAN_CD_CHANGER_NEXT_TRACK = 0X31FF,
    PSA_VAN_CD_CHANGER_CD_1 = 0X4101,
    PSA_VAN_CD_CHANGER_CD_2 = 0X4102,
    PSA_VAN_CD_CHANGER_CD_3 = 0X4103,
    PSA_VAN_CD_CHANGER_CD_4 = 0X4104,
    PSA_VAN_CD_CHANGER_CD_5 = 0X4105,
    PSA_VAN_CD_CHANGER_CD_6 = 0X4106,
    PSA_VAN_CD_CHANGER_PREVIOUS_CD = 0X41FE,
    PSA_VAN_CD_CHANGER_NEXT_CD = 0X41FF,
};

struct psa_van_cd_changer_command_struct
{
    uint16_t command; // Big endian
};

#endif
```

Kod 25: include/van_structs/van_radio_settings_struct.h

```
#ifndef PSA_VAN_RADIO_INFO_STRUCT_H
#define PSA_VAN_RADIO_INFO_STRUCT_H

#include <stdint.h>

// Radio audio settings 0x4D4 PSA_VAN_IDEN_AUDIO_SETTINGS

const uint8_t PSA_RADIO_INFO_SOURCE_RADIO = 0x51;
const uint8_t PSA_RADIO_INFO_SOURCE_CD = 0x52;

struct psa_radio_settings_byte_1
{
    uint8_t stalk_mute : 1;
    uint8_t external_mute : 1;
    uint8_t auto_volume : 1;
    uint8_t : 1;
    uint8_t loudness_on : 1;
    uint8_t audio_properties_menu_open : 1;
    uint8_t : 2;
} __attribute__((packed));
```

```
21
22  struct psa_radio_settings_byte_2
23  {
24      uint8_t power_on : 1;
25      uint8_t : 7;
26  } __attribute__((packed));
27
28  struct psa_radio_settings_byte_4
29  {
30      uint8_t source : 4;
31      uint8_t : 1;
32      uint8_t tape_present : 1;
33      uint8_t cd_present : 1;
34      uint8_t : 1;
35  } __attribute__((packed));
36
37  struct psa_radio_settings_values
38  {
39      uint8_t value : 7; // Add 0x3f to get actual value
40      uint8_t updating : 1;
41  } __attribute__((packed));
42
43  /**
44   * @brief Radio info struct. Ident 0x4D4
45   *
46   */
47  struct psa_van_radio_settings
48  {
49      uint8_t header;
50      struct psa_radio_settings_byte_1 audio_properties;
51      struct psa_radio_settings_byte_2 power;
52      uint8_t field3;
53      struct psa_radio_settings_byte_4 source;
54      struct psa_radio_settings_values volume;
55      struct psa_radio_settings_values balance;
56      struct psa_radio_settings_values fader;
57      struct psa_radio_settings_values bass;
58      struct psa_radio_settings_values treble;
59      uint8_t footer;
60  } __attribute__((packed));
61
62  int a = sizeof(struct psa_van_radio_settings);
63
64  #endif
```

Kod 26: include/van_structs/van_rpm_struct.h

```
1  #ifndef PSA_VAN_RPM_STRUCT_H
2  #define PSA_VAN_RPM_STRUCT_H
3
4  #include <stdint.h>
5
6  /**
7   * @brief VAN rpm and speed data struct. Iden 0x824
8   *
```

```
 */
struct psa_van_rpm_speed_struct
{

    uint16_t rpm;           // RPM in big endian. Divide by 8 to get actual rpm;
    uint16_t speed;         // Speed in big endian. Divide by 100 to get actual speed;
    uint16_t distance;      // Packet sequence in big endian. Increments with each passed
    ↪  decimeter
    uint8_t packet_changed; // Increments each time information significantly changes
} __attribute__((packed));

#endif
```

Kod 27: include/van_structs/van_cdc_emulator_structs.h

```
#ifndef VAN_CDC_EMU_H
#define VAN_CDC_EMU_H

#include <stdint.h>

enum van_cd_changer_status
{
    VAN_CDC_OFF = 0x41,
    VAN_CDC_ON_PAUSED = 0xC1,
    VAN_CDC_ON_BUSY = 0xD3,
    VAN_CDC_ON_PLAYING = 0xC3,
    VAN_CDC_ON_FAST_FORWARD = 0xC4,
    VAN_CDC_ON_FAST_REWIND = 0xC5,
};

enum van_cd_changer_cd_presence
{
    VAN_CDC_CD_PRESENT = 0x16,
    VAN_CDC_CD_NOT_PRESENT = 0x06,
};

struct van_cd_changer_packet
{
    uint8_t header;       // 0x80 to 0x87. Same as footer
    uint8_t shuffle;      // 1 if tracks are shuffled
    uint8_t status;       // enum van_cd_changer_status
    uint8_t cd_present;   // enum van_cd_changer_cd_presence
    uint8_t minutes;      // Track minute progress in bcd
    uint8_t seconds;      // Track seconds progress in the current minute in bcd
    uint8_t track_num;    // Number of the current track in bcd
    uint8_t cd_num;       // Number of the current CD in bcd
    uint8_t total_tracks; // Total number of tracks in bcd
    uint8_t unknown;
    uint8_t cd_flag; // bitwise representation of present CDs
    uint8_t footer;
} __attribute__((packed));

union van_cd_changer_union
{
    struct van_cd_changer_packet packet;
```

```
41      uint8_t buffer[sizeof(struct van_cd_changer_packet)];
42  };
43
44  #endif // VAN_CDC_EMU_H
```

Kod 28: include/van_structs/van_dash_struct.h

```
1   #ifndef PSA_VAN_DASHBOARD_STRUCT_H
2   #define PSA_VAN_DASHBOARD_STRUCT_H
3
4   #include <stdint.h>
5
6   /**
7    * @brief Dashboard struct. Ident 0x8A4. PSA_VAN_IDEN_ENGINE
8    *
9    */
10  struct psa_van_dashboard
11  {
12      uint8_t brightness : 4; // 0 - 15
13      uint8_t wiper : 1;
14      uint8_t : 2;
15      uint8_t is_backlight_off : 1; // 1 when light position is off
16
17      uint8_t accesories_on : 1;   // Key in ACC
18      uint8_t ignition_on : 1;     // Key in IGN
19      uint8_t engine_running : 1;  // Engine on
20      uint8_t door_open : 1;       // A door is open
21      uint8_t economy_mode : 1;    // Engine in economy mode
22      uint8_t reverse_gear : 1;    // Gearbox in reverse
23      uint8_t trailer_present : 1; // A trailer is hooked up
24      uint8_t : 1;
25
26      uint8_t water_temp;    // Subtract by 40
27      uint32_t mileage : 24; // Should be 3 bytes in big endian format, divide by 10 to get
        ↪  value
28      uint8_t external_temp; // 0xff if sensor not present (Subtract 0x50 and divide by 2
        ↪  to get celcius value)
29
30  } __attribute__((packed));
31
32  /**
33   * @brief Instrument cluster struct. Ident 0x4FC. Len 11. PSA_VAN_IDEN_LIGHTS_STATUS
34   *
35   */
36  struct psa_van_instrument_cluster_short
37  {
38      uint8_t cluster_lights_1;
39      uint8_t door_led;
40      uint16_t km_to_service;
41      uint8_t auto_gearbox_gear;
42      uint8_t light_indicators;
43      uint8_t oil_temp;
44      uint8_t fuel_level;
45      uint8_t oil_level; // 0 - 254
46      uint8_t unknown_1;
```

```
    uint8_t lpg_level;
} __attribute__((packed));

/**
 * @brief Instrument cluster struct. Ident 0x4FC. Len 14. PSA_VAN_IDEN_LIGHTS_STATUS
 *
 */
struct psa_van_instrument_cluster_long
{
    uint8_t cluster_lights_1;
    uint8_t door_led;
    uint16_t km_to_service;
    uint8_t auto_gearbox_gear;
    uint8_t light_indicators;
    uint8_t oil_temp;
    uint8_t fuel_level;
    uint8_t oil_level; // 0 - 254
    uint8_t unknown_1;
    uint8_t lpg_level;
    uint8_t cruise_control;
    uint8_t cruise_control_speed;
    uint8_t unknown_2;
} __attribute__((packed));

#endif
```

Kod 29: include/van_structs/van_radio_struct.h

```
#ifndef VAN_RADIO_STRUCTS_H
#define VAN_RADIO_STRUCTS_H

#include <stdint.h>

// Packet iden 0x554

enum psa_van_radio_band
{
    PSA_VAN_BAND_NONE = 0,
    PSA_VAN_BAND_FM1,
    PSA_VAN_BAND_FM2,
    PSA_VAN_BAND_FM3,
    PSA_VAN_BAND_FMAST,
    PSA_VAN_BAND_AM,
    PSA_VAN_BAND_PTY_SELECT,
};

struct psa_van_radio_freq_info_scan_info
{
    uint8_t : 3;
    uint8_t manual_scan_in_progress : 1;
    uint8_t freq_scan_is_running : 1;
    uint8_t : 2;
    uint8_t freq_scan_direction : 1;
} __attribute__((packed));

```

```
struct psa_van_radio_freq_info_ta_rds_flags
{
    uint8_t ta_active : 1;
    uint8_t rds_active : 1;
    uint8_t : 3;
    uint8_t rds_data_present : 1;
    uint8_t ta_data_present : 1;
    uint8_t : 1;
} __attribute__((packed));

// Len 22
struct psa_van_radio_freq_info
{
    uint8_t header;                                          // 0
    uint8_t data_type;                                       // Should be 0xD1 for
    ↪  frequency info // 1
    uint8_t band : 3;                                        // 2
    uint8_t memory_position : 5;                             // 2
    struct psa_van_radio_freq_info_scan_info scan_info;      // 3
    uint8_t frequency[2];                                    // 4,5
    uint8_t signal_info;                                     // 6 AND with 0x0F to get
    ↪  signal strength, not sure if accurate
    struct psa_van_radio_freq_info_ta_rds_flags rds_ta_flags; // 7
    uint8_t unknown2[4];                                     // 8,9,10,11
    uint8_t station[9];                                      //
    ↪  12,13,14,15,16,17,18,19,20
    uint8_t footer;
} __attribute__((packed));

// Len 10, didn't appear in my car
struct psa_van_radio_cd_info_len_10
{
    uint8_t header;
    uint8_t data_type; // Should be 0xD6 for CD track info
    uint8_t shuffle;
    uint8_t status; // loading, error, etc... to research
    uint8_t unknown1;
    uint8_t minutes;
    uint8_t seconds;
    uint8_t current_track;
    uint8_t total_tracks;
    uint8_t footer;
} __attribute__((packed));

// Len 19
struct psa_van_radio_cd_info_len_19
{
    uint8_t header;
    uint8_t data_type; // Should be 0xD6 for CD track info
    uint8_t shuffle;
    uint8_t status; // loading, error, etc... to research
    uint8_t unknown1;
    uint8_t minutes;
    uint8_t seconds;
    uint8_t current_track;
    uint8_t total_tracks;
    uint8_t total_minutes;
    uint8_t total_seconds;
```

```
    uint8_t unknown2[7];
    uint8_t footer;
} __attribute__((packed));

// Len 12, didn't appear in my car
struct psa_van_radio_preset_info
{
    uint8_t header;
    uint8_t data_type;       // Should be D3 for preset info
    uint8_t position : 4;    // Preset position
    uint8_t : 4;             // Unknown
    uint8_t station_name[8]; // ASCII, not NULL-terminated
    uint8_t footer;
} __attribute__((packed));

// int a = sizeof(struct psa_van_radio_preset_info);
#endif
```

Kod 30: include/van_structs/van_trip_computer_struct.h

```
#ifndef PSA_VAN_TRIP_COMPUTER_STRUCT_H
#define PSA_VAN_TRIP_COMPUTER_STRUCT_H

#include <stdint.h>

// IDEN 0x564 or PSA_VAN_IDEN_CAR_STATUS1

struct psa_van_trip_computer_door_struct
{
    uint8_t fuel_flap : 1;   // bit 0
    uint8_t sunroof : 1;     // bit 1
    uint8_t hood : 1;        // bit 2
    uint8_t boot_lid : 1;    // bit 3
    uint8_t rear_left : 1;   // bit 4
    uint8_t rear_right : 1;  // bit 5
    uint8_t front_left : 1;  // bit 6
    uint8_t front_right : 1; // bit 7
} __attribute__((packed));

union psa_van_trip_computer_door_union
{
    struct psa_van_trip_computer_door_struct unpacked;
    uint8_t packed;
};

struct psa_van_trip_computer_button_struct_maybe
{
    uint8_t trip_button : 1;
    uint8_t : 7;
} __attribute__((packed));

/**
 * @brief Trip computer info. Also includes door info. Iden 0x564
 *
 */
```

```
36 struct psa_van_trip_computer
37 {
38
39     uint8_t header;
40     uint8_t unknown1[6];
41
42     union psa_van_trip_computer_door_union door_status;
43
44     uint8_t unknown2[2];
45
46     struct psa_van_trip_computer_button_struct_maybe trip_button;
47
48     uint8_t trip_1_speed;
49     uint8_t trip_2_speed;
50     uint8_t unknown3;
51
52     uint16_t trip_1_distance;          // I'm guessing in KM
53     uint16_t trip_1_fuel_consumption;  // Says divide by 10 (probably in big endian)
54     uint16_t trip_2_distance;          // I'm guessing in KM
55     uint16_t trip_2_fuel_consumption;  // Says divide by 10 (probably in big endian)
56     uint16_t current_fuel_consumption; // Says divide by 10 (probably in big endian)
57     uint16_t distance_to_empty;        // I'm guessing in KM
58     uint8_t footer;
59
60 } __attribute__((packed));
61
62 #endif
```